Team Reference Document (Python 3 / PyPy)

Built from tested sources (`python3 run_tests.sh`). Sections: 00 Core: template, I/O, recursion, testing; 01 Techniques: search, greedy, offline, heuristics; 02 Data structures; 03 Dynamic programming; 04 Graphs; 05 Flows and matching; 06 Trees; 07 Strings; 08 Number theory, combinatorics, games; 09 Linear algebra and polynomials; 10 Geometry; Cheat sheet.

00 Core: template, I/O, recursion, testing

– 1.2 Choosing a language: Python performance notes

```
# - Submit with PyPy when available (5–50x faster on loops); CPython for heavy bigint / string builtins.
# - Rough budget: CPython ~1e7 simple ops/s, PyPy ~1e8. Push work into builtins
#   (sorted, sum, map, str.join, bytes.translate, list slicing) -- they run in C.
# - Local variables are faster than globals: put the solution inside main().
# - PyPy: lists of ints are unboxed and fast; avoid lists of tuples in hot loops (use parallel
#   arrays or pack (a<<20|b) into one int). Recursion is slow & stack-hungry on PyPy too.
# - PyPy: `sys.setrecursionlimit` big + deep recursion can segfault -> use bootstrap or iterative.
# - CPython: `x in set` O(1), but dict/set of tuples is slow & memory heavy -> encode as int.
# - Avoid `deepcopy`, `list.pop(0)` (O(n): use collections.deque), `str +=` in loops.
# - Fractions/Decimal are slow; prefer integer arithmetic (cross-multiply) where possible.
```

– 1.3 Complexity: constraints -> intended complexity

```
# n<=10: n!   n<=20: 2^n*n   n<=40: 2^(n/2) MITM   n<=500: n^3   n<=5000: n^2
# n<=1e5..2e5: n log n (n sqrt n risky in Python)   n<=1e6: O(n)   n<=1e18: log n / O(1)
# In Python divide those budgets by ~10 (CPython) or ~2 (PyPy) vs C++.
# Memory: Python int ~28 bytes, list slot 8 bytes -> 1e7 ints list ~ 80MB+ (use array('i') to cut).
```

– 1.4 Input/Output: contest template (fast input, fast output, multi-test)

```
import sys, os

def contest_template():
  # Read everything as bytes tokens; int(b'123') works directly. O(input size).
  data = sys.stdin.buffer.read().split()
  pos = 0
  def nxt():
    nonlocal pos
    pos += 1
    return data[pos - 1]
  out = []
  t = int(nxt())  # drop if single test case
  for _ in range(t):
    n = int(nxt())
    a = [int(nxt()) for _ in range(n)]
    s = nxt().decode()  # string token
    out.append(str(sum(a)))
  sys.stdout.write("\n".join(out) + "\n")
# Gotchas: read until EOF -> just loop `while pos < len(data)`. Line-based input
# (strings with spaces): lines = sys.stdin.buffer.read().decode().split('\n').
# Avoid input()/print() per line for >1e5 lines; if needed: input = sys.stdin.readline (keeps '\n').
# Bulk ints from one line fast: a = list(map(int, data[pos:pos+n])); pos += n

class FastIO:
  """Token reader over a bytes buffer. io=FastIO(); n=io.int(); a=io.ints(n)."""
  def __init__(self, buf=None):
    self.d = (buf if buf is not None else sys.stdin.buffer.read()).split()
    self.p = 0
  def tok(self):
    self.p += 1
    return self.d[self.p - 1]
  def int(self):
    self.p += 1
    return int(self.d[self.p - 1])
  def ints(self, n):
    self.p += n
    return list(map(int, self.d[self.p - n:self.p]))
  def str(self):
    return self.tok().decode()
  def eof(self):
    return self.p >= len(self.d)
# Output precision: f"{x:.10f}" or "%.10f" % x. Never print(x) raw for floats (may give 1e-05 form).
def fmt_float(x, k=10):
  s = "%.*f" % (k, x)
  return "0." + "0" * k if s.startswith("-") and float(s) == 0 else s  # avoid "-0.000"
# Exact rationals: fractions.Fraction; modular p/q -> p * pow(q, MOD-2, MOD) % MOD.
# Special judge (multiple answers): any valid answer accepted -- don't over-think tie-breaking.
```

– 1.4 Interactive problems

```
# Every query MUST be flushed; read the reply with readline (NOT the whole-buffer reader!).
def ask(*args, inp=None, outp=None):
  inp = inp or sys.stdin
  outp = outp or sys.stdout
  outp.write("? " + " ".join(map(str, args)) + "\n")
  outp.flush()
  r = inp.readline().split()
  if not r or r[0] == "-1":  # judge signals error: exit immediately (avoid TLE verdict)
    sys.exit(0)
  return list(map(int, r))

def answer(*args, outp=None):
  outp = outp or sys.stdout
  outp.write("! " + " ".join(map(str, args)) + "\n")
  outp.flush()
# Typical: binary search via queries -- count queries vs limit (ceil(log2 n)).
# Local test: write a judge script and connect with a FIFO:
#   mkfifo f; python3 judge.py < f | python3 sol.py > f
```

– 1.2 Recursion strategies

```
# Default recursion limit is 1000. Option A: raise limit + run in a thread with a big stack.
import threading

def run_with_big_stack(main, stack_mb=256, limit=10 ** 6):
  # Usage: run_with_big_stack(main). CPython only reliable; on PyPy prefer bootstrap/iterative.
  sys.setrecursionlimit(limit)
  threading.stack_size(stack_mb << 20)
  th = threading.Thread(target=main)
  th.start()
  th.join()
# Option B: generator trampoline. Recursive calls become `yield f(x)`, returns become `yield val`
# (or plain `return val` in py3). Depth limited only by memory. Works on PyPy.
from types import GeneratorType

def bootstrap(f, stack=[]):
  def wrappedfunc(*args, **kwargs):
    if stack:
      return f(*args, **kwargs)
    to = f(*args, **kwargs)
    while True:
      if type(to) is GeneratorType:
        stack.append(to)
        to = next(to)
      else:
        stack.pop()
        if not stack:
          break
        to = stack[-1].send(to)
    return to
  return wrappedfunc
# Example:
#   @bootstrap
#   def dfs(u, p):
#       s = 1
#       for v in g[u]:
#           if v != p: s += yield dfs(v, u)
#       yield s  # MUST yield (or return) at end, even if None: `yield None`
# Gotcha: every path of the function must end with `yield value`; a non-generator call breaks it.
# Option C (best): manual stack -- iterative DFS with order list, process reversed order for post-order.
```

– 4.3/19 Anti-hash protection (random salt for dict/set keys)

```
# Hack tests can make CPython/PyPy dicts degrade to O(n) per op with crafted int keys
# (and PyPy sets of ints). XOR keys with a random salt; same salt for insert & lookup.
import random
SALT = random.getrandbits(62) | 1

def salted_counter(arr):
  # O(n) expected. Returns dict keyed by x ^ SALT; look up with d.get(x ^ SALT, 0).
  d = {}
  for x in arr:
    k = x ^ SALT
    d[k] = d.get(k, 0) + 1
  return d
# Alternatives: str keys (string hash is randomized per process); or sort + groupby instead of dict.
# Counter/defaultdict have same weakness -- salt the keys before inserting.
```

– 14.3 Implementation skills

```
# - Wrap per-test logic in solve() returning the answer; build fresh arrays per test (no stale globals).
# - Allocate arrays sized by this test's n, not max-N, when sum of n is bounded but t is large
#   (t=1e4 tests * [0]*2e5 init = TLE).
# - Python ints are arbitrary precision: no overflow, but huge ints get slow -- reduce mod early.
# - Keep MOD = 998244353 / 10**9+7 as a constant; `%` the result of every multiplication in loops.
```

– 1.2/15 Python pitfalls

```
# - [[0]*m]*n aliases rows!  Use [[0]*m for _ in range(n)].
# - s += ch in a loop can be O(n^2); collect pieces in a list and "".join(parts).
# - 7 // 2 == 3, -7 // 2 == -4 (floors!), int(-7/2) == -3; -7 % 3 == 2. Use // for ints; / gives float.
# - Float loses precision past 2^53: int(1e18/3) is wrong; use integer //.  Use math.isqrt, not int(sqrt).
# - round() is banker's rounding: round(2.5) == 2. For half-up: int(x + 0.5) (x >= 0) or Decimal.
# - Default recursion limit 1000 -> RecursionError (RE); see Recursion strategies above.
# - sort(key=...) is fast; custom comparator needs functools.cmp_to_key (slow; prefer a key tuple).
#   Comparator must return negative/0/positive, not bool.
# - Default mutable args (def f(a=[])) persist across calls (bootstrap uses this on purpose).
# - Closures in loops capture the variable, not value: lambda x, i=i: ...
# - list.copy()/a[:] is shallow; nested lists need [row[:] for row in a].
# - max()/min() of empty sequence raises: use default=.
# - str.split() vs split(' '): the latter keeps empty strings for repeated spaces.
# - heapq is a min-heap; push -x for max-heap; tuples compare element-wise (ties compare next field!).
# - `is` vs `==`: never use `is` to compare ints/strings.
# - Modifying a list/dict while iterating over it -> bugs / RuntimeError.
# - input() strips '\n', sys.stdin.readline() does not -> .strip() / int() handles it.
# - Exponentiation: pow(a, b, m) is fast modular; a**b % m builds the huge number first.
```

– 15 Debugging: debug print, asserts

```
DEBUG = os.environ.get("LOCAL") is not None  # run locally with LOCAL=1 python3 sol.py

def dbg(*args):
  # Prints to stderr (judges ignore stderr, but it still costs time -- guarded by DEBUG).
  if DEBUG:
    print(*args, file=sys.stderr)
# assert cond, msg -- crashes with RE instead of WA; good for checking invariants. `python3 -O` disables.
# Profile: python3 -m cProfile -s cumtime sol.py < in.txt ; time: /usr/bin/time -v (max RSS = memory).
# TLE checklist: I/O (use buffer read + join), recursion -> iterative, tuples in hot loops,
#   attribute lookups in loops (bind local: ap = a.append), unnecessary copies/slices, CPython vs PyPy.
# MLE checklist: lists of lists/tuples -> flat arrays (idx = i*m + j), array('i'), del big temporaries.
# Edge cases: n=1, n=0/empty, all equal, sorted/reversed, max values, negatives, duplicates, t large.
```

– 15 Stress testing

```
def stress(solve, brute, gen, iters=1000, seed=0):
  # solve/brute: input -> output; gen(rng) -> random small input. Returns failing case or None.
  rng = random.Random(seed)
  for it in range(iters):
    inp = gen(rng)
    a, b = solve(inp), brute(inp)
    if a != b:
      print(f"MISMATCH at iter {it}\ninput: {inp!r}\nsolve: {a!r}\nbrute: {b!r}", file=sys.stderr)
      return inp
  return None
# Shell version (files sol.py, brute.py, gen.py taking a seed argument):
#   for i in $(seq 1 1000); do
#     python3 gen.py $i > in; python3 sol.py < in > o1; python3 brute.py < in > o2
#     cmp -s o1 o2 || { echo "diff on seed $i"; cat in; break; }
#   done
# Generators: keep sizes tiny (n<=8, values<=10) so failing cases are easy to read.
# Special judge: replace equality with a checker(inp, out) -> bool.
```

01 Techniques: search, greedy, offline, heuristics

```
import random, math, heapq
from bisect import bisect_right
from collections import deque
from itertools import combinations, permutations
```

– 3.1 Brute force: subsets / permutations

```
# Subsets of n items: for m in range(1 << n): items = [i for i in range(n) if m >> i & 1]
# itertools: combinations(a, k), permutations(a[, k]), product(range(3), repeat=n) (base-3 enum).
# Recursion-free k-subsets in increasing mask order: Gosper's hack (see 3.8).

def next_permutation(a):
  # O(n) in place, lexicographic next; returns False if a was the last (and leaves it). Handles dups.
  i = len(a) - 2
  while i >= 0 and a[i] >= a[i + 1]:
    i -= 1
  if i < 0:
    return False
  j = len(a) - 1
  while a[j] <= a[i]:
    j -= 1
  a[i], a[j] = a[j], a[i]
  a[i + 1:] = a[:i:-1]
  return True
```

– 3.1 Backtracking with pruning: N-queens count

```
def n_queens(n):
  # O(#solutions-ish), bitmask backtracking, depth n (fine recursively). n=12 -> 14200.
  full = (1 << n) - 1
  def go(cols, d1, d2):
    if cols == full:
      return 1
    cnt = 0
    free = full & ~(cols | d1 | d2)
    while free:
      b = free & -free
      free ^= b
      cnt += go(cols | b, ((d1 | b) << 1) & full, (d2 | b) >> 1)
    return cnt
  return go(0, 0, 0)
```

– 3.1 / 12.3 Meet in the middle: count subsets with sum <= S

```
def subset_sums(a):
  # O(2^n): all subset sums (list doubling, no recursion)
  s = [0]
  for x in a:
    s += [v + x for v in s]
  return s
```

```
def mitm_count_leq(a, S):
  # O(2^(n/2) * n). n <= 40. For "== S" use Counter on one half.
  h = len(a) // 2
  L, R = subset_sums(a[:h]), sorted(subset_sums(a[h:]))
  return sum(bisect_right(R, S - x) for x in L)
```

– 3.2 Grid problems: direction arrays

```
DIR4 = ((1, 0), (-1, 0), (0, 1), (0, -1))
DIR8 = ((1, 0), (-1, 0), (0, 1), (0, -1), (1, 1), (1, -1), (-1, 1), (-1, -1))
KNIGHT = ((1, 2), (2, 1), (-1, 2), (-2, 1), (1, -2), (2, -1), (-1, -2), (-2, -1))
# 'URDL' -> index: {c: i for i, c in enumerate('DURL')} matching DIR4 order (D=+row).
# Fast grid BFS: flatten id = r*W + c, pad grid with a wall border to skip bounds checks.

def grid_bfs(grid, sr, sc, wall="#"):
  # O(HW). Returns dist matrix (-1 unreachable).
  H, W = len(grid), len(grid[0])
  dist = [[-1] * W for _ in range(H)]
  dist[sr][sc] = 0
  q = deque([(sr, sc)])
  while q:
    r, c = q.popleft()
    for dr, dc in DIR4:
      nr, nc = r + dr, c + dc
      if 0 <= nr < H and 0 <= nc < W and dist[nr][nc] < 0 and grid[nr][nc] != wall:
        dist[nr][nc] = dist[r][c] + 1
        q.append((nr, nc))
  return dist
```

– 3.2 Date/time and calendar

```
# Leap: y%4==0 and (y%100!=0 or y%400==0). Gregorian cycle = 400 years = 146097 days = 20871 weeks.
# datetime.date(y,m,d) works for 1..9999: .weekday() (Mon=0), (d2-d1).days, date.fromordinal.
def is_leap(y):
  return y % 4 == 0 and (y % 100 != 0 or y % 400 == 0)

def days_in_month(y, m):
  return 29 if m == 2 and is_leap(y) else (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)[m - 1]

def day_of_week(y, m, d):
  # Sakamoto. 0 = Sunday ... 6 = Saturday. Gregorian.
  t = (0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4)
  if m < 3:
    y -= 1
  return (y + y // 4 - y // 100 + y // 400 + t[m - 1] + d) % 7

def days_from_civil(y, m, d):
  # Days since 1970-01-01 (any year, proleptic Gregorian). Diff of two = days between.
  y -= m <= 2
  era = y // 400
  yoe = y - era * 400
  doy = (153 * (m + (-3 if m > 2 else 9)) + 2) // 5 + d - 1
  doe = yoe * 365 + yoe // 4 - yoe // 100 + doy
  return era * 146097 + doe - 719468
# Time: convert hh:mm:ss to seconds, work mod 86400.
```

– 3.2 Parsing expressions (shunting-yard, iterative)

```
def eval_expr(s):
  # O(len). Ints, + - * /, parentheses, unary +/-. '/' truncates toward zero (C-style).
  # Add '^' (right-assoc): pop only while prec[top] > prec[c].
  prec = {"+": 1, "-": 1, "*": 2, "/": 2, "u": 3}
  nums, ops = [], []
  def apply():
    op = ops.pop()
    if op == "u":
      nums.append(-nums.pop())
      return
    b = nums.pop()
    a = nums.pop()
    if op == "+":
      nums.append(a + b)
    elif op == "-":
      nums.append(a - b)
    elif op == "*":
      nums.append(a * b)
    else:
      q = abs(a) // abs(b)
      nums.append(q if (a >= 0) == (b >= 0) else -q)
  i, n, prev = 0, len(s), False  # prev: last token was an operand
  while i < n:
    c = s[i]
    if c.isdigit():
      j = i
      while j < n and s[j].isdigit():
        j += 1
      nums.append(int(s[i:j]))
      i, prev = j, True
      continue
    if c == "(":
      ops.append(c)
      prev = False
    elif c == ")":
      while ops[-1] != "(":
        apply()
      ops.pop()
      prev = True
    elif c in prec:
      if not prev:  # unary (prefix ops never pop)
        if c == "-":
          ops.append("u")
      else:
        while ops and ops[-1] != "(" and prec[ops[-1]] >= prec[c]:
          apply()
        ops.append(c)
        prev = False
    i += 1
  while ops:
    apply()
  return nums[-1]
# Edge cases checklist (3.2): empty input, n=1, inclusive/exclusive bounds, 0- vs 1-indexing,
# overflow irrelevant in Python but mod arithmetic still needed; re-read output format.
```

– 3.3 Sorting: exchange arguments

```
# To order items so that cost is optimal, compare adjacent pair a,b: a before b iff
# cost(a,b) < cost(b,a). If it is a strict weak order, sort by key, e.g. concatenation
# "largest number": sort by cmp_to_key(lambda x, y: (y+x > x+y) - (y+x < x+y)).
# Weighted jobs min sum w*C: sort by t/w (key t_i*w_j < t_j*w_i -> use cmp_to_key or Fraction).
```

– 3.3 Coordinate compression

```
def compress(a):
  # O(n log n). Returns (ranks, sorted unique values). Value of rank r: vals[r].
  vals = sorted(set(a))
  idx = {v: i for i, v in enumerate(vals)}
  return [idx[x] for x in a], vals
# Lookups of non-present values: bisect_left(vals, x).
```

– 3.3 Counting sort / radix sort

```
def counting_sort(a, maxv):
  # O(n + maxv), values in [0, maxv].
  cnt = [0] * (maxv + 1)
  for x in a:
    cnt[x] += 1
  out = []
  for v in range(maxv + 1):
    if cnt[v]:
      out += [v] * cnt[v]
  return out

def radix_sort(a, bits=16):
  # O(n * w/bits), nonneg ints, stable LSD. Usually sorted() is faster in Python; useful in PyPy
  # for sorting pairs packed as ints: key = (x << 20) | i.
  if not a:
    return a
  mask = (1 << bits) - 1
  mx, sh = max(a), 0
  while (mx >> sh) or sh == 0:
    buckets = [[] for _ in range(mask + 1)]
    for x in a:
      buckets[(x >> sh) & mask].append(x)
    a = [x for b in buckets for x in b]
    sh += bits
  return a
```

– 3.3 Binary search on sorted arrays

```
# bisect_left(a, x): first idx with a[i] >= x; bisect_right: first a[i] > x.
# count in [lo, hi]: bisect_right(a, hi) - bisect_left(a, lo). Py3.10+: key= param.
```

– 3.3 Binary search on the answer (monotonic predicate)

```
def first_true(lo, hi, pred):
  # O(log(hi-lo)) calls. Smallest x in [lo, hi) with pred(x) true (pred F..FT..T); hi if none.
  while lo < hi:
    mid = (lo + hi) // 2
    if pred(mid):
      hi = mid
    else:
      lo = mid + 1
  return lo
# Last true for T..TF..F: first_true(lo, hi, lambda x: not pred(x)) - 1.

def bsearch_real(lo, hi, pred, iters=100):
  # Fixed iterations (no eps loop -> no infinite loop). Smallest x with pred(x) true. 100 iters
  # halves 2^100 times: enough for any double range. ~60 is enough for 1e18-range to 1e-9 rel.
  for _ in range(iters):
    mid = (lo + hi) / 2
    if pred(mid):
      hi = mid
    else:
      lo = mid
  return hi
```

– 3.3 Ternary search on unimodal functions

```
def ternary_int_max(lo, hi, f):
  # argmax of f on [lo, hi], f strictly increasing then strictly decreasing (plateaus break it!).
  while hi - lo > 2:
    m1 = lo + (hi - lo) // 3
    m2 = hi - (hi - lo) // 3
    if f(m1) < f(m2):
      lo = m1 + 1
    else:
      hi = m2
  return max(range(lo, hi + 1), key=f)
# Alternative for ints: first_true(lo, hi, lambda x: f(x) >= f(x+1)) -- binary search on slope.

def ternary_real_min(lo, hi, f, iters=200):
  # min of unimodal (convex) f on [lo, hi]; returns x. Each iter keeps 2/3.
  for _ in range(iters):
    m1 = lo + (hi - lo) / 3
    m2 = hi - (hi - lo) / 3
    if f(m1) < f(m2):
      hi = m2
    else:
      lo = m1
  return (lo + hi) / 2
```

– 3.3 / 11 Parallel binary search

```
def parallel_binary_search(Q, T, reset, apply, check):
  # For each query q: smallest t in [1, T] such that check(q) holds after apply(0..t-1); T+1 if never.
  # O((T + Q) log T) * cost(apply/check). reset() clears structure. check must be monotone in t.
  lo, hi = [1] * Q, [T + 1] * Q
  while True:
    act = [q for q in range(Q) if lo[q] < hi[q]]
    if not act:
      return lo
    act.sort(key=lambda q: (lo[q] + hi[q]) // 2)
    reset()
    t = 0
    for q in act:
      mid = (lo[q] + hi[q]) // 2
      while t < mid:
        apply(t)
        t += 1
      if check(q):
        hi[q] = mid
      else:
        lo[q] = mid + 1
```

– 3.4 Prefix sums 1D / 2D

```
# 1D: P = [0]; for x in a: P.append(P[-1] + x)  -> sum a[l..r) = P[r] - P[l]. itertools.accumulate(a, initial=0).
# Prefix XOR: same with ^ (xor of [l,r) = X[r]^X[l]). Prefix products: only invertible (mod p, no zeros)
# -> count zeros separately; or use sparse table / segment tree.
def prefix2d(g):
  # O(HW). P[i][j] = sum g[0..i)[0..j). rect sum [r1,r2) x [c1,c2): see rect().
  H, W = len(g), len(g[0])
  P = [[0] * (W + 1) for _ in range(H + 1)]
  for i in range(H):
    row, pr, cur = g[i], P[i], P[i + 1]
    s = 0
    for j in range(W):
      s += row[j]
      cur[j + 1] = pr[j + 1] + s
  return P

def rect(P, r1, c1, r2, c2):
  return P[r2][c2] - P[r1][c2] - P[r2][c1] + P[r1][c1]
```

– 3.4 Difference arrays (1D / 2D range add)

```
# 1D: D=[0]*(n+1); add v on [l,r): D[l]+=v; D[r]-=v; then a = prefix sums of D.
def diff2d_apply(H, W, updates):
  # updates: (r1, c1, r2, c2, v) half-open rectangles. O(HW + U).
  D = [[0] * (W + 1) for _ in range(H + 1)]
  for r1, c1, r2, c2, v in updates:
    D[r1][c1] += v
    D[r1][c2] -= v
    D[r2][c1] -= v
    D[r2][c2] += v
  for i in range(H + 1):
    for j in range(1, W + 1):
      D[i][j] += D[i][j - 1]
  for i in range(1, H + 1):
    for j in range(W + 1):
      D[i][j] += D[i - 1][j]
  return [row[:W] for row in D[:H]]
```

– 3.4 Higher-order difference: add polynomial to ranges

```
def add_polys(n, updates, deg):
  # updates: (l, r, p): a[i] += p(i - l) for l <= i <= r; p: int -> int, degree <= deg.
  # O(n*K + U*K^2), K = deg+1. AP: deg=1, p = lambda x: a0 + d*x.
  K = deg + 1
  sgn = [math.comb(K, j) * (-1) ** j for j in range(K)]
  D = [0] * (n + 2 * K + 1)
  def put(start, f, s):
    vals = [f(x) for x in range(K)]
    for j in range(K):
      D[start + j] += s * sum(sgn[j - i] * vals[i] for i in range(j + 1))
  for l, r, p in updates:
    put(l, p, 1)
    sh = r + 1 - l
    put(r + 1, lambda x, p=p, sh=sh: p(x + sh), -1)
  for _ in range(K):  # K prefix-sum passes undo K-th difference
    s = 0
    for i in range(len(D)):
      s += D[i]
      D[i] = s
  return D[:n]
```

– 3.5 Two pointers on sorted arrays

```
def count_pairs_leq(a, S):
  # a sorted. #pairs i<j with a[i]+a[j] <= S. O(n).
  i, j, cnt = 0, len(a) - 1, 0
  while i < j:
    if a[i] + a[j] <= S:
      cnt += j - i
      i += 1
    else:
      j -= 1
  return cnt
```

– 3.5 Variable-size sliding window

```
def longest_at_most_k_distinct(a, k):
  # O(n). Longest subarray with <= k distinct values. Pattern: extend r, shrink l while invalid.
  cnt, l, best, distinct = {}, 0, 0, 0
  for r, x in enumerate(a):
    c = cnt.get(x, 0)
    if c == 0:
      distinct += 1
    cnt[x] = c + 1
    while distinct > k:
      y = a[l]
      cnt[y] -= 1
      if cnt[y] == 0:
        distinct -= 1
      l += 1
```

```
    best = max(best, r - l + 1)
  return best
```

– 3.5 Sliding window min/max (monotonic deque)

```
def sliding_min(a, k):
  # O(n). min of each window a[i-k+1..i] for i >= k-1. For max
  ↪  flip comparison (<=  -> >=).
  dq, out = deque(), []
  for i, x in enumerate(a):
    while dq and a[dq[-1]] >= x:
      dq.pop()
    dq.append(i)
    if dq[0] <= i - k:
      dq.popleft()
    if i >= k - 1:
      out.append(a[dq[0]])
  return out
```

– 3.5 Fast and slow pointers (Floyd cycle detection)

```
def floyd(f, x0):
  # O(mu + lam) time, O(1) memory. Returns (mu, lam): first
  ↪  index on cycle, cycle length.
  t, h = f(x0), f(f(x0))
  while t != h:
    t, h = f(t), f(f(h))
  mu, t = 0, x0
  while t != h:
    t, h = f(t), f(h)
    mu += 1
  lam, h = 1, f(t)
  while t != h:
    h = f(h)
    lam += 1
  return mu, lam
```

– 3.6 Greedy: proving correctness / matroids

```
# Prove by exchange argument (swap an optimal solution's choice
↪  with greedy's, never worse) or
# "greedy stays ahead" induction. Greedy max-weight independent
↪  set is optimal iff the family is a
# matroid (hereditary + exchange): e.g. forests (Kruskal),
↪  linearly independent vectors (xor basis),
# scheduling unit jobs with deadlines. Matroid intersection (2
↪  matroids) -> see 05_flow_matching.py.
```

– 3.6 Interval scheduling (max non-overlapping)

```
def max_disjoint(iv):
  # half-open [l, r). Sort by right end. O(n log n).
  last, cnt = -float("inf"), 0
  for l, r in sorted(iv, key=lambda x: x[1]):
    if l >= last:
      cnt += 1
      last = r
  return cnt
```

– 3.6 Interval covering (min intervals to cover [L, R])

```
def min_cover(iv, L, R):
  # closed intervals; returns min count or -1. O(n log n).
  iv = sorted(iv)
  i, n, cur, cnt = 0, len(iv), L, 0
  while True:
    best = None
    while i < n and iv[i][0] <= cur:
      if best is None or iv[i][1] > best:
        best = iv[i][1]
      i += 1
    if best is None or best < cur:
      return -1
    cnt += 1
    if best >= R:
      return cnt
    if best == cur:  # can't extend (only zero-progress
    ↪  intervals)
      return -1
    cur = best
```

– 3.6 Interval partitioning (min rooms, heap)

```
def min_rooms(iv):
  # half-open [l, r). Returns (k, room id per interval). O(n log
  ↪  n). k = max overlap.
  order = sorted(range(len(iv)), key=lambda i: iv[i][0])
  heap, free, room, k = [], [], [0] * len(iv), 0
  for i in order:
    l, r = iv[i]
    while heap and heap[0][0] <= l:
      heapq.heappush(free, heapq.heappop(heap)[1])
    if free:
      rid = heapq.heappop(free)
    else:
      rid, k = k, k + 1
    room[i] = rid
    heapq.heappush(heap, (r, rid))
  return k, room
```

– 3.6 Job sequencing with deadlines (unit time)

```
def job_sequencing(jobs):
  # jobs: (deadline, profit); each takes 1 unit, must finish by
  ↪  deadline. Max profit. O(n log n).
  h = []
  for d, p in sorted(jobs):
    heapq.heappush(h, p)
    if len(h) > d:
      heapq.heappop(h)  # drop least profitable
  return sum(h)
# Variant with durations (max #jobs, deadlines): sort by
↪  deadline, keep max-heap of durations,
# pop the longest when total time exceeds deadline
↪  (Moore-Hodgson).
```

– 3.6 Huffman coding (optimal merge cost)

```
def huffman_cost(w):
  # O(n log n). Sum of merge costs = total weighted code length.
  h = list(w)
  heapq.heapify(h)
  cost = 0
  while len(h) > 1:
    a = heapq.heappop(h)
    b = heapq.heappop(h)
    cost += a + b
    heapq.heappush(h, a + b)
  return cost
# k-ary: pad with zeros until (n-1) % (k-1) == 0, merge k at a
↪  time.
```

– 3.6 Regret greedy (greedy with undo)

```
def stock_max_profit(prices):
  # Each day: buy 1, sell 1, or nothing (CF 865D). O(n log n).
  ↪  Push p twice: one is the
  # "undo" option (selling at p now can later be revoked and
  ↪  resold at higher q, gain q-p).
  h, ans = [], 0
  for p in prices:
    if h and h[0] < p:
      ans += p - heapq.heappop(h)
      heapq.heappush(h, p)
    heapq.heappush(h, p)
  return ans
# General pattern: when taking an item, push a compensating
↪  "regret" item (value -gain) so a later
# choice can undo it. Also: choose k non-adjacent max sum
↪  (linked list + heap, merge neighbors a+c-b).
```

– 3.7 Merge sort and counting inversions

```
def count_inversions(a):
  # O(n log n), iterative bottom-up. #pairs i<j with a[i] >
  ↪  a[j]. (Alt: BIT over compressed values.)
  a = list(a)
  n, inv, w = len(a), 0, 1
  buf = [0] * n
  while w < n:
    for lo in range(0, n, 2 * w):
      mid, hi = min(lo + w, n), min(lo + 2 * w, n)
      i, j, k = lo, mid, lo
      while i < mid and j < hi:
        if a[j] < a[i]:
          buf[k] = a[j]
          inv += mid - i
          j += 1
        else:
          buf[k] = a[i]
          i += 1
        k += 1
      buf[k:hi] = a[i:mid] + a[j:hi]
    a, buf = buf, a
    w *= 2
  return inv
```

– 3.7 Quickselect

```
def quickselect(a, k, rng=random):
  # k-th smallest (0-based). Expected O(n). 3-way partition
  ↪  handles duplicates.
  a = list(a)
  while True:
    p = a[rng.randrange(len(a))]
    lt = [x for x in a if x < p]
    if k < len(lt):
      a = lt
      continue
    eq = sum(1 for x in a if x == p)
    if k < len(lt) + eq:
      return p
    k -= len(lt) + eq
    a = [x for x in a if x > p]
# In practice: sorted(a)[k] or heapq.nsmallest(k+1, a)[-1] is
↪  often faster in CPython.
```

– 3.7 Closest pair

```
# see 10_geometry.py: closest pair
```

– 3.7 Master theorem / Karatsuba

```
# T(n) = a T(n/b) + O(n^d): d > log_b a -> O(n^d); d == log_b a
↪  -> O(n^d log n); d < -> O(n^log_b a).
# Examples: merge sort 2T(n/2)+n = n log n; binary search
↪  T(n/2)+1 = log n; Karatsuba 3T(n/2)+n = n^1.585.
# Karatsuba: Python ints already use it for big multiplication
↪  -- just multiply ints. For polynomial
# multiplication pack coefficients into one big int (Kronecker)
↪  or use NTT: see 09_algebra_poly.py.
```

– 3.8 Bit manipulation

```
# lowbit: x & -x; clear lowest: x & (x-1); test: x >> i & 1;
↪  set: x | 1<<i; clear: x & ~(1<<i);
# toggle: x ^ 1<<i; popcount: x.bit_count() (3.10+) or
↪  bin(x).count("1"); highest bit: x.bit_length()-1;
# trailing zeros: (x & -x).bit_length() - 1; is power of 2: x
↪  and not x & (x-1); all n bits: (1<<n)-1.
# Python ints are unbounded: ~x is negative (-x-1) -> mask with
↪  & ((1<<n)-1).
def submasks(m):
  # all submasks of m incl. 0, decreasing. Over all m: total
  ↪  O(3^n).
  s = m
  while True:
    yield s
    if s == 0:
      return
    s = (s - 1) & m

def supersets(m, n):
  # all supersets of m within n bits, increasing.
  s = m
  full = 1 << n
  while s < full:
    yield s
    s = (s + 1) | m

def k_subsets(n, k):
  # Gosper's hack: all n-bit masks with k bits set, increasing.
  ↪  k >= 1.
  s = (1 << k) - 1
  while s < 1 << n:
    yield s
    c = s & -s
    r = s + c
    s = (((r ^ s) >> 2) // c) | r

def gray(i):
  return i ^ (i >> 1)

def gray_inv(g):
  i = 0
  while g:
    i ^= g
    g >>= 1
  return i
# Gray: consecutive codes differ in exactly 1 bit (bit =
↪  trailing zeros of i+1) -> enumerate subsets
# updating state by one add/remove per step.
# XOR facts: a^a=0, xor is its own inverse (prefix xor), xor of
↪  0..n = [n,1,n+1,0][n%4];
# a+b = (a^b) + 2(a&b); a^b <= a+b; min pairwise xor of a set is
↪  among adjacent elements after sorting;
# max xor pair / subset -> binary trie or linear basis (see
↪  07_strings.py: BinaryTrie / 09_algebra_poly.py).
# SOS DP (sum over subsets) -> see 03_dp.py.
```

– 11 Offline queries: overview

```
# Offline: read all queries, reorder (sort by r, by value, by
↪  time), answer, output in original order.
# Sweep line + BIT: sort queries by right end, add elements as r
↪  advances (e.g. distinct count:
#   keep only last occurrence of each value in BIT).
# Persistent structures (online version of the sweep) -> see
↪  02_data_structures.py.
# Answering in reverse: deletions are hard, insertions easy ->
↪  process ops backwards
#   (deleting edges -> adding edges with DSU), then reverse the
↪  answers.
# Sqrt decomposition on queries: buffer up to ~sqrt(q) pending
↪  updates, answer queries with the
#   static structure + brute over the buffer, rebuild when
↪  buffer is full.
# Segment tree over time (offline dynamic connectivity, DSU
↪  with rollback) -> see 04_graphs.py.
```

– 11 Mo's algorithm (Hilbert order)

```
def hilbert_order(x, y, k=21):
  # Position on Hilbert curve of order k (coords < 2^k). Sorting
  ↪  by this beats block order.
  d, s, n = 0, 1 << (k - 1), 1 << k
  while s:
    rx = 1 if x & s else 0
    ry = 1 if y & s else 0
    d += s * s * ((3 * rx) ^ ry)
    if ry == 0:
      if rx == 1:
        x, y = n - 1 - x, n - 1 - y
      x, y = y, x
    s >>= 1
  return d

def mo_distinct(a, queries):
  # O((n + q) sqrt n). Distinct count on inclusive [l, r].
  ↪  Template: replace add/remove bodies.
  # Classic order alternative: key (l // B, r if (l//B) even
  ↪  else -r), B = n / sqrt(q).
  vals, _ = compress(a)
  cnt = [0] * (len(a) + 1)
  order = sorted(range(len(queries)), key=lambda i:
↪  hilbert_order(*queries[i]))
  ans = [0] * len(queries)
  L, R, cur = 0, -1, 0
  for qi in order:
    l, r = queries[qi]
    while R < r:
      R += 1
      v = vals[R]; cnt[v] += 1
      if cnt[v] == 1: cur += 1
    while L > l:
      L -= 1
      v = vals[L]; cnt[v] += 1
      if cnt[v] == 1: cur += 1
    while R > r:
      v = vals[R]; cnt[v] -= 1
      if cnt[v] == 0: cur -= 1
      R -= 1
    while L < l:
      v = vals[L]; cnt[v] -= 1
      if cnt[v] == 0: cur -= 1
      L += 1
    ans[qi] = cur
  return ans
```

– 11 Mo's algorithm with updates

```
def mo_updates_distinct(a, ops):
  # ops: ('Q', l, r) inclusive or ('U', pos, val). O(n^(5/3)).
  ↪  Block B ~ n^(2/3).
  # Sort queries by (l//B, r//B, t); move time pointer
  ↪  applying/undoing updates by swapping values.
  a = list(a)
  allv = list(a) + [o[2] for o in ops if o[0] == "U"]
  _, vs = compress(allv)
  idx = {v: i for i, v in enumerate(vs)}
  a = [idx[x] for x in a]
  ups, qs = [], []
  for o in ops:
    if o[0] == "U":
      ups.append([o[1], idx[o[2]]])
    else:
      qs.append((o[1], o[2], len(ups), len(qs)))
  n = len(a)
  B = max(1, round(n ** (2 / 3)))
  qs.sort(key=lambda q: (q[0] // B, q[1] // B, q[2]))
  cnt = [0] * len(vs)
  ans = [0] * len(qs)
  L, R, T, cur = 0, -1, 0, 0
  for l, r, t, qi in qs:
    while T < t or T > t:
      if T < t:
        p = ups[T]; T += 1
```

```
      else:
        T -= 1; p = ups[T]
      pos, v = p
      if L <= pos <= R:
        cnt[a[pos]] -= 1
        if cnt[a[pos]] == 0: cur -= 1
        cnt[v] += 1
        if cnt[v] == 1: cur += 1
      a[pos], p[1] = v, a[pos]  # swap: applying again undoes it
    while R < r:
      R += 1; cnt[a[R]] += 1
      if cnt[a[R]] == 1: cur += 1
    while L > l:
      L -= 1; cnt[a[L]] += 1
      if cnt[a[L]] == 1: cur += 1
    while R > r:
      cnt[a[R]] -= 1
      if cnt[a[R]] == 0: cur -= 1
      R -= 1
    while L < l:
      cnt[a[L]] -= 1
      if cnt[a[L]] == 0: cur -= 1
      L += 1
    ans[qi] = cur
  return ans
```

– 11 CDQ divide and conquer (3D partial order)

```
def cdq_3d(pts):
  # For each i: #j != i with a_j<=a_i, b_j<=b_i, c_j<=c_i. O(n
  ↪  log^2 n). Recursion depth log n.
  # Idea: split by a; left half contributes to right half via
  ↪  sweep on b + BIT on c.
  uniq = sorted(set(pts))
  from collections import Counter
  mult = Counter(pts)
  m = len(uniq)
  w = [mult[p] for p in uniq]
  _, cs = compress([p[2] for p in uniq])
  cidx = {v: i + 1 for i, v in enumerate(cs)}
  C = [cidx[p[2]] for p in uniq]
  Bv = [p[1] for p in uniq]
  res = [0] * m
  bit = [0] * (len(cs) + 1)
  K = len(cs)
  def solve(lo, hi):
    if hi - lo <= 1:
      return
    mid = (lo + hi) // 2
    solve(lo, mid)
    solve(mid, hi)
    left = sorted(range(lo, mid), key=lambda i: Bv[i])
    right = sorted(range(mid, hi), key=lambda i: Bv[i])
    p = 0
    for i in right:
      while p < len(left) and Bv[left[p]] <= Bv[i]:
        j = C[left[p]]
        while j <= K:
          bit[j] += w[left[p]]; j += j & -j
        p += 1
      j, s = C[i], 0
      while j:
        s += bit[j]; j -= j & -j
      res[i] += s
    for q in range(p):  # undo BIT
      j = C[left[q]]
      while j <= K:
        bit[j] -= w[left[q]]; j += j & -j
  solve(0, m)
  pos = {p: i for i, p in enumerate(uniq)}
  return [res[pos[p]] + w[pos[p]] - 1 for p in pts]
# Uniques sorted by (a,b,c) => any j dominating-before i with
↪  equal a lies left of i. Duplicates merged.
# CDQ also: offline dynamic problems where updates in left half
↪  affect queries in right half.
```

– 12.1 Sqrt techniques

```
# Sqrt decomposition on arrays: blocks of size B~sqrt(n), keep
↪  per-block aggregate (+ lazy tag);
#   query = partial blocks brute + full blocks aggregate. (Block
↪  structures: see 02_data_structures.py.)
# Heavy/light by degree: vertices with deg > sqrt(m) are heavy
↪  (<= 2 sqrt(m) of them); handle heavy
#   specially (precompute / push updates to them), light ones by
↪  brute over neighbors.
#   Triangle counting: orient edges low->high (deg, id), O(m
↪  sqrt m).
# By frequency: values occurring > sqrt(n) times are few ->
↪  precompute per heavy value.
# Python: sqrt algorithms are often too slow in CPython; prefer
↪  PyPy and flat lists.
```

– 12.2 Randomization

```
# Las Vegas (always correct, random time:
↪  quicksort/quickselect) vs Monte Carlo (fixed time, may err:
#   Miller-Rabin, hashing, Freivalds' matrix check A*B==C in
↪  O(n^2) per trial).
# Shuffle input (random.shuffle) to defeat anti-quicksort /
↪  anti-incremental tests (e.g. randomized
#   incremental min enclosing circle -> see 10_geometry.py).
# Hashing with random base/mod (string hashing ->
↪  07_strings.py); dict keys salt -> 00_core.py.
# Random sampling: element occurring > n/2 times: pick random,
↪  check (P(fail) 2^-k after k tries).
# Sum/xor hashing of sets: assign random 64-bit value per
↪  element; multiset equality = sum/xor equality.
# Probabilistic counting: estimate #distinct by min hash.
# Color coding: find path of k distinct vertices: color
↪  randomly with k colors, DP over (v, colorset)
#   O(2^k m); success prob k!/k^k ~ e^-k -> repeat ~e^k times.
```

– 12.2 / 13.4 Simulated annealing / hill climbing

```
def simulated_annealing(init, cost, neighbor, iters, t0, t1,
↪  rng):
  # Minimize cost. neighbor(state, rng) -> new state (don't
  ↪  mutate input). Geometric cooling t0 -> t1.
  # Contest tips: time-based schedule (time.perf_counter), O(1)
  ↪  delta evaluation instead of full cost,
  # t0 ~ typical |delta| at start, t1 ~ smallest meaningful
  ↪  delta.
  cur, cc = init, cost(init)
  best, bc = cur, cc
  for i in range(iters):
    T = t0 * (t1 / t0) ** (i / iters)
    nxt = neighbor(cur, rng)
    nc = cost(nxt)
    if nc <= cc or rng.random() < math.exp((cc - nc) / T):
      cur, cc = nxt, nc
      if cc < bc:
        best, bc = cur, cc
  return best, bc

def hill_climb(init, cost, neighbor, iters, rng, restarts=1,
↪  rand_init=None):
  # Accept only non-worsening moves; random restarts via
  ↪  rand_init(rng).
  best, bc = None, float("inf")
  for r in range(restarts):
    cur = init if r == 0 or rand_init is None else rand_init(rng)
    cc = cost(cur)
    for _ in range(iters):
      nxt = neighbor(cur, rng)
      nc = cost(nxt)
      if nc <= cc:
        cur, cc = nxt, nc
    if cc < bc:
      best, bc = cur, cc
  return best, bc
# Beam search: keep the best W partial states per depth, expand
↪  each, keep top W (heapq.nlargest).
# Genetic algorithms: rarely worth it in contests; SA usually
↪  beats them.
```

– 12.3 Iterative deepening (IDDFS / IDA*)

```
def ida_star(start, is_goal, nbrs, h, limit=10 ** 9):
  # Returns optimal cost or -1. Memory O(depth). h admissible.
  ↪  nbrs(u) -> iterable of (v, w).
  # Recursion depth = solution length (small by nature). Order
  ↪  nbrs by h for speed.
  path, found = {start}, [None]
  def dfs(u, g, bound):
    f = g + h(u)
    if f > bound:
      return f
    if is_goal(u):
      found[0] = g
      return -1
    mn = float("inf")
    for v, w in nbrs(u):
      if v in path:
        continue
      path.add(v)
      t = dfs(v, g + w, bound)
      path.discard(v)
      if t == -1:
        return -1
      mn = min(mn, t)
    return mn
  bound = h(start)
  while bound <= limit:
    t = dfs(start, 0, bound)
    if t == -1:
      return found[0]
    if t == float("inf"):
      return -1
    bound = t
  return -1
# IDDFS = IDA* with h = 0: DFS with depth limit 0,1,2,...
↪  (BFS-optimal, DFS memory).
```

– 12.3 A* search

```
def astar(start, goal, nbrs, h):
  # Dijkstra with priority g + h(u). h consistent (h(u) <= w +
  ↪  h(v)) => first pop of goal optimal.
  # O(E log V) worst. Returns dist or -1. nbrs(u) -> (v, w).
  dist = {start: 0}
  pq = [(h(start), 0, start)]
  while pq:
    f, g, u = heapq.heappop(pq)
    if g > dist[u]:
      continue
    if u == goal:
      return g
    for v, w in nbrs(u):
      ng = g + w
      if ng < dist.get(v, float("inf")):
        dist[v] = ng
        heapq.heappush(pq, (ng + h(v), ng, v))
  return -1
# States must be hashable & comparable (tuples) for the heap
↪  tie-breaks; else add a counter.
```

– 12.3 Bidirectional BFS

```
def bidir_bfs(s, t, nbrs):
  # Unweighted, undirected (or give reverse nbrs for t side).
  ↪  O(b^(d/2)). Returns dist or -1.
  if s == t:
    return 0
  ds, dt = {s: 0}, {t: 0}
  fs, ft = [s], [t]
  while fs and ft:
    if len(fs) > len(ft):
      fs, ft, ds, dt = ft, fs, dt, ds
    best = None
    nf = []
    for u in fs:
      du = ds[u] + 1
      for v in nbrs(u):
        if v in dt:
          c = du + dt[v]
          if best is None or c < best:
            best = c
        if v not in ds:
          ds[v] = du
          nf.append(v)
    if best is not None:
      return best
    fs = nf
  return -1
# Expand a whole level of the smaller frontier, take min over
↪  all meetings in that level.
```

– 12.3 Branch and bound / pruning

```
# Keep global best; prune a branch when (current + optimistic
↪  bound of remaining) can't beat best.
# Good bounds: fractional relaxation (knapsack: greedy by
↪  ratio), sum of remaining positives.
# Order branches best-first (greedy choice first) so a good
↪  best is found early.
# Pruning strategies: symmetry breaking (fix first element),
↪  feasibility checks, memo of dead states,
# sort items descending so infeasibility shows early.
```

– 12.4 Binary lifting / doubling on functional graph

```
class FuncGraph:
  # nxt[v] = successor. Build O(n log K), kth(v, k) O(log K).
  ↪  Also carry sums/min in parallel tables.
  def __init__(self, nxt, maxk):
    self.up = [list(nxt)]
    for _ in range(max(1, maxk.bit_length()) - 1):
      p = self.up[-1]
      self.up.append([p[p[v]] for v in range(len(nxt))])
  def kth(self, v, k):
    j = 0
    while k:
      if k & 1:
        v = self.up[j][v]
      k >>= 1
      j += 1
    return v
# Huge k (1e18) with one start: walk until cycle found (dict of
↪  first visit), then k mod cycle length.
# Small-to-large merging, binary lifting LCA -> see 06_trees.py.
```

– 12.4 Other advanced techniques (notes)

```
# Bitset speedup: Python big ints as bitsets: subset sum reach
↪  |= reach << w (mask to limit);
#   adjacency rows as ints, (row_u & row_v).bit_count() for
↪  common neighbors. Huge constant win.
# Four Russians: split into blocks of size ~log n, precompute
↪  all 2^blk block answers/transitions
#   (e.g. boolean matrix mult n^3/log n, LCS bit-parallel, +-1
↪  RMQ). In Python use the bitset trick.
# FPT (conceptual): exponential only in a parameter k, e.g.
↪  vertex cover O(2^k * m) by branching on
#   an uncovered edge's endpoints; kernelization shrinks the
↪  instance first.
# Matroid intersection -> 05_flow_matching.py.
# Offline-to-online: if queries are forced online (xor with
↪  last answer), use persistent DS or
#   sqrt rebuilding; binary grouping (keep log n static
↪  structures of sizes 2^i, merge like a counter).
```

– 13 Problem types and paradigms (notes)

```
# 13.1 Constructive: brute force small n and print answers to
↪  spot the pattern; try simple families
#   (sorted, alternating, 1..n shifted, identity + swaps,
↪  star/path graphs); prove impossibility via
#   invariant (parity, sum mod k, count bound). Invariant =
↪  never changes; monovariant = strictly
#   monotone -> termination.
# 13.2 Interactive: bound queries by information theory (need
↪  >= log2(#answers) bits; ternary replies
#   -> log3). Binary search via queries; adaptive judges may
↪  change the hidden answer consistently,
#   so strategies must be worst-case. Flush after each query
↪  (00_core.py: ask/answer).
# 13.3 Ad hoc: look for invariants, parity, symmetry;
↪  reformulate (reduce to known problem); count by
#   contribution (sum over elements of #subarrays/pairs where
↪  it matters) instead of over structures.
# 13.4 Output-only / heuristic: run long offline searches;
↪  scoring-based -> SA / beam search / local
#   search; always keep a valid greedy baseline answer first.
# 13.5 Communication (two-run): encode info in output of run 1
↪  for run 2. Grid puzzles / games:
#   state search (BFS/IDA*) or game theory (09). Subtasks: grab
↪  partial points with brute force first.
```

02 Data structures

```
import math
import random
from bisect import bisect_left, bisect_right, insort
from heapq import heappush, heappop
from collections import deque, Counter
```

– 2.2 STL containers -> Python

```
# vector/array -> list (append/pop O(1); pop(0)/insert(0)
↪  O(n)!). 2D: [[0]*m for _ in range(n)].
# deque/queue/stack -> collections.deque
↪  (append/appendleft/pop/popleft O(1); d[i] is O(n)).
# priority_queue -> heapq (MIN-heap on a list): heappush,
↪  heappop, h[0]=top, heapify O(n).
#   max-heap: push -x; key-ordered: push (key, id, payload);
↪  tie id avoids comparing payloads.
#   heappushpop(h,x) / heapreplace(h,x) are faster combined
↪  ops; nlargest/nsmallest(k, it).
# set/multiset/map + pbds ordered set -> SortedList below (no
↪  stdlib balanced BST).
#   SortedDict-ish: SortedList of keys + plain dict key->value;
↪  iterate keys in order.
# unordered_set/map -> set/dict (int keys are hackable on CF:
↪  salt keys, see 00_core.py 4.3/19);
#   Counter(it) multiset counts, defaultdict(list) adjacency,
↪  dict keeps insertion order.
# bitset -> Python int (see 4.8 bitset note). pair/tuple ->
↪  tuple (compare lexicographic).
```

```
# string -> str (immutable: build with list + ''.join); bytes
↪  for fast byte ops.
```

– 2.3 STL algorithms -> Python

```
# sort/stable_sort -> list.sort()/sorted(): Timsort, ALWAYS
↪  stable; key=... (fast) or
#   key=functools.cmp_to_key(cmp) with cmp(a,b)-> neg/0/pos
↪  (slow, needs strict weak order).
#   multi-key: sort by secondary first, then primary
↪  (stability), or key=lambda t:(t[0],-t[1]).
# lower_bound/upper_bound -> bisect_left/bisect_right(a, x, lo,
↪  hi, key=...) (key= py3.10+).
#   equal_range = (bisect_left, bisect_right); binary_search =
↪  i<len(a) and a[i]==x.
# next_permutation -> itertools.permutations(sorted(a))
↪  (lexicographic, duplicates repeated);
#   combinations, combinations_with_replacement,
↪  product(range(2), repeat=n) for masks.
# unique (sorted) -> sorted(set(a)); keep order:
↪  list(dict.fromkeys(a)). reverse -> a[::-1].
# rotate left k -> a[k:]+a[:k]. nth_element/partial_sort ->
↪  heapq.nsmallest(k,a) or sort.
# accumulate/partial_sum -> itertools.accumulate(a, initial=0)
↪  (prefix sums), op= any func.
# iota -> list(range(n)); max_element -> max(range(n),
↪  key=a.__getitem__); min/max/sum builtins.
# SortedList: sqrt-bucket sorted multiset. add/discard O(sqrt
↪  n) (memmove, fast), index/kth O(n^(1/2)).
# Replaces std::multiset and pbds order-statistics tree. For a
↪  set, check `x in s` before add.
class SortedList:
  BUCKET_RATIO, SPLIT_RATIO = 16, 24
  def __init__(self, a=()):
    a = sorted(a)
    n = self.size = len(a)
    k = int(math.ceil(math.sqrt(n / self.BUCKET_RATIO))) or 1
    self.a = [a[n * i // k: n * (i + 1) // k] for i in range(k)]
    if n == 0: self.a = []
  def __iter__(self):
    for b in self.a: yield from b
  def __len__(self): return self.size
  def __contains__(self, x):
    if self.size == 0: return False
    b, _, i = self._pos(x)
    return i != len(b) and b[i] == x
  def _pos(self, x):  # bucket, bucket idx, bisect_left inside
  ↪  bucket
    for j, b in enumerate(self.a):
      if x <= b[-1]: return b, j, bisect_left(b, x)
    return b, j, len(b)
  def add(self, x):
    if self.size == 0:
      self.a = [[x]]; self.size = 1; return
    b, j, i = self._pos(x)
    b.insert(i, x); self.size += 1
    if len(b) > len(self.a) * self.SPLIT_RATIO:
      m = len(b) >> 1
      self.a[j:j + 1] = [b[:m], b[m:]]
  def _pop(self, b, j, i):
    x = b.pop(i); self.size -= 1
    if not b: del self.a[j]
    return x
  def discard(self, x):  # remove one copy; returns False if
  ↪  absent
    if self.size == 0: return False
    b, j, i = self._pos(x)
    if i == len(b) or b[i] != x: return False
    self._pop(b, j, i); return True
  def __getitem__(self, k):  # k-th smallest (0-indexed,
  ↪  negative ok)
    if k < 0: k += self.size
    for b in self.a:
      if k < len(b): return b[k]
      k -= len(b)
    raise IndexError
  def pop(self, k=-1):
    if k < 0: k += self.size
    for j, b in enumerate(self.a):
      if k < len(b): return self._pop(b, j, k)
      k -= len(b)
    raise IndexError
  def index(self, x):  # number of elements < x  (= bisect_left)
    c = 0
    for b in self.a:
      if b[-1] >= x: return c + bisect_left(b, x)
      c += len(b)
    return c
  def index_right(self, x):  # number of elements <= x (=
  ↪  bisect_right)
    c = 0
    for b in self.a:
      if b[-1] > x: return c + bisect_right(b, x)
      c += len(b)
    return c
  def count(self, x): return self.index_right(x) - self.index(x)
  def lt(self, x):  # largest < x, else None (also le/gt/ge via
  ↪  index)
    i = self.index(x); return self[i - 1] if i else None
  def ge(self, x):  # smallest >= x, else None
    i = self.index(x); return self[i] if i < self.size else None
```

– 4.1 Linear structures

```
# next_greater: O(n). res[i] = smallest j>i with a[j]>a[i],
↪  else n. (>= : pop on <=; prev: scan reversed)
def next_greater(a):
  n = len(a); res = [n] * n; st = []
  for i, x in enumerate(a):
    while st and a[st[-1]] < x: res[st.pop()] = i
    st.append(i)
  return res
# largest_rectangle: O(n) max area in histogram h.
def largest_rectangle(h):
  st = []; best = 0  # stack of (start, height), heights
↪  increasing
  for i, x in enumerate(h + [0]):
    start = i
    while st and st[-1][1] >= x:
      s, y = st.pop(); best = max(best, y * (i - s)); start = s
    st.append((start, x))
  return best
# sliding_min: O(n) monotonic deque, min of each window
↪  a[i-k+1..i] for i>=k-1. (max: flip compare)
def sliding_min(a, k):
  dq = deque(); res = []
  for i, x in enumerate(a):
    while dq and a[dq[-1]] >= x: dq.pop()
    dq.append(i)
    if dq[0] <= i - k: dq.popleft()
    if i >= k - 1: res.append(a[dq[0]])
  return res
# Linked list via arrays: nxt=[i+1..], prv=[i-1..]; delete i:
↪  nxt[prv[i]]=nxt[i]; prv[nxt[i]]=prv[i].
#   Use sentinels 0 and n+1. Undo in reverse order (dancing
↪  links). O(1) per op, no objects.
# XOR linked list: memory trick only, pointless in Python.
```

– 4.2 Heaps and priority queues

```
# Binary heap: array, children 2i+1,2i+2; push/pop O(log n),
↪  heapify O(n) bottom-up -> use heapq.
# Pairing / binomial heaps: mergeable heaps, rarely needed;
↪  leftist heap below covers merging.
# LazyHeap: min-heap with arbitrary deletion O(log n)
↪  amortized. remove(x) assumes x is present.
class LazyHeap:
  def __init__(self): self.h = []; self.d = Counter(); self.n =
  ↪  0
  def push(self, x): heappush(self.h, x); self.n += 1
  def remove(self, x): self.d[x] += 1; self.n -= 1
  def _clean(self):
    h, d = self.h, self.d
    while h and d[h[0]]: d[h[0]] -= 1; heappop(h)
  def top(self): self._clean(); return self.h[0]
  def pop(self): self._clean(); self.n -= 1; return
  ↪  heappop(self.h)
  def __len__(self): return self.n
# RunningMedian: two heaps, add O(log n), median() = lower
↪  median. lo is max-heap (negated).
class RunningMedian:
  def __init__(self): self.lo = []; self.hi = []
  def add(self, x):
    if self.lo and x > -self.lo[0]: heappush(self.hi, x)
    else: heappush(self.lo, -x)
    if len(self.lo) > len(self.hi) + 1: heappush(self.hi,
    ↪  -heappop(self.lo))
    elif len(self.hi) > len(self.lo): heappush(self.lo,
    ↪  -heappop(self.hi))
  def median(self): return -self.lo[0]
# LeftistHeap: pool of mergeable min-heaps; merge/pop O(log n).
↪  Heap = root id, 0 = empty.
# Recursion depth <= 2 log n (right spines). Use: r = H.merge(r,
↪  H.new(x)); r = H.pop(r).
class LeftistHeap:
  def __init__(self): self.key = [0]; self.L = [0]; self.R =
  ↪  [0]; self.D = [0]
  def new(self, x):
    self.key.append(x); self.L.append(0); self.R.append(0);
    ↪  self.D.append(1)
    return len(self.key) - 1
  def merge(self, a, b):
    if not a or not b: return a or b
    K, L, R, D = self.key, self.L, self.R, self.D
    if K[b] < K[a]: a, b = b, a
    R[a] = self.merge(R[a], b)
    if D[L[a]] < D[R[a]]: L[a], R[a] = R[a], L[a]
    D[a] = D[R[a]] + 1
    return a
  def top(self, a): return self.key[a]
  def pop(self, a): return self.merge(self.L[a], self.R[a])
# Skew heap: same merge but always swap children, no D
↪  (amortized log n, recursion can go deep).
```

– 4.3 Hashing

```
# Hash tables: dict/set (open addressing). Anti-hash for int
↪  keys: see 00_core.py 4.3/19.
# Rolling / polynomial hash, randomized bases: see
↪  07_strings.py.
# Zobrist: give each (position, state) a random 64-bit r;
↪  hash(state) = XOR of r's of its parts;
#   updates are O(1) (xor out old, xor in new). Used for board
↪  states / game memo / sets in DP.
# Set hashing: XOR of random values (elements of even
↪  multiplicity cancel -> parity sets).
# Multiset hashing: SUM of random values mod 2^64 (or 2^61-1);
↪  order-independent, O(1) update.
# multiset_prefix_hash: O(n). a[l:r] and b[l2:r2] equal as
↪  multisets (whp) iff
#   P[r]-P[l] == Q[r2]-Q[l2] (mod 2^64), using the SAME rnd dict
↪  for both arrays.
def multiset_prefix_hash(a, rnd=None, M=(1 << 64) - 1):
  if rnd is None: rnd = {}
  P = [0]; s = 0
  for x in a:
    v = rnd.get(x)
    if v is None: v = rnd[x] = random.getrandbits(64)
    s = (s + v) & M; P.append(s)
  return P, rnd
```

– 4.4 Disjoint Set Union

```
# Path compression + union by size: O(alpha(n)) amortized
↪  (inverse Ackermann, <= 4 in practice).
class DSU:
  def __init__(self, n): self.p = [-1] * n  # p[x] < 0: root,
  ↪  -size
  def find(self, x):
    p = self.p; r = x
    while p[r] >= 0: r = p[r]
    while p[x] >= 0 and p[x] != r: p[x], x = r, p[x]
    return r
  def union(self, a, b):
    a, b = self.find(a), self.find(b)
    if a == b: return False
    p = self.p
    if p[a] > p[b]: a, b = b, a
    p[a] += p[b]; p[b] = a
    return True
  def same(self, a, b): return self.find(a) == self.find(b)
  def size(self, x): return -self.p[self.find(x)]
# WeightedDSU (potentials): maintains val[b]-val[a].
↪  union(a,b,w) asserts val[b]-val[a]=w,
# returns False on contradiction. diff(a,b) -> val[b]-val[a] or
↪  None. O(alpha) amortized.
# Parity DSU ("same/different color"): same thing with w in
↪  {0,1} and ^ instead of +/-.
class WeightedDSU:
  def __init__(self, n): self.p = list(range(n)); self.sz = [1]
  ↪  * n; self.pot = [0] * n  # val[x]-val[p[x]]
  def find(self, x):
    p, pot = self.p, self.pot; path = []
    while p[x] != x: path.append(x); x = p[x]
    for y in reversed(path):  # nearest-to-root first; pot[root]
    ↪  == 0
      pot[y] += pot[p[y]]; p[y] = x
    return x
  def union(self, a, b, w):
    ra, rb = self.find(a), self.find(b)
    pa, pb = self.pot[a], self.pot[b]
    if ra == rb: return pb - pa == w
    if self.sz[ra] < self.sz[rb]: self.p[ra] = rb; self.pot[ra]
    ↪  = pb - pa - w; self.sz[rb] += self.sz[ra]
    else: self.p[rb] = ra; self.pot[rb] = w + pa - pb;
    ↪  self.sz[ra] += self.sz[rb]
    return True
  def diff(self, a, b):
    if self.find(a) != self.find(b): return None
    return self.pot[b] - self.pot[a]
# RollbackDSU: union by size, NO path compression; find O(log
↪  n). snapshot() -> token, rollback(token).
# Used for offline dynamic connectivity (segment tree over
↪  time, DFS with undo).
class RollbackDSU:
  def __init__(self, n): self.p = [-1] * n; self.hist = [];
  ↪  self.comps = n
  def find(self, x):
    p = self.p
    while p[x] >= 0: x = p[x]
    return x
  def union(self, a, b):
    a, b = self.find(a), self.find(b)
    if a == b: return False
    p = self.p
    if p[a] > p[b]: a, b = b, a
    self.hist.append((a, b, p[b])); p[a] += p[b]; p[b] = a;
    ↪  self.comps -= 1
    return True
  def snapshot(self): return len(self.hist)
  def rollback(self, t):
    p, h = self.p, self.hist
    while len(h) > t:
      a, b, pb = h.pop(); p[a] -= pb; p[b] = pb; self.comps += 1
# Persistent DSU: union by size (no compression) on a
↪  persistent array (persistent segment tree)
#   -> O(log^2 n) per op with versions. Offline alternative: DFS
↪  over version tree + RollbackDSU.
```

– 4.5 Range query structures

```
# SparseTable: build O(n log n), query O(1) for IDEMPOTENT op
↪  (min, max, gcd, and, or). q(l, r) on [l, r).
class SparseTable:
  def __init__(self, a, op=min):
    self.op = op; t = [list(a)]; j = 1
    while 2 * j <= len(a):
      p = t[-1]; t.append([op(p[i], p[i + j]) for i in
↪  range(len(a) - 2 * j + 1)]); j *= 2
    self.t = t
  def query(self, l, r):  # requires l < r
    k = (r - l).bit_length() - 1
    return self.op(self.t[k][l], self.t[k][r - (1 << k)])
# DisjointSparseTable: build O(n log n), O(1) query for ANY
↪  associative op (sum, product mod, matrix).
class DisjointSparseTable:
  def __init__(self, a, op):
    self.op = op; n = len(a); a = list(a); self.t = [a]
    for h in range(1, max(1, (n - 1).bit_length())):
      row = a[:]; half = 1 << h
      for mid in range(half, n, 2 * half):
        for i in range(mid - 2, mid - half - 1, -1): row[i] =
        ↪  op(a[i], row[i + 1])
        for i in range(mid + 1, min(mid + half, n)): row[i] =
        ↪  op(row[i - 1], a[i])
      self.t.append(row)
  def query(self, l, r):  # [l, r), l < r
    r -= 1
    if l == r: return self.t[0][l]
    h = (l ^ r).bit_length() - 1
    return self.op(self.t[h][l], self.t[h][r])
# SqrtDecomp: range add + range sum, O(sqrt n) per op. Template
↪  for any "block + lazy" trick.
class SqrtDecomp:
  def __init__(self, a):
    n = len(a); self.B = B = max(1, int(n ** 0.5)); self.a =
↪  list(a)
    self.s = [sum(a[i:i + B]) for i in range(0, n, B)]; self.lz
    ↪  = [0] * len(self.s)
  def add(self, l, r, x):
    B, a = self.B, self.a
    while l < r and l % B: a[l] += x; self.s[l // B] += x; l += 1
    while l + B <= r: self.lz[l // B] += x; l += B
    while l < r: a[l] += x; self.s[l // B] += x; l += 1
  def sum(self, l, r):
    B, a, res = self.B, self.a, 0
    while l < r and l % B: res += a[l] + self.lz[l // B]; l += 1
    while l + B <= r: res += self.s[l // B] + self.lz[l // B] *
    ↪  B; l += B
    while l < r: res += a[l] + self.lz[l // B]; l += 1
    return res
```

```python
# Fenwick: point add / prefix sum O(log n), build O(n). kth via
↪ binary lifting.
# Range add + point query: Fenwick on differences: add(l,x),
↪ add(r,-x); point i = sum(i+1).
class Fenwick:
  def __init__(self, n_or_a):
    if isinstance(n_or_a, int): self.n = n_or_a; self.t = [0] *
    ↪ (n_or_a + 1); return
    self.n = n = len(n_or_a); t = self.t = [0] + list(n_or_a)
    for i in range(1, n + 1):
      j = i + (i & -i)
      if j <= n: t[j] += t[i]
  def add(self, i, x):
    i += 1; t = self.t
    while i <= self.n: t[i] += x; i += i & -i
  def sum(self, i):  # sum of [0, i)
    t = self.t; s = 0
    while i: s += t[i]; i &= i - 1
    return s
  def range_sum(self, l, r): return self.sum(r) - self.sum(l)
  def lower_bound(self, k):  # min i with sum(i+1) >= k (values
  ↪ >= 0); = k-th element (k>=1) of counts
    t = self.t; pos = 0; step = 1 << self.n.bit_length()
    while step:
      if pos + step <= self.n and t[pos + step] < k: pos +=
      ↪ step; k -= t[pos]
      step >>= 1
    return pos  # == n if total < k
# RangeFenwick: range add + range sum, O(log n). prefix(i) =
↪ sum1(i)*i - sum2(i).
class RangeFenwick:
  def __init__(self, n): self.n = n; self.b1 = [0] * (n + 1);
  ↪ self.b2 = [0] * (n + 1)
  def _add(self, t, i, x):
    i += 1
    while i <= self.n: t[i] += x; i += i & -i
  def add(self, l, r, x):  # a[l:r] += x
    self._add(self.b1, l, x); self._add(self.b1, r, -x)
    self._add(self.b2, l, x * l); self._add(self.b2, r, -x * r)
  def prefix(self, i):  # sum of [0, i)
    s1 = s2 = 0; j = i
    while j: s1 += self.b1[j]; s2 += self.b2[j]; j &= j - 1
    return s1 * i - s2
  def sum(self, l, r): return self.prefix(r) - self.prefix(l)
# Fenwick2D: point add, rectangle sum, O(log n log m).
↪ Flattened array. Sparse coords: compress.
class Fenwick2D:
  def __init__(self, n, m): self.n, self.m = n, m; self.t = [0]
  ↪ * ((n + 1) * (m + 1))
  def add(self, x, y, v):
    m1 = self.m + 1; i = x + 1
    while i <= self.n:
      j = y + 1
      while j <= self.m: self.t[i * m1 + j] += v; j += j & -j
      i += i & -i
  def prefix(self, x, y):  # sum over [0,x) x [0,y)
    m1 = self.m + 1; s = 0; i = x
    while i:
      j = y
      while j: s += self.t[i * m1 + j]; j &= j - 1
      i &= i - 1
    return s
  def rect(self, x1, y1, x2, y2):  # [x1,x2) x [y1,y2)
    return self.prefix(x2, y2) - self.prefix(x1, y2) -
    ↪ self.prefix(x2, y1) + self.prefix(x1, y1)
```

– 4.6 Segment trees

```python
# SegTree (ACL-style, iterative bottom-up) over monoid (op, e).
↪ set/get/prod O(log n).
# max_right(l, f): largest r with f(prod(l, r)) True (f(e) must
↪ be True, f monotone).
# min_left(r, f): smallest l with f(prod(l, r)) True. Custom
↪ node: use tuples, e.g. (sum, maxpref).
class SegTree:
  def __init__(self, op, e, v):
    if isinstance(v, int): v = [e] * v
    self.op, self.e, self.n = op, e, len(v)
    self.log = max(0, (self.n - 1).bit_length()); self.size = 1
    ↪ << self.log
    d = self.d = [e] * (2 * self.size); d[self.size:self.size +
    ↪ self.n] = v
    for i in range(self.size - 1, 0, -1): d[i] = op(d[2 * i],
    ↪ d[2 * i + 1])
  def set(self, p, x):
    p += self.size; d, op = self.d, self.op; d[p] = x
    while p > 1: p >>= 1; d[p] = op(d[2 * p], d[2 * p + 1])
  def get(self, p): return self.d[p + self.size]
  def prod(self, l, r):
    op, d = self.op, self.d; sml = smr = self.e; l += self.size;
    ↪ r += self.size
    while l < r:
      if l & 1: sml = op(sml, d[l]); l += 1
      if r & 1: r -= 1; smr = op(d[r], smr)
      l >>= 1; r >>= 1
    return op(sml, smr)
  def all_prod(self): return self.d[1]
  def max_right(self, l, f):
    if l == self.n: return self.n
    op, d, size = self.op, self.d, self.size; l += size; sm =
    ↪ self.e
    while True:
      while l % 2 == 0: l >>= 1
      if not f(op(sm, d[l])):
        while l < size:
          l *= 2
          if f(op(sm, d[l])): sm = op(sm, d[l]); l += 1
        return l - size
      sm = op(sm, d[l]); l += 1
      if l & -l == l: return self.n
  def min_left(self, r, f):
    if r == 0: return 0
    op, d, size = self.op, self.d, self.size; r += size; sm =
    ↪ self.e
    while True:
      r -= 1
      while r > 1 and r % 2: r >>= 1
      if not f(op(d[r], sm)):
        while r < size:
          r = 2 * r + 1
          if f(op(d[r], sm)): sm = op(d[r], sm); r -= 1
        return r + 1 - size
      sm = op(d[r], sm)
      if r & -r == r: return 0
# LazySegTree (ACL-style): monoid (op, e), maps F with
↪ mapping(f, x), composition(f, g) = f after g,
# identity idf. apply(l, r, f), prod(l, r), set/get,
↪ max_right/min_left: all O(log n).
# Ex. range add/range min: op=min, e=INF, mapping=lambda f,x:
↪ f+x, comp=lambda f,g: f+g, idf=0.
# Ex. range affine/range sum: x=(sum,len), f=(b,c): (b*s+c*len,
↪ len); comp(f,g)=(fb*gb, fb*gc+fc).
class LazySegTree:
  def __init__(self, op, e, mapping, composition, idf, v):
    if isinstance(v, int): v = [e] * v
    self.op, self.e, self.mp, self.cp, self.idf, self.n = op, e,
    ↪ mapping, composition, idf, len(v)
    self.log = max(0, (self.n - 1).bit_length()); self.size = 1
    ↪ << self.log
    self.d = [e] * (2 * self.size); self.lz = [idf] * self.size
    self.d[self.size:self.size + self.n] = v
    for i in range(self.size - 1, 0, -1): self._up(i)
  def _up(self, k): self.d[k] = self.op(self.d[2 * k], self.d[2
  ↪ * k + 1])
  def _ap(self, k, f):
    self.d[k] = self.mp(f, self.d[k])
    if k < self.size: self.lz[k] = self.cp(f, self.lz[k])
  def _push(self, k):
    if self.lz[k] != self.idf:
      self._ap(2 * k, self.lz[k]); self._ap(2 * k + 1,
      ↪ self.lz[k]); self.lz[k] = self.idf
  def set(self, p, x):
    p += self.size
    for i in range(self.log, 0, -1): self._push(p >> i)
    self.d[p] = x
    for i in range(1, self.log + 1): self._up(p >> i)
  def get(self, p):
    p += self.size
    for i in range(self.log, 0, -1): self._push(p >> i)
    return self.d[p]
  def prod(self, l, r):
    if l == r: return self.e
    l += self.size; r += self.size
    for i in range(self.log, 0, -1):
      if ((l >> i) << i) != l: self._push(l >> i)
      if ((r >> i) << i) != r: self._push((r - 1) >> i)
    op, d = self.op, self.d; sml = smr = self.e
    while l < r:
      if l & 1: sml = op(sml, d[l]); l += 1
      if r & 1: r -= 1; smr = op(d[r], smr)
      l >>= 1; r >>= 1
    return op(sml, smr)
  def apply(self, l, r, f):
    if l == r: return
    l += self.size; r += self.size
    for i in range(self.log, 0, -1):
      if ((l >> i) << i) != l: self._push(l >> i)
      if ((r >> i) << i) != r: self._push((r - 1) >> i)
    l2, r2 = l, r
    while l < r:
      if l & 1: self._ap(l, f); l += 1
      if r & 1: r -= 1; self._ap(r, f)
      l >>= 1; r >>= 1
    l, r = l2, r2
    for i in range(1, self.log + 1):
      if ((l >> i) << i) != l: self._up(l >> i)
      if ((r >> i) << i) != r: self._up((r - 1) >> i)
  def max_right(self, l, g):
    if l == self.n: return self.n
    l += self.size
    for i in range(self.log, 0, -1): self._push(l >> i)
    op, d, sm = self.op, self.d, self.e
    while True:
      while l % 2 == 0: l >>= 1
      if not g(op(sm, d[l])):
        while l < self.size:
          self._push(l); l *= 2
          if g(op(sm, d[l])): sm = op(sm, d[l]); l += 1
        return l - self.size
      sm = op(sm, d[l]); l += 1
      if l & -l == l: return self.n
  def min_left(self, r, g):
    if r == 0: return 0
    r += self.size
    for i in range(self.log, 0, -1): self._push((r - 1) >> i)
    op, d, sm = self.op, self.d, self.e
    while True:
      r -= 1
      while r > 1 and r % 2: r >>= 1
      if not g(op(d[r], sm)):
        while r < self.size:
          self._push(r); r = 2 * r + 1
          if g(op(d[r], sm)): sm = op(d[r], sm); r -= 1
        return r + 1 - self.size
      sm = op(d[r], sm)
      if r & -r == r: return 0
# MergeSortTree: build O(n log n) memory O(n log n); count of
↪ a[i] <= x in [l, r): O(log^2 n).
class MergeSortTree:
  def __init__(self, a):
    n = len(a); self.size = s = 1 << max(0, (n - 1).bit_length())
    t = self.t = [[] for _ in range(2 * s)]
    for i, x in enumerate(a): t[s + i] = [x]
    for i in range(s - 1, 0, -1): t[i] = sorted(t[2 * i] + t[2 *
    ↪ i + 1])  # Timsort merges runs
  def count_le(self, l, r, x):
    t = self.t; l += self.size; r += self.size; c = 0
    while l < r:
      if l & 1: c += bisect_right(t[l], x); l += 1
      if r & 1: r -= 1; c += bisect_right(t[r], x)
      l >>= 1; r >>= 1
    return c
# PersistentSegTree: k-th smallest in a[l:r] in O(log n); build
↪ O(n log n) nodes. Arrays, iterative.
# roots[i] = count tree of a[:i] over compressed values; node 0
↪ = shared null.
# Same pattern gives versioned arrays: copy the root-to-leaf
↪ path on each update.
class PersistentKth:
  def __init__(self, a):
    self.vals = vals = sorted(set(a)); idx = {v: i for i, v in
    ↪ enumerate(vals)}; self.m = m = len(vals)
    L, R, C = self.L, self.R, self.C = [0], [0], [0]
    self.roots = roots = [0]
    for x in a:
      pos = idx[x]; prev = roots[-1]; lo, hi = 0, m;
↪ roots.append(len(C))
      while True:
        nid = len(C); L.append(L[prev]); R.append(R[prev]);
↪ C.append(C[prev] + 1)
        if hi - lo == 1: break
        mid = (lo + hi) >> 1
        if pos < mid: L[nid] = nid + 1; prev = L[prev]; hi = mid
        else: R[nid] = nid + 1; prev = R[prev]; lo = mid
  def kth(self, l, r, k):  # k-th smallest (0-indexed) of a[l:r]
    L, R, C = self.L, self.R, self.C; u, v = self.roots[r],
↪ self.roots[l]; lo, hi = 0, self.m
    while hi - lo > 1:
      mid = (lo + hi) >> 1; c = C[L[u]] - C[L[v]]
      if k < c: u, v, hi = L[u], L[v], mid
      else: k -= c; u, v, lo = R[u], R[v], mid
    return self.vals[lo]
# DynamicSegTree: point add / range sum on huge coordinate
↪ range [lo, hi), nodes created lazily.
# O(log(hi-lo)) per op, memory O(q log C). Online alternative
↪ to coordinate compression.
class DynamicSegTree:
  def __init__(self, lo, hi): self.lo, self.hi = lo, hi; self.L
  ↪ = [0, 0]; self.R = [0, 0]; self.S = [0, 0]
  def _new(self): self.L.append(0); self.R.append(0);
  ↪ self.S.append(0); return len(self.S) - 1
  def add(self, p, x):
    u, lo, hi = 1, self.lo, self.hi
    while True:
      self.S[u] += x
      if hi - lo == 1: return
      mid = (lo + hi) >> 1
      if p < mid:
        if not self.L[u]: self.L[u] = self._new()
        u, hi = self.L[u], mid
      else:
        if not self.R[u]: self.R[u] = self._new()
        u, lo = self.R[u], mid
  def query(self, l, r):
    res = 0; st = [(1, self.lo, self.hi)]; L, R, S = self.L,
↪ self.R, self.S
    while st:
      u, a, b = st.pop()
      if not u or b <= l or r <= a: continue
      if l <= a and b <= r: res += S[u]; continue
      mid = (a + b) >> 1; st.append((L[u], a, mid));
↪ st.append((R[u], mid, b))
    return res
# 2D segment tree / segment tree of BITs: outer tree over x,
↪ each node holds a structure over y
#   (sorted ys + Fenwick, built offline) -> O(log^2)
↪ query/update. In Python prefer offline
#   sweep over x + Fenwick over y (see 01_techniques.py / 11
↪ offline), or Fenwick2D above.
# SegTreeBeats (Ji Driver): chmin(l,r,x), sum(l,r), max(l,r).
↪ O((n+q) log n) amortized.
# Recursion depth = log n. Extend with chmax/add by tracking
↪ min/second-min the same way.
class SegTreeBeats:
  NEG = -(1 << 62)
  def __init__(self, a):
    n = len(a); s = self.size = 1 << max(0, (n -
↪ 1).bit_length()); NEG = self.NEG
    self.mx = [NEG] * (2 * s); self.smx = [NEG] * (2 * s);
    ↪ self.cnt = [0] * (2 * s); self.sm = [0] * (2 * s)
    for i, x in enumerate(a): self.mx[s + i] = x; self.cnt[s +
    ↪ i] = 1; self.sm[s + i] = x
    for k in range(s - 1, 0, -1): self._pull(k)
  def _pull(self, k):
    mx, smx, cnt = self.mx, self.smx, self.cnt; l, r = 2 * k, 2
↪ * k + 1
    self.sm[k] = self.sm[l] + self.sm[r]
    if mx[l] > mx[r]: mx[k] = mx[l]; cnt[k] = cnt[l]; smx[k] =
    ↪ max(smx[l], mx[r])
    elif mx[l] < mx[r]: mx[k] = mx[r]; cnt[k] = cnt[r]; smx[k] =
    ↪ max(mx[l], smx[r])
    else: mx[k] = mx[l]; cnt[k] = cnt[l] + cnt[r]; smx[k] =
    ↪ max(smx[l], smx[r])
  def _tag(self, k, x):  # requires smx[k] < x
    if self.mx[k] > x: self.sm[k] -= (self.mx[k] - x) *
    ↪ self.cnt[k]; self.mx[k] = x
  def _push(self, k):
    self._tag(2 * k, self.mx[k]); self._tag(2 * k + 1,
    ↪ self.mx[k])
  def chmin(self, l, r, x, k=1, a=0, b=None):
    if b is None: b = self.size
    if r <= a or b <= l or self.mx[k] <= x: return
    if l <= a and b <= r and self.smx[k] < x: self._tag(k, x);
    ↪ return
    self._push(k); m = (a + b) >> 1
    self.chmin(l, r, x, 2 * k, a, m); self.chmin(l, r, x, 2 * k
    ↪ + 1, m, b); self._pull(k)
  def _query(self, l, r, k, a, b, is_sum):
    if r <= a or b <= l: return 0 if is_sum else self.NEG
    if l <= a and b <= r: return self.sm[k] if is_sum else
    ↪ self.mx[k]
    self._push(k); m = (a + b) >> 1
    x, y = self._query(l, r, 2 * k, a, m, is_sum),
↪ self._query(l, r, 2 * k + 1, m, b, is_sum)
    return x + y if is_sum else max(x, y)
```

```
  def sum(self, l, r): return self._query(l, r, 1, 0,
  ↪  self.size, True)
  def max(self, l, r): return self._query(l, r, 1, 0,
  ↪  self.size, False)
# LiChao: min of lines y=a*x+b at query points xs (known in
↪  advance, any order). add O(log n),
# query O(log n), iterative. For max: insert (-a,-b) and
↪  negate. Used by 03_dp.py CHT.
class LiChao:
  INF = float('inf')
  def __init__(self, xs):
    xs = sorted(set(xs)); self.idx = {x: i for i, x in
↪  enumerate(xs)}
    self.size = s = 1 << max(0, (len(xs) - 1).bit_length())
    self.xs = xs + [xs[-1]] * (s - len(xs)); self.A = [0] * (2 *
    ↪  s); self.B = [self.INF] * (2 * s)
  def add_line(self, a, b):
    A, B, xs = self.A, self.B, self.xs; k, lo, hi = 1, 0,
↪  self.size
    while True:
      mid = (lo + hi) >> 1; xm = xs[mid]
      if a * xm + b < A[k] * xm + B[k]: A[k], a = a, A[k]; B[k],
      ↪  b = b, B[k]
      if hi - lo == 1 or b == self.INF: return
      xl, xr = xs[lo], xs[hi - 1]
      if a * xl + b < A[k] * xl + B[k]: k, hi = 2 * k, mid
      elif a * xr + b < A[k] * xr + B[k]: k, lo = 2 * k + 1, mid
      else: return
  def query(self, x):
    k = self.idx[x] + self.size; res = self.INF
    while k: res = min(res, self.A[k] * x + self.B[k]); k >>= 1
    return res
# Segment insertion (line only on [l,r)): decompose [l,r) into
↪  O(log n) nodes like SegTree.prod
#   and run add_line's descent from each node -> O(log^2 n).
# SegMerge: pool of dynamic count-segment-trees over values [0,
↪  M); merge(a,b) destroys both,
# total merge cost O(n log M) amortized. Typical:
↪  small-to-large on trees -> merge children's trees.
class SegMerge:
  def __init__(self, M): self.M = M; self.L = [0]; self.R = [0];
  ↪  self.C = [0]
  def _node(self, c):
    self.L.append(0); self.R.append(0); self.C.append(c);
    ↪  return len(self.C) - 1
  def single(self, v):  # new tree containing value v once
    root = u = self._node(1); lo, hi = 0, self.M
    while hi - lo > 1:
      mid = (lo + hi) >> 1; w = self._node(1)
      if v < mid: self.L[u] = w; hi = mid
      else: self.R[u] = w; lo = mid
      u = w
    return root
  def merge(self, a, b):  # recursion depth log M
    if not a or not b: return a or b
    self.C[a] += self.C[b]
    self.L[a] = self.merge(self.L[a], self.L[b]); self.R[a] =
    ↪  self.merge(self.R[a], self.R[b])
    return a
  def kth(self, u, k):  # k-th smallest (0-indexed)
    lo, hi = 0, self.M
    while hi - lo > 1:
      mid = (lo + hi) >> 1; c = self.C[self.L[u]]
      if k < c: u, hi = self.L[u], mid
      else: k -= c; u, lo = self.R[u], mid
    return lo
# Split (by value k): walk down, detaching right parts into new
↪  nodes; same O(log M).
# Kinetic segment tree: each node stores current max line plus
↪  "melt time" when the winner changes;
#   supports heaten(l,r,dt) (x += dt for all) with amortized
↪  O(log^2 n). Rare; Li Chao/CHT usually suffice.
```

– 4.7 Balanced trees

```
# BST (unbalanced): O(depth) ops, degenerates on sorted input
↪  -> never use plain in contests.
# Order statistics tree / ordered set: use SortedList (2.2
↪  above), or Fenwick.lower_bound on
#   compressed values (offline), or the treap below with size
↪  fields.
# ImplicitTreap: sequence with insert/erase at position,
↪  reverse(l,r), sum(l,r), O(log n) expected.
# Arrays-based; split/merge recurse to depth O(log n) expected
↪  (~3 log n), safe. Also = rope.
# Add more lazies (range add, assign) in _push/_pull the same
↪  way as rev.
class ImplicitTreap:
  def __init__(self, a=()):
    self.val = [0]; self.pri = [0]; self.L = [0]; self.R = [0];
    ↪  self.sz = [0]; self.sm = [0]; self.rev = [0]
    self.root = 0
    for x in a: self.root = self.merge(self.root, self._new(x))
  def _new(self, x):
    self.val.append(x); self.pri.append(random.random());
    ↪  self.L.append(0); self.R.append(0)
    self.sz.append(1); self.sm.append(x); self.rev.append(0)
    return len(self.val) - 1
  def _pull(self, t):
    l, r = self.L[t], self.R[t]
    self.sz[t] = self.sz[l] + 1 + self.sz[r]; self.sm[t] =
    ↪  self.sm[l] + self.val[t] + self.sm[r]
  def _push(self, t):
    if self.rev[t]:
      l, r = self.L[t], self.R[t]; self.L[t], self.R[t] = r, l
      if l: self.rev[l] ^= 1
      if r: self.rev[r] ^= 1
      self.rev[t] = 0
  def split(self, t, k):  # -> (first k elements, rest)
    if not t: return 0, 0
    self._push(t)
    if self.sz[self.L[t]] >= k:
      a, b = self.split(self.L[t], k); self.L[t] = b;
↪  self._pull(t); return a, t
    a, b = self.split(self.R[t], k - self.sz[self.L[t]] - 1);
↪  self.R[t] = a; self._pull(t); return t, b
  def merge(self, a, b):
    if not a or not b: return a or b
    if self.pri[a] > self.pri[b]:
      self._push(a); self.R[a] = self.merge(self.R[a], b);
      ↪  self._pull(a); return a
    self._push(b); self.L[b] = self.merge(a, self.L[b]);
    ↪  self._pull(b); return b
  def __len__(self): return self.sz[self.root]
  def insert(self, pos, x):
    a, b = self.split(self.root, pos); self.root =
↪  self.merge(self.merge(a, self._new(x)), b)
  def erase(self, pos):
    a, b = self.split(self.root, pos); _, c = self.split(b, 1);
↪  self.root = self.merge(a, c)
  def reverse(self, l, r):
    a, b = self.split(self.root, l); b, c = self.split(b, r - l)
    self.rev[b] ^= 1; self.root = self.merge(self.merge(a, b),
    ↪  c)
  def sum(self, l, r):
    a, b = self.split(self.root, l); b, c = self.split(b, r - l)
    s = self.sm[b]; self.root = self.merge(self.merge(a, b),
↪  c); return s
  def to_list(self):  # iterative in-order
    res, st, t = [], [], self.root
    while st or t:
      while t: self._push(t); st.append(t); t = self.L[t]
      t = st.pop(); res.append(self.val[t]); t = self.R[t]
    return res
# Keyed treap (ordered set): split by key instead of size: go
↪  left if key[t] >= k.
# Splay tree: amortized O(log n) by rotating accessed node to
↪  root (zig-zig/zig-zag); used inside
#   link-cut trees (4.8). As a sequence tree it is equivalent to
↪  the implicit treap.
# AVL (|h(l)-h(r)|<=1) / red-black (black-height equal, no
↪  red-red): worst-case O(log n), conceptual.
# Scapegoat: rebuild the subtree whose size violates
↪  alpha-balance; amortized O(log n), no rotations.
# Skip list: randomized levels, expected O(log n) search; no
↪  advantage over SortedList in Python.
```

– 4.8 Advanced and specialized structures

```
# Sqrt tree: O(1) query for any associative op with O(n log log
↪  n) build; DisjointSparseTable
#   (4.5) gives O(1) with O(n log n) build, enough in practice.
# WaveletMatrix: nonneg ints. kth(l,r,k) k-th smallest in
↪  a[l:r]; count_less(l,r,x) #{a[i]<x};
# rank(x, r) = #x in a[:r]. Build O(n log V), queries O(log V).
↪  Range freq [lo,hi) = cl(hi)-cl(lo).
class WaveletMatrix:
  def __init__(self, a):
    self.lg = lg = max(1, max(a, default=0).bit_length()); n =
    ↪  len(a)
    self.pre = []; self.z = []; cur = list(a)
    for b in range(lg - 1, -1, -1):
      p = [0] * (n + 1); s = 0; zs = []; os_ = []
      for i, x in enumerate(cur):
        if x >> b & 1: s += 1; os_.append(x)
        else: zs.append(x)
        p[i + 1] = s
      self.pre.append(p); self.z.append(len(zs)); cur = zs + os_
  def kth(self, l, r, k):
    res = 0
    for lv in range(self.lg):
      p = self.pre[lv]; ol, orr = p[l], p[r]; zc = (r - l) -
↪  (orr - ol)
      if k < zc: l -= ol; r -= orr
      else: k -= zc; res |= 1 << (self.lg - 1 - lv); l =
      ↪  self.z[lv] + ol; r = self.z[lv] + orr
    return res
  def count_less(self, l, r, x):
    if x >= 1 << self.lg: return r - l
    if x <= 0: return 0
    res = 0
    for lv in range(self.lg):
      p = self.pre[lv]; ol, orr = p[l], p[r]
      if x >> (self.lg - 1 - lv) & 1: res += (r - l) - (orr -
      ↪  ol); l = self.z[lv] + ol; r = self.z[lv] + orr
      else: l -= ol; r -= orr
    return res
  def rank(self, x, r): return self.count_less(0, r, x + 1) -
  ↪  self.count_less(0, r, x)
# Wavelet tree: same idea as a tree over value ranges; matrix is
↪  simpler & faster.
# van Emde Boas: O(log log U) pred/succ via sqrt(U) clusters +
↪  summary (conceptual). In Python a
#   64-ary bitset trie (levels of ints, use bit_length) gives
↪  fast pred/succ for U ~ 1e6..1e7.
# LinkCutTree: dynamic forest, nodes 1..n. link(u,v) (different
↪  trees), cut(u,v) (edge exists),
# connected, path_sum(u,v), set_val. O(log n) amortized;
↪  iterative splay. Non-commutative
# aggregate: also keep reversed aggregate and swap it in _push.
class LinkCutTree:
  def __init__(self, n, vals=None):
    self.L = [0] * (n + 1); self.R = [0] * (n + 1); self.P = [0]
    ↪  * (n + 1); self.rev = [0] * (n + 1)
    self.val = [0] + (list(vals) if vals else [0] * n); self.agg
    ↪  = self.val[:]
  def _isroot(self, x):
    p = self.P[x]; return p == 0 or (self.L[p] != x and
↪  self.R[p] != x)
  def _pull(self, x): self.agg[x] = self.agg[self.L[x]] +
  ↪  self.val[x] + self.agg[self.R[x]]
  def _push(self, x):
    if self.rev[x]:
      l, r = self.L[x], self.R[x]; self.L[x], self.R[x] = r, l
      self.rev[l] ^= 1; self.rev[r] ^= 1; self.rev[x] = 0;
      ↪  self.rev[0] = 0
  def _rot(self, x):
    L, R, P = self.L, self.R, self.P; p = P[x]; g = P[p]
    if not self._isroot(p):
      if L[g] == p: L[g] = x
      else: R[g] = x
    P[x] = g
    if L[p] == x: b = R[x]; L[p] = b; R[x] = p
    else: b = L[x]; R[p] = b; L[x] = p
    if b: P[b] = p
    P[p] = x; self._pull(p); self._pull(x)
  def _splay(self, x):
    st = [x]; y = x
    while not self._isroot(y): y = self.P[y]; st.append(y)
    for y in reversed(st): self._push(y)
    P, L = self.P, self.L
    while not self._isroot(x):
      p = P[x]
      if not self._isroot(p):
        g = P[p]
        self._rot(p if (L[g] == p) == (L[p] == x) else x)
      self._rot(x)
  def _access(self, x):
    last, y = 0, x
    while y: self._splay(y); self.R[y] = last; self._pull(y);
    ↪  last = y; y = self.P[y]
    self._splay(x)
  def evert(self, x): self._access(x); self.rev[x] ^= 1  # make
  ↪  x the root of its tree
  def find_root(self, x):
    self._access(x)
    while True:
      self._push(x)
      if not self.L[x]: break
      x = self.L[x]
    self._splay(x); return x
  def connected(self, u, v): return u == v or self.find_root(u)
  ↪  == self.find_root(v)
  def link(self, u, v): self.evert(u); self.P[u] = v
  def cut(self, u, v):
    self.evert(u); self._access(v)
    self.L[v] = 0; self.P[u] = 0; self._pull(v)
  def path_sum(self, u, v): self.evert(u); self._access(v);
  ↪  return self.agg[v]
  def set_val(self, x, v): self._access(x); self.val[x] = v;
  ↪  self._pull(x)
# Euler tour tree: forest as Euler tour sequences in a balanced
↪  BST (treap keyed by implicit
#   position); link/cut = O(1) splits/merges; subtree
↪  aggregates & connectivity in O(log n).
# Top trees: cluster tree over contractions (rake/compress);
↪  path + subtree queries on dynamic
#   forests in O(log n). Conceptual; LCT with virtual-subtree
↪  sums covers most uses.
# Persistent structures: path copying (copy the O(log n) nodes
↪  on the update path, keep old root;
#   see PersistentKth) or fat nodes (each field stores
↪  (version, value) list, bisect by version).
#   Persistent array = persistent segment tree with leaves as
↪  cells. Treap persistence: copy in split/merge.
# Retroactive structures: insert/delete operations in the past;
↪  partial (query present) vs full.
#   Conceptual; in contests do offline: segment tree over time +
↪  rollback (RollbackDSU).
# Rope: sequence with O(log n) cut/concat/insert of substrings
↪  = ImplicitTreap (4.7).
# cartesian_tree: O(n) stack build of min-Cartesian tree;
↪  returns parent (-1 for root).
# Root = leftmost minimum (ties: earlier index is ancestor).
↪  RMQ(l,r) = LCA(l,r) in this tree,
# and LCA reduces to RMQ on Euler tour (see 06_trees.py).
def cartesian_tree(a):
  n = len(a); par = [-1] * n; st = []
  for i in range(n):
    last = -1
    while st and a[st[-1]] > a[i]: last = st.pop()
    if last != -1: par[last] = i
    if st: par[i] = st[-1]
    st.append(i)
  return par
# Bitset via Python int: b |= b << w (subset sum), b & mask,
↪  popcount int.bit_count() (3.10+) or
#   bin(b).count('1'), lowest bit (b & -b).bit_length()-1,
↪  highest b.bit_length()-1. Shifts/ands
#   are C-speed (~n/64 words): n^2/64-style algorithms
↪  (reachability rows, triangle counting).
# Fractional cascading: k sorted lists searched for x in O(log
↪  n + k) by storing every 2nd element
#   of list i+1 inside list i with bridge pointers. Mostly
↪  conceptual; merge sort tree with it
#   gives O(log n) per query. In Python, bisect on each list is
↪  usually fast enough.
```

03 Dynamic programming

```
from bisect import bisect_left, bisect_right
from collections import deque
import heapq
```

– 5.1 Fundamentals: memoization vs tabulation

```
# Memo: @functools.cache (or lru_cache(None)) on f(state).
↪  Easy, but each call ~1us+, hashing
#   tuples is slow, and depth > ~1e4 needs
↪  sys.setrecursionlimit + bigger stack
#   (threading.stack_size(1<<27); run in a Thread) -- on PyPy
↪  deep recursion may still crash.
#   Call f.cache_clear() between test cases. Encode state as
↪  one int (i*M+j) for speed.
# Tabulation: loop states in topological order (dependencies
↪  first). Preferred in Python:
#   flat lists, no call overhead, enables rolling arrays. Use
↪  memo only when reachable
#   states are sparse or the order is awkward.
# State design: "what minimal info about the prefix/subproblem
↪  determines the future?"
```

– 5.1 Pull vs push DP

```
# Pull: dp[i] = combine(dp[j] for predecessors j)    -- natural
↪  when predecessors are easy to list.
# Push: for each i (already final), dp[j] op= f(dp[i]) for
↪  successors j -- natural for
```

```
#   "from state i I can go to ...", probability/expectation
↪  forward, and skipping dp[i]==0/INF.
```

– 5.1 Space optimization (rolling arrays)

```
# dp[i][*] depends only on dp[i-1][*] -> keep prev/cur (cur = [0]*m each row, or swap).
# 1D in-place: iterate j DESCENDING to use old values (0/1 knapsack), ASCENDING to reuse the
#   updated row (unbounded knapsack). Reconstruction needs full table (or Hirschberg).
```

– 5.1 Reconstructing solutions: LCS string

```
# O(nm) time & memory. Returns one longest common subsequence (works for str or list).
def lcs_string(a, b):
  n, m = len(a), len(b)
  dp = [[0] * (m + 1) for _ in range(n + 1)]
  for i in range(n):
    ai, row, nrow = a[i], dp[i], dp[i + 1]
    for j in range(m):
      nrow[j + 1] = row[j] + 1 if ai == b[j] else max(row[j + 1], nrow[j])
  res, i, j = [], n, m
  while i and j:  # walk back from (n,m)
    if a[i - 1] == b[j - 1]:
      res.append(a[i - 1]); i -= 1; j -= 1
    elif dp[i - 1][j] >= dp[i][j - 1]:
      i -= 1
    else:
      j -= 1
  res.reverse()
  return ''.join(res) if isinstance(a, str) else res
```

– 5.2 LCS length (rolling)

```
# O(nm) time, O(m) memory.
def lcs_len(a, b):
  m = len(b)
  prev = [0] * (m + 1)
  for x in a:
    cur = [0] * (m + 1)
    for j in range(m):
      cur[j + 1] = prev[j] + 1 if x == b[j] else max(prev[j + 1], cur[j])
    prev = cur
  return prev[m]
```

– 5.2 Fibonacci-style recurrences / counting paths / grid DP

```
# Count right/down paths from (0,0) to (n-1,m-1) avoiding '#'. O(nm). Fibonacci-like
# linear recurrences with huge n -> see 09_algebra_poly.py: mat_pow.
def grid_paths(grid, mod=None):
  m = len(grid[0])
  dp = [0] * m
  dp[0] = 1 if grid[0][0] != '#' else 0
  for r, row in enumerate(grid):
    for c in range(m):
      if row[c] == '#':
        dp[c] = 0
      elif c:
        dp[c] += dp[c - 1]  # dp[c] (from above) + left
        if mod: dp[c] %= mod
  return dp[m - 1]
# Min-cost right/down path sum. O(nm).
def grid_min_path(cost):
  m = len(cost[0])
  dp = [INF] * m
  dp[0] = 0
  for row in cost:
    dp[0] += row[0]
    for c in range(1, m):
      dp[c] = min(dp[c], dp[c - 1]) + row[c]
  return dp[m - 1]
```

– 5.2 Coin change

```
# Number of ways (order doesn't matter: coins outer loop). O(len(coins)*T).
# For ordered sequences (compositions) swap loops: for t: for c.
def coin_ways(coins, T, mod=None):
  dp = [0] * (T + 1)
  dp[0] = 1
  for c in coins:
    for t in range(c, T + 1):
      dp[t] += dp[t - c]
      if mod: dp[t] %= mod
  return dp[T]
# Min number of coins to make T (unbounded); -1 if impossible. O(len(coins)*T).
def coin_min(coins, T):
  dp = [0] + [INF] * T
  for c in coins:
    for t in range(c, T + 1):
      if dp[t - c] + 1 < dp[t]:
        dp[t] = dp[t - c] + 1
  return dp[T] if dp[T] != INF else -1
```

– 5.2 Knapsack

```
# 0/1 knapsack, items=[(w,v)], max value with weight <= W. O(nW). j descending!
# For "exactly W" init dp=[0]+[-INF]*W.
def knap01(items, W):
  dp = [0] * (W + 1)
  for w, v in items:
    for j in range(W, w - 1, -1):
      if dp[j - w] + v > dp[j]:
        dp[j] = dp[j - w] + v
  return dp[W]
# Unbounded knapsack. O(nW). j ascending.
def knap_unbounded(items, W):
  dp = [0] * (W + 1)
  for w, v in items:
    for j in range(w, W + 1):
      if dp[j - w] + v > dp[j]:
        dp[j] = dp[j - w] + v
  return dp[W]
# Bounded knapsack via binary splitting, items=[(w,v,cnt)]. O(W * sum log cnt).
def knap_bounded_binary(items, W):
  dp = [0] * (W + 1)
  for w, v, c in items:
    k = 1
    while c > 0:
      t = min(k, c); c -= t; k <<= 1
      tw, tv = t * w, t * v
      for j in range(W, tw - 1, -1):
        if dp[j - tw] + tv > dp[j]:
          dp[j] = dp[j - tw] + tv
  return dp[W]
# Bounded knapsack via monotonic queue per residue mod w. O(nW).
# new[r+k*w] = k*v + max_{k-c<=t<=k} (dp[r+t*w] - t*v)
def knap_bounded_mq(items, W):
  dp = [0] * (W + 1)
  for w, v, c in items:
    new = dp[:]
    for r in range(min(w, W + 1)):
      q = deque()  # (t, value) with decreasing value
      t = 0
      for j in range(r, W + 1, w):
        val = dp[j] - t * v
        while q and q[-1][1] <= val:
          q.pop()
        q.append((t, val))
        if q[0][0] < t - c:
          q.popleft()
        new[j] = q[0][1] + t * v
        t += 1
    dp = new
  return dp[W]
```

– 5.2 Longest increasing subsequence

```
# O(n log n). Returns list of indices of one LIS. strict=False -> non-decreasing.
# O(n^2) version: dp[i] = 1 + max(dp[j] for j<i if a[j]<a[i]).
def lis(a, strict=True):
  find = bisect_left if strict else bisect_right
  tails, tidx, par = [], [], [-1] * len(a)
  for i, x in enumerate(a):
    p = find(tails, x)
    if p == len(tails):
      tails.append(x); tidx.append(i)
    else:
      tails[p] = x; tidx[p] = i
    par[i] = tidx[p - 1] if p else -1
  res, i = [], tidx[-1] if tidx else -1
  while i != -1:
    res.append(i); i = par[i]
  return res[::-1]
```

– 5.2 Edit distance (Levenshtein)

```
# O(nm) time, O(m) memory. Insert/delete/substitute cost 1.
def edit_distance(a, b):
  m = len(b)
  prev = list(range(m + 1))
  for i, x in enumerate(a, 1):
    cur = [i] + [0] * m
    for j in range(1, m + 1):
      cur[j] = min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (x != b[j - 1]))
    prev = cur
  return prev[m]
```

– 5.2 Matrix chain multiplication

```
# dims: matrix i is dims[i] x dims[i+1]. O(n^3). Min scalar multiplications.
def matrix_chain(dims):
  n = len(dims) - 1
  dp = [[0] * n for _ in range(n)]
  for length in range(2, n + 1):
    for i in range(n - length + 1):
      j = i + length - 1
      di, dj = dims[i], dims[j + 1]
      dp[i][j] = min(dp[i][k] + dp[k + 1][j] + di * dims[k + 1] * dj for k in range(i, j))
  return dp[0][n - 1] if n else 0
```

– 5.2 Subset sum with Python-int bitset

```
# Bit s of result set <=> some subset sums to s. O(n * S / 64) -- very fast.
# Mask with (1 << (S+1)) - 1 if only sums <= S matter (keeps ints small).
def subset_sums(a, S=None):
  bits = 1
  mask = (1 << (S + 1)) - 1 if S is not None else -1
  for x in a:
    bits |= bits << x
    if S is not None: bits &= mask
  return bits  # test: bits >> s & 1
```

– 5.2 Partition problems

```
# Min |sum(A) - sum(B)| over 2-partitions. O(n*sum/64).
# Many equal values: group counts + binary splitting (as knap_bounded_binary) first.
def min_partition_diff(a):
  tot = sum(a)
  bits = subset_sums(a, tot // 2)
  return tot - 2 * (bits.bit_length() - 1)  # highest reachable sum <= tot//2
```

– 5.3 Interval DP

```
# Merge adjacent piles, cost = size of merged pile; min total. O(n^3). (Knuth -> O(n^2) below.)
# Pattern: for length: for i: j=i+len-1: dp[i][j] = min over split k.
def merge_stones(a):
  n = len(a)
  P = [0] * (n + 1)
  for i, x in enumerate(a): P[i + 1] = P[i] + x
  dp = [[0] * n for _ in range(n)]
  for length in range(2, n + 1):
    for i in range(n - length + 1):
      j = i + length - 1
      dp[i][j] = min(dp[i][k] + dp[k + 1][j] for k in range(i, j)) + P[j + 1] - P[i]
  return dp[0][n - 1] if n else 0
# Burst balloons: bursting k gains l*k*r of current neighbours. O(n^3).
# Trick: choose the LAST balloon k burst in open interval (l, r).
def burst_balloons(a):
  v = [1] + a + [1]
  n = len(v)
  dp = [[0] * n for _ in range(n)]
  for length in range(2, n):
    for l in range(n - length):
      r = l + length
      vl, vr, best = v[l], v[r], 0
      for k in range(l + 1, r):
        t = dp[l][k] + dp[k][r] + vl * v[k] * vr
        if t > best: best = t
      dp[l][r] = best
  return dp[0][n - 1]
```

– 5.3 Tree DP (iterative): max weight independent set

```
# O(n). adj: adjacency lists, w: weights (default 1). Process vertices in reverse BFS order.
def tree_mis(n, adj, w=None, root=0):
  w = w or [1] * n
  par = [-1] * n
  order = [root]
  par[root] = root
  for v in order:
    for c in adj[v]:
      if par[c] == -1 and c != root:
        par[c] = v; order.append(c)
  take, skip = w[:], [0] * n
  for v in reversed(order):
    if v != root:
      p = par[v]
      take[p] += skip[v]
      skip[p] += max(take[v], skip[v])
  return max(take[root], skip[root])
```

– 5.3 Rerooting (all-roots DP), generic iterative

```
# O(n * cost(merge)). merge must be associative with identity e.
#   lift(acc, v): acc = merge of neighbour values around v (excluding the parent side)
#                 -> value of "subtree hanging at v" as seen from its parent.
# Returns res[v] = merge of ALL neighbour values of v (finalize yourself, e.g. lift(res[v], v)).
# Example (sum of distances): e=(0,0), merge=add pairwise, lift=(s+1, d+s+1), ans=res[v][1].
def reroot(n, adj, e, merge, lift, root=0):
  par = [-1] * n
  par[root] = root
  order = [root]
  for v in order:
    for c in adj[v]:
      if par[c] == -1:
        par[c] = v; order.append(c)
  par[root] = -1
  down = [e] * n
  for v in reversed(order):
    acc = e
    for c in adj[v]:
      if c != par[v]:
        acc = merge(acc, down[c])
    down[v] = lift(acc, v)
  up = [e] * n  # up[v]: value of the parent-side part, seen from v
  res = [e] * n
  for v in order:
    nb = adj[v]
    pv = par[v]
    vals = [up[v] if c == pv else down[c] for c in nb]
    k = len(nb)
    pre = [e] * (k + 1)
    for i in range(k):
      pre[i + 1] = merge(pre[i], vals[i])
    res[v] = pre[k]
    suf = e
    for i in range(k - 1, -1, -1):
      c = nb[i]
      if c != pv:
        up[c] = lift(merge(pre[i], suf), v)
      suf = merge(vals[i], suf)
  return res
```

– 5.3 DAG DP

```
# Any DP is a DAG over states. Explicit DAG: topological order (Kahn, see 04_graphs.py),
# then dp[v] = f(dp[u] for u->v) (longest path, #paths mod p). Cycles -> SCC condense first.
```

– 5.3 Bitmask DP

```
# TSP: min Hamiltonian cycle from 0. O(2^n n^2). dist: n x n matrix.
# Hamiltonian path variant: init dp[1<<v][v]=0 for all v, answer min dp[full][v].
def tsp(dist):
  n = len(dist)
  full = 1 << n
  dp = [[INF] * n for _ in range(full)]
  dp[1][0] = 0
  for mask in range(1, full, 2):  # masks containing 0
    row = dp[mask]
    for v in range(n):
      cur = row[v]
      if cur == INF or not (mask >> v & 1): continue
      dv = dist[v]
      for u in range(n):
        if not (mask >> u & 1):
          nm = mask | 1 << u
          if cur + dv[u] < dp[nm][u]:
            dp[nm][u] = cur + dv[u]
  return min(dp[full - 1][v] + dist[v][0] for v in range(n)) if n > 1 else 0
# Assignment: worker popcount(mask) takes a job in mask. O(2^n n). (Hungarian: 05_flow_matching.py)
def assignment_bitmask(cost):
```

```
  n = len(cost)
  dp = [INF] * (1 << n)
  dp[0] = 0
  for mask in range(1 << n):
    cur = dp[mask]
    i = bin(mask).count('1')
    if cur == INF or i == n: continue
    row = cost[i]
    for j in range(n):
      if not mask >> j & 1:
        nm = mask | 1 << j
        if cur + row[j] < dp[nm]:
          dp[nm] = cur + row[j]
  return dp[(1 << n) - 1]
```

– 5.3 Digit DP

```
# Count x in [0, N] with digit sum == S. Iterative with tight
↪  flag: O(len(N) * S * 10).
# Range [L,R]: f(R) - f(L-1). Generic: state = (pos, tight,
↪  extra); non-tight states pooled.
def count_digit_sum(N, S):
  if N < 0: return 0
  cnt = [0] * (S + 1)  # non-tight prefixes by digit sum
  tsum = 0  # digit sum of the tight prefix
  for d in map(int, str(N)):
    new = [0] * (S + 1)
    for s in range(S + 1):
      c = cnt[s]
      if c:
        for x in range(min(9, S - s) + 1):
          new[s + x] += c
    for x in range(d):  # leave tight by placing x < d
      if tsum + x <= S: new[tsum + x] += 1
    tsum += d
    cnt = new
  return cnt[S] + (tsum == S)
```

– 5.3 Probability / expected value DP

```
# Expected number of fair-die rolls until position >= n (start
↪  0). O(6n).
# E[i] = 1 + sum_d E[i+d]/6 computed backwards (states with no
↪  future first).
# Mod p version: replace /6 with * pow(6, p-2, p). Cyclic
↪  dependencies -> Gaussian elimination.
def expected_rolls(n, frac=False):
  from fractions import Fraction
  one = Fraction(1) if frac else 1.0
  E = [one * 0] * (n + 6)
  for i in range(n - 1, -1, -1):
    E[i] = one + sum(E[i + 1:i + 7]) / 6
  return E[0]
```

– 5.3 Game DP

```
# Players alternately take from either end of a; returns (first
↪  - second) under optimal play.
# O(n^2). General: win[state] = any(not win[next]) over moves
↪  (Sprague-Grundy: see 08_math_nt_comb.py 9.6).
def ends_game(a):
  n = len(a)
  dp = a[:]  # dp[i] for intervals of current length starting at
↪  i
  for length in range(2, n + 1):
    dp = [max(a[i] - dp[i + 1], a[i + length - 1] - dp[i]) for i
↪  in range(n - length + 1)]
  return dp[0] if n else 0
```

– 5.3 DP on subsequences and strings

```
# Count distinct subsequences: dp = 2*dp -
↪  dp_before_last_occurrence. O(n). (includes empty)
def count_distinct_subseq(s, mod=None):
  dp, last = 1, {}
  for ch in s:
    prev = dp
    dp = 2 * dp - last.get(ch, 0)
    if mod: dp %= mod
    last[ch] = prev
  return dp
```

– 5.3 Broken profile DP: domino tilings of n x m

```
# Cell-by-cell, mask bit j = cell in column j of the frontier
↪  already covered. O(n m 2^m).
def domino_tilings(n, m):
  if m > n: n, m = m, n
  dp = [0] * (1 << m)
  dp[0] = 1
  for i in range(n):
    for j in range(m):
      new = [0] * (1 << m)
      bj = 1 << j
      for mask in range(1 << m):
        c = dp[mask]
        if not c: continue
        if mask & bj:
          new[mask ^ bj] += c  # already covered from above
        else:
          if i + 1 < n: new[mask | bj] += c  # vertical, covers
          ↪  (i+1,j)
          if j + 1 < m and not mask >> (j + 1) & 1:
            new[mask | bj << 1] += c  # horizontal, covers (i,j+1)
      dp = new
  return dp[0]
```

– 5.3 SOS DP (zeta / Mobius over subsets)

```
# In place, O(n 2^n). len(f) == 1<<n.
def sos_subset(f, n):  # f[S] = sum_{T subset S} f[T]
  for i in range(n):
    b = 1 << i
    for S in range(1 << n):
      if S & b: f[S] += f[S ^ b]
  return f

def sos_superset(f, n):  # f[S] = sum_{T superset S} f[T]
  for i in range(n):
    b = 1 << i
    for S in range(1 << n):
      if not S & b: f[S] += f[S | b]
  return f

def mobius_subset(f, n):  # inverse of sos_subset
  for i in range(n):
    b = 1 << i
    for S in range(1 << n):
      if S & b: f[S] -= f[S ^ b]
  return f
# Subset convolution / FWHT: see 09_algebra_poly.py. Enumerate
↪  submasks: s=m; while s: ...; s=(s-1)&m (O(3^n) total).
```

– 5.3 DP on permutations: insertion / "connected components" DP

```
# Insertion DP: build permutation by inserting values 1..n (or
↪  positions) one at a time;
#   state tracks only what matters (e.g. # descents, last
↪  relative rank -> dp[i][j] with prefix sums).
# Components DP: insert values in increasing order; state =
↪  #components (blocks to be glued).
#   New value: open new component, append to an end of a
↪  component, or merge two.
# Example (CEOI16 kangaroo): count zigzag permutations of 1..n
↪  with p[0]=s, p[-1]=t. O(n^2).
def zigzag_perms(n, s, t, mod=None):
  dp = [1] + [0] * (n + 1)  # dp[j]: j components
  for i in range(1, n + 1):
    new = [0] * (n + 2)
    for j in range(1, i + 1):
      if i == s or i == t:  # endpoint: new comp at its end, or
      ↪  attach to comp at that end
        v = dp[j - 1] + dp[j]
      else:  # new comp in free gaps, or merge two (zigzag
      ↪  forbids append)
        v = dp[j - 1] * (j - (i > s) - (i > t)) + dp[j + 1] * j
      new[j] = v % mod if mod else v
    dp = new
  return dp[1]
```

– 5.3 DP with data structures (BIT transitions)

```
# Count strictly increasing subsequences (non-empty). dp[i] = 1
↪  + sum dp[j], j<i, a[j]<a[i];
# Fenwick over compressed values -> O(n log n). Same pattern:
↪  max-weight LIS with a max-BIT,
# or range-of-j transitions with a segment tree
↪  (02_data_structures.py).
def count_increasing_subseq(a, mod=None):
  vals = sorted(set(a))
  rk = {v: i + 1 for i, v in enumerate(vals)}
  m = len(vals)
  bit = [0] * (m + 1)
  total = 0
  for x in a:
    i = rk[x] - 1  # prefix sum over ranks < x
    s = 1
    while i > 0:
      s += bit[i]; i &= i - 1
    if mod: s %= mod
    total += s
    i = rk[x]
    while i <= m:
      bit[i] += s
      if mod: bit[i] %= mod
      i += i & -i
  return total % mod if mod else total
```

– 5.3 Counting DP with inclusion-exclusion

```
# #objects avoiding all "bad" properties = sum_{S subset bad}
↪  (-1)^|S| #(objects having all of S).
# DP form: sort bad points, g[i] = ways(start->p_i) - sum_{j
↪  before i} g[j]*ways(p_j->p_i)
#   (first-bad-point decomposition, e.g. grid paths avoiding k
↪  blocked cells in O(k^2)).
# "Exactly k" from "at least k": e_k = sum_{j>=k} (-1)^(j-k)
↪  C(j,k) a_j (binomial inversion).
```

– 5.3 Plug DP (connectivity profile DP)

```
# Broken profile where the mask stores, per frontier edge, a
↪  plug id / bracket type
# (0 none, 1 '(' , 2 ')') encoding which plugs connect ->
↪  Hamiltonian cycles, single loop
# tiling on grids. States kept in dict {encoded_profile:
↪  count}; normalize labels each step.
# Cases at each cell: (none,none) open pair; one plug ->
↪  extend; two plugs -> merge (fix match).
```

– 5.4 Prefix sum optimization

```
# dp[i][j] = sum_{k=j-L..j} dp[i-1][k] -> keep prefix sums of
↪  row i-1: O(1) per state.
# Same for min/max over sliding ranges (monotonic queue) and
↪  for "sum over all j<i" (running total).
```

– 5.4 Monotonic queue optimization

```
# dp[i] = c[i] + min(dp[i-k..i-1]): choose positions so every
↪  window of k consecutive has
# a chosen one; min total cost. O(n). Answer = min over last k
↪  dp's.
def min_cost_cover_windows(c, k):
  n = len(c)
  dp = [0] * n
  q = deque()  # indices with increasing dp
  for i in range(n):
    while q and q[0] < i - k:
      q.popleft()
    dp[i] = c[i] + (dp[q[0]] if i >= k else 0)  # first pick
↪  must be < k
    while q and dp[q[-1]] >= dp[i]:
      q.pop()
    q.append(i)
  return min(dp[max(0, n - k):])
```

– 5.4 Convex Hull Trick (monotone slopes)

```
# Min of lines y = m*x + b. Add lines in DECREASING slope order
↪  (for max: negate m,b and result).
# add amortized O(1); query: qmono O(1) amortized for
↪  non-decreasing x, query O(log n) any x.
class CHT:
  def __init__(self):
    self.M, self.B, self.p = [], [], 0
  def add(self, m, b):
    M, B = self.M, self.B
    if M and M[-1] == m:
      if b >= B[-1]: return
      M.pop(); B.pop()
    while len(M) >= 2 and (b - B[-2]) * (M[-2] - M[-1]) <=
    ↪  (B[-1] - B[-2]) * (M[-2] - m):
      M.pop(); B.pop()
    M.append(m); B.append(b)
  def query(self, x):  # binary search, any x
    M, B = self.M, self.B
    lo, hi = 0, len(M) - 1
    while lo < hi:
      mid = (lo + hi) // 2
      if M[mid] * x + B[mid] >= M[mid + 1] * x + B[mid + 1]: lo
      ↪  = mid + 1
      else: hi = mid
    return M[lo] * x + B[lo]
  def qmono(self, x):  # x non-decreasing across calls
    M, B = self.M, self.B
    p = min(self.p, len(M) - 1)
    while p + 1 < len(M) and M[p + 1] * x + B[p + 1] <= M[p] * x
    ↪  + B[p]:
      p += 1
    self.p = p
    return M[p] * x + B[p]
# Arbitrary slope order / online: Li Chao tree -> see
↪  02_data_structures.py: LiChao
```

– 5.4 Divide and conquer optimization

```
# new[i] = min_{j<i} prev[j] + cost(j, i), when opt(i) is
↪  monotone (cost satisfies QI).
# O(n log n) per layer with explicit stack (no recursion).
↪  cost(j,i): segment (j, i].
def dc_layer(prev, cost, n):
  new = [INF] * (n + 1)
  st = [(1, n, 0, n - 1)]
  while st:
    lo, hi, ol, orr = st.pop()
    if lo > hi: continue
    mid = (lo + hi) // 2
    best, bj = INF, ol
    for j in range(ol, min(mid - 1, orr) + 1):
      v = prev[j] + cost(j, mid)
      if v < best: best, bj = v, j
    new[mid] = best
    st.append((lo, mid - 1, ol, bj))
    st.append((mid + 1, hi, bj, orr))
  return new
# Example: split a into k contiguous groups minimizing sum of
↪  (group sum)^2. O(k n log n).
def partition_k_squares(a, k):
  n = len(a)
  P = [0] * (n + 1)
  for i, x in enumerate(a): P[i + 1] = P[i] + x
  cost = lambda j, i: (P[i] - P[j]) ** 2
  dp = [0] + [INF] * n
  for _ in range(k):
    dp = dc_layer(dp, cost, n)
    dp[0] = INF
  return dp[n]
```

– 5.4 Knuth optimization

```
# dp[i][j] = min_{i<=k<j} dp[i][k]+dp[k+1][j] + C(i,j), with
↪  opt[i][j-1] <= opt[i][j] <= opt[i+1][j]
# (C satisfies QI + monotone). O(n^2). Example: merging
↪  adjacent piles (cf. merge_stones).
# Optimal BST is identical in shape (C(i,j) = sum of
↪  frequencies in [i,j]).
def merge_stones_knuth(a):
  n = len(a)
  if n == 0: return 0
  P = [0] * (n + 1)
  for i, x in enumerate(a): P[i + 1] = P[i] + x
  dp = [[0] * n for _ in range(n)]
  opt = [[0] * n for _ in range(n)]
  for i in range(n): opt[i][i] = i
  for length in range(2, n + 1):
    for i in range(n - length + 1):
      j = i + length - 1
      best, bk = INF, i
      dpi = dp[i]
      for k in range(opt[i][j - 1], min(opt[i + 1][j], j - 1) +
      ↪  1):
        v = dpi[k] + dp[k + 1][j]
        if v < best: best, bk = v, k
      dp[i][j] = best + P[j + 1] - P[i]
      opt[i][j] = bk
  return dp[0][n - 1]
```

– 5.4 Aliens trick / WQS binary search / Lagrangian relaxation

```
# Need g(k) = best with EXACTLY k parts, g convex in k. Solve
↪  h(lam) = min (cost + lam*#parts)
# without k (tie-break: fewest parts); find smallest lam with
↪  cnt <= k; answer h - lam*k.
# Integer costs -> integer lam suffices. Inner DP here O(n^2)
↪  (use CHT for O(n)).
# Example: same as partition_k_squares (a positive), total
↪  O(n^2 log C).
def aliens_partition(a, k):
  n = len(a)
  P = [0] * (n + 1)
  for i, x in enumerate(a): P[i + 1] = P[i] + x
  def solve(lam):  # returns (value, #parts) minimal value, min
  ↪  parts
    dp = [0] * (n + 1); cnt = [0] * (n + 1)
    for i in range(1, n + 1):
      best, bc = INF, 0
      for j in range(i):
        v = dp[j] + (P[i] - P[j]) ** 2 + lam
        if v < best or (v == best and cnt[j] + 1 < bc):
          best, bc = v, cnt[j] + 1
      dp[i], cnt[i] = best, bc
    return dp[n], cnt[n]
  lo, hi = 0, P[n] ** 2 + 1  # lam=hi forces 1 part
```

```
  while lo < hi:
    mid = (lo + hi) // 2
    if solve(mid)[1] <= k: hi = mid
    else: lo = mid + 1
  v, _ = solve(lo)
  return v - lo * k
```

– 5.4 Slope trick

```
# Min sum |a_i - b_i| with b non-decreasing. Max-heap of
↪  breakpoints. O(n log n).
# (Strictly increasing: apply to a_i - i.) Generalize: keep L
↪  (max-heap) / R (min-heap) + lazy add.
def min_cost_nondecreasing(a):
  h, ans = [], 0
  for x in a:
    heapq.heappush(h, -x)
    if -h[0] > x:
      ans += -h[0] - x
      heapq.heapreplace(h, -x)
  return ans
```

– 5.4 Matrix exponentiation for linear recurrences

```
# see 09_algebra_poly.py: mat_pow (and Berlekamp-Massey /
↪  Kitamasa there).
```

– 5.4 SMAWK / monotone matrices (conceptual)

```
# Row minima of a totally monotone n x m matrix in O(n+m)
↪  (REDUCE columns to n, recurse on
# even rows, interpolate odd rows). Rarely needed: D&C opt
↪  gives O((n+m) log n) for the same
# monotone-argmin property and is far simpler. LARSCH handles
↪  online dp[i] = min_j dp[j]+w(j,i).
```

– 5.4 Bitset DP speedups

```
# Python int = bitset: shifts/or/and are C-speed over whole
↪  words (~n/64 ops).
#   subset sum: bits |= bits << x;  reachability/transitive
↪  closure: reach[v] |= reach[u];
#   LCS-style bit-parallel (Hyyro): O(n m / 64). Count bits:
↪  int.bit_count() (3.10+) or bin(x).count('1').
#   Keep ints bounded by masking, otherwise they grow without
↪  limit.
```

– 5.4 Open and close interval trick

```
# Sort items; process left to right, state = #currently "open"
↪  groups (+ accumulated cost).
# Cost of a group = max - min = sum of gaps it spans, so when
↪  moving from a[i-1] to a[i]
# add (a[i]-a[i-1]) * open_count. Transitions per item: open
↪  new group / close / add to
# an open one (x open choices) / singleton. Makes "sum over
↪  groups of (max-min) <= K" O(n^2 K).
```

04 Graphs

```
import random
from heapq import heappush, heappop
from collections import deque
from itertools import combinations, permutations, product
```

– 6.1 Graph basics

```
# Adjacency list: g = [[] for _ in range(n)]; g[u].append(v) (+
↪  g[v].append(u) if undirected).
# Matrix: a = [[0]*n for _ in range(n)] (n<=~2000); bitmask
↪  rows adj[u] |= 1<<v for n<=~60 tricks.
# Edge list [(u,v,w)] for Kruskal/Bellman-Ford. Implicit graphs
↪  (grid, states): generate
# neighbours on the fly; encode state as int (r*W+c, mask*n+v)
↪  for flat dist arrays.
# Degrees: sum(deg) = 2m (undirected). Reading fast: data =
↪  sys.stdin.buffer.read().split().

def csr(n, edges):  # O(n+m) compressed adjacency: neighbours
↪  of u are to[start[u]:start[u+1]]
  start = [0] * (n + 1)  # faster/leaner than list-of-lists for
↪  n,m ~ 1e6 (PyPy)
  for u, v in edges:
    start[u + 1] += 1
  for i in range(n):
    start[i + 1] += start[i]
  pos = start[:]; to = [0] * len(edges)
  for u, v in edges:
    to[pos[u]] = v; pos[u] += 1
  return start, to

def bipartite(g):  # O(n+m); returns color list (0/1) or None
↪  if odd cycle
  n = len(g); c = [-1] * n
  for s in range(n):
    if c[s] >= 0: continue
    c[s] = 0; st = [s]
    while st:
      u = st.pop()
      for v in g[u]:
        if c[v] < 0: c[v] = c[u] ^ 1; st.append(v)
        elif c[v] == c[u]: return None
  return c
```

– 6.2 Graph traversal

```
def dfs_times(g):  # O(n+m) iterative DFS on directed g.
↪  kind[u][i] for edge g[u][i]:
  n = len(g)  # 0 tree, 1 back (incl. self-loop), 2 forward, 3
↪  cross. Undirected: only 0/1.
  tin = [-1] * n; tout = [-1] * n; par = [-1] * n; it = [0] * n;
↪  col = [0] * n
  kind = [[0] * len(g[u]) for u in range(n)]; t = 0
  for r in range(n):
    if col[r]: continue
    tin[r] = t; t += 1; col[r] = 1; st = [r]
    while st:
      u = st[-1]
      if it[u] < len(g[u]):
        i = it[u]; v = g[u][i]; it[u] += 1
        if col[v] == 0:
          par[v] = u; tin[v] = t; t += 1; col[v] = 1;
↪  st.append(v)
        else:
          kind[u][i] = 1 if col[v] == 1 else (2 if tin[v] >
↪  tin[u] else 3)
      else:
        st.pop(); col[u] = 2; tout[u] = t; t += 1
  return tin, tout, par, kind  # v in subtree(u) <=> tin[u] <=
↪  tin[v] and tout[v] <= tout[u]
# DFS topological order = vertices sorted by tout descending
↪  (DAG).

def bfs(g, srcs):  # O(n+m) unweighted dist from a list of
↪  sources (multi-source); -1 = unreachable
  n = len(g); d = [-1] * n; par = [-1] * n; q = list(srcs)
  for s in q: d[s] = 0
  for u in q:  # list as queue (appending while iterating is
↪  fine)
    for v in g[u]:
      if d[v] < 0: d[v] = d[u] + 1; par[v] = u; q.append(v)
  return d, par

def bfs01(g, s):  # O(n+m); g[u]=[(v,w)] with w in {0,1}
  d = [INF] * len(g); d[s] = 0; dq = deque([s])
  while dq:
    u = dq.popleft(); du = d[u]
    for v, w in g[u]:
      if du + w < d[v]:
        d[v] = du + w
        if w: dq.append(v)
        else: dq.appendleft(v)
  return d

def flood_fill(grid, free='.'):  # O(RC) 4-dir labelling;
↪  returns (lab, cnt), lab=-1 for walls
  R = len(grid); C = len(grid[0]); lab = [[-1] * C for _ in
↪  range(R)]; cnt = 0
  for i in range(R):
    for j in range(C):
      if grid[i][j] != free or lab[i][j] >= 0: continue
      lab[i][j] = cnt; st = [(i, j)]
      while st:
        r, c = st.pop()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c
↪  - 1)):
          if 0 <= nr < R and 0 <= nc < C and lab[nr][nc] < 0 and
↪  grid[nr][nc] == free:
            lab[nr][nc] = cnt; st.append((nr, nc))
      cnt += 1
  return lab, cnt

def components(g):  # O(n+m) undirected; returns (comp id list,
↪  count)
  n = len(g); comp = [-1] * n; k = 0
  for s in range(n):
    if comp[s] >= 0: continue
    comp[s] = k; st = [s]
    while st:
      u = st.pop()
      for v in g[u]:
        if comp[v] < 0: comp[v] = k; st.append(v)
    k += 1
  return comp, k

def find_cycle_directed(g):  # O(n+m); returns [v0..vk-1] with
↪  v_i->v_{i+1}, v_{k-1}->v0, or None
  n = len(g); col = [0] * n; par = [-1] * n; it = [0] * n
  for r in range(n):
    if col[r]: continue
    col[r] = 1; st = [r]
    while st:
      u = st[-1]
      if it[u] < len(g[u]):
        v = g[u][it[u]]; it[u] += 1
        if col[v] == 0: col[v] = 1; par[v] = u; st.append(v)
        elif col[v] == 1:
          cyc = [u]
          while cyc[-1] != v: cyc.append(par[cyc[-1]])
          return cyc[::-1]
      else:
        col[u] = 2; st.pop()
  return None

def find_cycle_undirected(n, edges):  # O(n+m);
↪  multi-edges/self-loops ok (edge ids); vertex list or None
  g = [[] for _ in range(n)]
  for i, (u, v) in enumerate(edges):
    g[u].append((v, i)); g[v].append((u, i))
  col = [0] * n; par = [-1] * n; pe = [-1] * n; it = [0] * n
  for r in range(n):
    if col[r]: continue
    col[r] = 1; st = [r]
    while st:
      u = st[-1]
      if it[u] < len(g[u]):
        v, e = g[u][it[u]]; it[u] += 1
        if e == pe[u]: continue
        if col[v] == 0: col[v] = 1; par[v] = u; pe[v] = e;
↪  st.append(v)
        elif col[v] == 1:  # back edge to ancestor
          cyc = [u]
          while cyc[-1] != v: cyc.append(par[cyc[-1]])
          return cyc
      else:
        col[u] = 2; st.pop()
  return None
```

– 6.3 Shortest paths

```
def dijkstra(g, s):  # O(m log m) lazy heap; g[u]=[(v,w)],
↪  w>=0; returns (dist, parent)
  n = len(g); d = [INF] * n; p = [-1] * n; d[s] = 0; pq = [(0,
↪  s)]
  while pq:  # speed: push single int du*n+u instead of tuples
  ↪  (if dist ints bounded)
    du, u = heappop(pq)
    if du > d[u]: continue
    for v, w in g[u]:
      nd = du + w
      if nd < d[v]: d[v] = nd; p[v] = u; heappush(pq, (nd, v))
  return d, p

def get_path(p, t):  # path root..t from parent array (check t
↪  reachable first)
  path = []
  while t != -1: path.append(t); t = p[t]
  return path[::-1]

def dijkstra_dense(W, s):  # O(n^2) for dense graphs; W[u][v] =
↪  weight or INF
  n = len(W); d = [INF] * n; d[s] = 0; done = [False] * n
  for _ in range(n):
    u = -1; best = INF
    for v in range(n):
      if not done[v] and d[v] < best: best = d[v]; u = v
    if u < 0: break
    done[u] = True; row = W[u]
    for v in range(n):
      if best + row[v] < d[v]: d[v] = best + row[v]
  return d

def bellman_ford(n, edges, s):  # O(nm); edges (u,v,w). returns
↪  (dist, neg_cycle or None)
  d = [INF] * n; p = [-1] * n; d[s] = 0  # cycle only if
↪  reachable from s; it's a vertex list
  for _ in range(n):  # with edges cyc[i]->cyc[i+1] (cyclic)
    x = -1
    for u, v, w in edges:
      if d[u] + w < d[v]: d[v] = d[u] + w; p[v] = u; x = v
    if x < 0: return d, None
  for _ in range(n): x = p[x]  # step back n times -> surely on
  ↪  the cycle
  cyc = [x]; y = p[x]
  while y != x: cyc.append(y); y = p[y]
  return d, cyc[::-1]
# Vertices with dist -inf: everything reachable from any vertex
↪  relaxed in the n-th pass.
# Detect ANY negative cycle (not only reachable): start with
↪  d=[0]*n (virtual source).
# SPFA = Bellman-Ford with a queue of changed vertices. Avg
↪  fast, worst O(nm) and setters
# hack it (grid + random weights). Prefer Dijkstra
↪  (+potentials) whenever weights allow.

def floyd(d):  # O(n^3) in place; d[i][i]=0, INF if no edge. Neg
↪  cycle through i iff d[i][i] < 0
  n = len(d)
  for k in range(n):
    dk = d[k]
    for i in range(n):
      di = d[i]; dik = di[k]
      if dik == INF: continue
      for j in range(n):
        if dik + dk[j] < di[j]: di[j] = dik + dk[j]
  return d

def transitive_closure(adj):  # O(n^3/64) bitsets; adj[i] mask;
↪  r[i] bit j <=> path i->j (len>=1)
  r = adj[:]; n = len(r)
  for k in range(n):
    bk = 1 << k; rk = r[k]
    for i in range(n):
      if r[i] & bk: r[i] |= rk
  return r

def johnson(n, edges):  # O(nm log m) APSP with negative edges;
↪  matrix (INF unreachable) or None
  h, cyc = bellman_ford(n + 1, edges + [(n, v, 0) for v in
↪  range(n)], n)  # uses bellman_ford, dijkstra
  if cyc: return None
  g = [[] for _ in range(n)]
  for u, v, w in edges: g[u].append((v, w + h[u] - h[v]))  #
  ↪  reduced weights >= 0
  res = []
  for s in range(n):
    d, _ = dijkstra(g, s)
    res.append([d[v] - h[s] + h[v] if d[v] < INF else INF for v
↪  in range(n)])
  return res

def dag_shortest(g, s):  # O(n+m) any weights on DAG;
↪  g[u]=[(v,w)]; longest path: negate weights
  n = len(g); indeg = [0] * n
  for u in range(n):
    for v, _ in g[u]: indeg[v] += 1
  order = [u for u in range(n) if indeg[u] == 0]
  for u in order:
    for v, _ in g[u]:
      indeg[v] -= 1
      if indeg[v] == 0: order.append(v)
  d = [INF] * n; d[s] = 0
  for u in order:
    if d[u] < INF:
      for v, w in g[u]:
        if d[u] + w < d[v]: d[v] = d[u] + w
  return d

def k_shortest_walks(g, s, t, k):  # O(k m log(km)); k smallest
↪  s->t WALK lengths (repeats allowed)
  cnt = [0] * len(g); res = []; pq = [(0, s)]  # each vertex
↪  popped at most k times; w>=0
  while pq and len(res) < k:
    d, u = heappop(pq)
    if cnt[u] >= k: continue
    cnt[u] += 1
    if u == t: res.append(d)
    for v, w in g[u]:
      if cnt[v] < k: heappush(pq, (d + w, v))
  return res
# Simple (loopless) k-shortest paths: Yen = for each prefix of
↪  previous path, ban its next edges
#   + prefix vertices, run Dijkstra for spur; O(k n (m log n)).
↪  Eppstein: O(m + n log n + k log k)
```

```
#   walks via sidetrack-edge heaps on the shortest-path tree
↪  (persistent leftist heaps).
# Layered/state graph: vertex = (v, extra) e.g. (v, coupons
↪  used), (v, parity), (v, time mod k);
#   index id = v*K + extra, run Dijkstra/BFS on the product
↪  graph (n*K vertices).

def diff_constraints(n, cons):  # cons (u,v,c) meaning x_v -
↪  x_u <= c; returns x or None. O(nm)
  d, cyc = bellman_ford(n + 1, list(cons) + [(n, i, 0) for i in
↪  range(n)], n)
  return None if cyc else d[:n]  # solution has max(x)<=0; shift
  ↪  freely. x_v>=x_u+c: (v,u,-c)
# Tight-ness: d gives the max x with x_i <= 0; min of x_t-x_s
↪  subject to constraints: dist s->t.
```

– 6.4 Minimum spanning trees

```
def kruskal(n, edges):  # O(m log m); edges (u,v,w); returns
↪  (weight, used edge ids); forest if disconnected
  p = list(range(n))  # minimal inline DSU (full DSU in 02)
  def find(x):
    while p[x] != x: p[x] = p[p[x]]; x = p[x]
    return x
  tot = 0; used = []
  for i in sorted(range(len(edges)), key=lambda i: edges[i][2]):
    u, v, w = edges[i]; a, b = find(u), find(v)
    if a != b: p[a] = b; tot += w; used.append(i)
  return tot, used

def prim(g):  # O(m log m); g[u]=[(v,w)] undirected connected;
↪  returns weight or None if disconnected
  n = len(g); seen = [False] * n; tot = 0; cnt = 0; pq = [(0, 0)]
  while pq:
    w, u = heappop(pq)
    if seen[u]: continue
    seen[u] = True; tot += w; cnt += 1
    for v, wv in g[u]:
      if not seen[v]: heappush(pq, (wv, v))
  return tot if cnt == n else None  # dense graph: O(n^2) array
  ↪  version like dijkstra_dense

def boruvka(n, edges):  # O(m log n); edges (u,v,w); returns
↪  (weight, used ids). ties broken by id
  p = list(range(n))
  def find(x):
    while p[x] != x: p[x] = p[p[x]]; x = p[x]
    return x
  tot = 0; used = []
  while True:
    best = [-1] * n
    for i, (u, v, w) in enumerate(edges):
      a, b = find(u), find(v)
      if a == b: continue
      for c in (a, b):
        j = best[c]
        if j < 0 or (w, i) < (edges[j][2], j): best[c] = i
    merged = False
    for i in best:
      if i < 0: continue
      u, v, w = edges[i]; a, b = find(u), find(v)
      if a != b: p[a] = b; tot += w; used.append(i); merged =
      ↪  True
    if not merged: return tot, used

def second_mst(n, edges):  # O(n^2 + m log m) STRICTLY
↪  second-best spanning tree weight, or None
  tot, used = kruskal(n, edges)  # uses kruskal. large n: LCA
↪  lifting storing (max, strict 2nd max)
  if len(used) < n - 1: return None
  inT = [False] * len(edges); t = [[] for _ in range(n)]; other
↪  = [[] for _ in range(n)]
  for i in used:
    u, v, w = edges[i]; inT[i] = True; t[u].append((v, w));
↪  t[v].append((u, w))
  for i, (u, v, w) in enumerate(edges):
    if not inT[i] and u != v: other[u].append((v, w))
  best = INF
  for s in range(n):
    if not other[s]: continue
    m1 = [-INF] * n; m2 = [-INF] * n; seen = [False] * n;
↪  seen[s] = True; st = [s]
    while st:  # m1/m2 = max / strictly smaller second max on
    ↪  tree path s..x
      u = st.pop()
      for v, w in t[u]:
        if not seen[v]:
          seen[v] = True; a, b = m1[u], m2[u]
          if w > a: a, b = w, a
          elif a > w > b: b = w
          m1[v], m2[v] = a, b; st.append(v)
    for x, w in other[s]:
      if w > m1[x]: best = min(best, w - m1[x])
      elif m2[x] > -INF: best = min(best, w - m2[x])
  return None if best == INF else tot + best
# Minimum bottleneck spanning tree: any MST is one. Minimax
↪  path u-v (min possible max edge)
#   = max edge on MST path = val[lca] in the Kruskal
↪  reconstruction tree below.

def kruskal_tree(n, edges):  # O(m log m) Kruskal
↪  reconstruction tree. leaves 0..n-1, internal n..
  p = list(range(2 * n)); par = [-1] * (2 * n - 1); val = [0] *
↪  (2 * n - 1); x = n
  def find(a):
    while p[a] != a: p[a] = p[p[a]]; a = p[a]
    return a
  for u, v, w in sorted(edges, key=lambda e: e[2]):
    a, b = find(u), find(v)
    if a != b:
      p[a] = p[b] = x; par[a] = par[b] = x; val[x] = w; x += 1
  return par[:x], val[:x]  # val increases toward root (a heap).
  ↪  minimax(u,v) = val[lca(u,v)];
# vertices reachable from u via edges <= X = leaves under
↪  highest ancestor a of u with val[a] <= X
# (binary lifting, see 06). Max-spanning variant: sort
↪  descending.

def directed_mst(n, edges, root):  # Chu-Liu/Edmonds O(nm);
↪  edges (u,v,w) u->v; min arborescence or None
  res = 0
  while True:
    inw = [INF] * n; pre = [-1] * n
    for u, v, w in edges:
      if u != v and w < inw[v]: inw[v] = w; pre[v] = u
    for v in range(n):
      if v != root and inw[v] == INF: return None
    inw[root] = 0; cid = [-1] * n; vis = [-1] * n; cnt = 0
    for v in range(n):
      res += inw[v]; x = v
      while vis[x] != v and cid[x] == -1 and x != root: vis[x] =
      ↪  v; x = pre[x]
      if x != root and cid[x] == -1:  # found a new cycle
      ↪  through x
        y = pre[x]
        while y != x: cid[y] = cnt; y = pre[y]
        cid[x] = cnt; cnt += 1
    if cnt == 0: return res
    for v in range(n):
      if cid[v] == -1: cid[v] = cnt; cnt += 1
    edges = [(cid[u], cid[v], w - inw[v]) for u, v, w in edges
↪  if cid[u] != cid[v]]
    n = cnt; root = cid[root]
# Edge recovery needs bookkeeping per contraction; O(m log n)
↪  version uses mergeable heaps + DSU.

def steiner_tree(g, terms):  # O(3^k n + 2^k m log m); min
↪  weight tree connecting terminals
  n = len(g); k = len(terms); full = (1 << k) - 1
  dp = [[INF] * n for _ in range(1 << k)]
  for i, t in enumerate(terms): dp[1 << i][t] = 0
  for S in range(1, full + 1):
    d = dp[S]; low = S & -S; sub = (S - 1) & S
    while sub:
      if sub & low:  # each split once
        a = dp[sub]; b = dp[S ^ sub]
        for v in range(n):
          if a[v] + b[v] < d[v]: d[v] = a[v] + b[v]
      sub = (sub - 1) & S
    pq = [(d[v], v) for v in range(n) if d[v] < INF]; pq.sort()
    while pq:
      dv, v = heappop(pq)
      if dv > d[v]: continue
      for u, w in g[v]:
        if dv + w < d[u]: d[u] = dv + w; heappush(pq, (d[u], u))
  return min(dp[full]) if k else 0
```

– 6.5 Directed graphs

```
def toposort(g, lex=False):  # Kahn O(n+m) (lex=True:
↪  smallest-first heap, O(m log n)); None if cycle
  n = len(g); indeg = [0] * n
  for u in range(n):
    for v in g[u]: indeg[v] += 1
  q = [u for u in range(n) if indeg[u] == 0]; order = []
  if lex:
    while q:
      u = heappop(q); order.append(u)  # q already sorted ->
↪  valid heap
      for v in g[u]:
        indeg[v] -= 1
        if indeg[v] == 0: heappush(q, v)
  else:
    order = q
    for u in order:
      for v in g[u]:
        indeg[v] -= 1
        if indeg[v] == 0: order.append(v)
  return order if len(order) == n else None
# Lexicographically smallest order where small LABELS come
↪  early (not same!): reverse graph,
# pop largest first via max-heap, reverse result.

def scc(g):  # iterative Tarjan O(n+m); returns (k, comp); comp
↪  ids in TOPOLOGICAL order (u->v => comp[u]<=comp[v])
  n = len(g); idx = [-1] * n; low = [0] * n; onst = [False] * n;
↪  st = []
  comp = [-1] * n; it = [0] * n; t = 0; k = 0
  for s in range(n):
    if idx[s] >= 0: continue
    idx[s] = low[s] = t; t += 1; st.append(s); onst[s] = True;
↪  cs = [s]
    while cs:
      u = cs[-1]
      if it[u] < len(g[u]):
        v = g[u][it[u]]; it[u] += 1
        if idx[v] < 0:
          idx[v] = low[v] = t; t += 1; st.append(v); onst[v] =
↪  True; cs.append(v)
        elif onst[v] and idx[v] < low[u]: low[u] = idx[v]
      else:
        cs.pop()
        if cs and low[u] < low[cs[-1]]: low[cs[-1]] = low[u]
        if low[u] == idx[u]:
          while True:
            x = st.pop(); onst[x] = False; comp[x] = k
            if x == u: break
          k += 1
  return k, [k - 1 - c for c in comp]
# Kosaraju alternative: order by DFS exit time, then DFS on
↪  reversed graph in decreasing exit.

def condensation(g, k, comp):  # DAG on k SCC nodes
↪  (deduplicated edges), ids already topo-sorted
  seen = set(); dag = [[] for _ in range(k)]
  for u in range(len(g)):
    for v in g[u]:
      a, b = comp[u], comp[v]
      if a != b and (a, b) not in seen: seen.add((a, b));
      ↪  dag[a].append(b)
  return dag

class TwoSat:  # O(n+m). literal: 2*i = x_i, 2*i+1 = not x_i
↪  (neg = lit^1). uses scc above
  def __init__(self, n): self.n = n; self.g = [[] for _ in
  ↪  range(2 * n)]
  def either(self, a, b):  # clause (a or b)
    self.g[a ^ 1].append(b); self.g[b ^ 1].append(a)
  def implies(self, a, b): self.either(a ^ 1, b)
  def force(self, a): self.either(a, a)
  def solve(self):  # list of bools or None
    _, comp = scc(self.g); res = []
    for i in range(self.n):
      if comp[2 * i] == comp[2 * i + 1]: return None
      res.append(comp[2 * i] > comp[2 * i + 1])  # later in topo
↪  order => true
    return res
# at-most-one of many literals: prefix-variable trick (3n
↪  clauses) instead of O(n^2) pairs.

def functional_graph(f):  # O(n); f[v] = successor. returns
↪  (cycles, cyc_id, dist_to_cycle, entry)
  n = len(f); state = [0] * n; cid = [-1] * n; dist = [0] * n;
↪  entry = [-1] * n; cycles = []
  for s in range(n):
    if state[s]: continue
    path = []; x = s
    while state[x] == 0: state[x] = 1; path.append(x); x = f[x]
    if state[x] == 1:  # closed a new cycle starting at x
      i = path.index(x); cyc = path[i:]
      for v in cyc: cid[v] = len(cycles); entry[v] = v
      cycles.append(cyc); tail = path[:i]
    else: tail = path
    for v in reversed(tail): dist[v] = dist[f[v]] + 1; entry[v]
    ↪  = entry[f[v]]
    for v in path: state[v] = 2
  return cycles, cid, dist, entry  # k-th successor: binary
  ↪  lifting, or via tail+cycle index.
# Floyd (tortoise-hare) / Brent: cycle of a single sequence
↪  x->f(x) in O(1) memory.

def dominator_tree(g, root):  # Lengauer-Tarjan O(m log n),
↪  iterative; idom[v] (root: itself, -1 unreachable)
  n = len(g); rg = [[] for _ in range(n)]
  for u in range(n):
    for v in g[u]: rg[v].append(u)
  num = [-1] * n; order = [root]; par = [0]; num[root] = 0; it =
↪  [0] * n; st = [root]
  while st:  # true DFS preorder; vertices renumbered 0..N-1
    u = st[-1]
    if it[u] < len(g[u]):
      v = g[u][it[u]]; it[u] += 1
      if num[v] < 0: num[v] = len(order); order.append(v);
      ↪  par.append(num[u]); st.append(v)
    else: st.pop()
  N = len(order); sdom = list(range(N)); label =
↪  list(range(N)); anc = [-1] * N
  idom = [0] * N; bucket = [[] for _ in range(N)]
  def ev(v):
    if anc[v] < 0: return v
    path = []
    while anc[anc[v]] >= 0: path.append(v); v = anc[v]
    for x in reversed(path):
      a = anc[x]
      if sdom[label[a]] < sdom[label[x]]: label[x] = label[a]
      anc[x] = anc[a]
    return label[path[0]] if path else label[v]
  for w in range(N - 1, 0, -1):
    for pv in rg[order[w]]:
      if num[pv] >= 0:
        u = ev(num[pv])
        if sdom[u] < sdom[w]: sdom[w] = sdom[u]
    bucket[sdom[w]].append(w); p = par[w]; anc[w] = p
    for v in bucket[p]:
      u = ev(v); idom[v] = u if sdom[u] < sdom[v] else p
    bucket[p] = []
  for w in range(1, N):
    if idom[w] != sdom[w]: idom[w] = idom[idom[w]]
  res = [-1] * n
  for w in range(N): res[order[w]] = order[idom[w]]
  return res
```

– 6.6 Connectivity

```
def lowlink(n, edges):  # O(n+m) iterative; undirected,
↪  multi-edges ok (edge ids), self-loops ignored.
  g = [[] for _ in range(n)]  # returns (is_art[v],
↪  is_bridge[e], bccs = vertex lists of blocks)
  for i, (u, v) in enumerate(edges):
    if u != v: g[u].append((v, i)); g[v].append((u, i))
  tin = [-1] * n; low = [0] * n; it = [0] * n; pe = [-1] * n
  is_art = [False] * n; is_bridge = [False] * len(edges); bccs =
↪  []; t = 0
  for r in range(n):
    if tin[r] >= 0: continue
    tin[r] = low[r] = t; t += 1
    if not g[r]: bccs.append([r]); continue  # isolated vertex =
    ↪  own block
    st = [r]; vs = [r]; ch = 0
    while st:
      u = st[-1]
      if it[u] < len(g[u]):
        v, e = g[u][it[u]]; it[u] += 1
        if e == pe[u]: continue
        if tin[v] < 0:
          tin[v] = low[v] = t; t += 1; pe[v] = e; st.append(v);
↪  vs.append(v)
        elif tin[v] < low[u]: low[u] = tin[v]
      else:
        st.pop()
        if not st: break
        p = st[-1]
        if low[u] < low[p]: low[p] = low[u]
        if low[u] > tin[p]: is_bridge[pe[u]] = True
        if low[u] >= tin[p]:
          if p == r: ch += 1
          else: is_art[p] = True
          comp = [p]
```

```
        while True:
          x = vs.pop(); comp.append(x)
          if x == u: break
        bccs.append(comp)
    if ch > 1: is_art[r] = True
  return is_art, is_bridge, bccs

def two_edge_cc(n, edges):  # O(n+m) 2-edge-connected component ids (remove bridges, flood). uses lowlink
  _, br, _ = lowlink(n, edges); g = [[] for _ in range(n)]
  for i, (u, v) in enumerate(edges):
    if not br[i]: g[u].append(v); g[v].append(u)
  return components(g)  # (comp, k); bridge tree: comps as nodes, bridges as edges

def block_cut_tree(n, bccs):  # nodes 0..n-1 vertices, n+i block i; edge vertex--block (forest)
  t = [[] for _ in range(n + len(bccs))]  # variant: keep only articulation vertices as nodes
  for i, b in enumerate(bccs):
    for v in b: t[v].append(n + i); t[n + i].append(v)
  return t
# Edge connectivity lambda(s,t) = max flow unit caps; vertex connectivity: split v -> v_in->v_out
# cap 1 (see 05). Global min cut: Stoer-Wagner (05). Online dynamic connectivity: Holm et al.
# (HDT, levels of Euler-tour trees, O(log^2) amortized) or link-cut trees for forests - rarely needed.

def offline_connectivity(n, queries):  # O(q log q log n) segment tree over time + rollback DSU (inline; also in 02)
  # queries: (0,u,v) add edge, (1,u,v) remove existing edge, (2,u,v) ask connected. multi-edges ok
  Q = len(queries); open_ = {}; iv = []
  for t, (tp, u, v) in enumerate(queries):
    if u > v: u, v = v, u
    if tp == 0: open_.setdefault((u, v), []).append(t)
    elif tp == 1: iv.append((open_[(u, v)].pop(), t, u, v))
  for (u, v), ts in open_.items():
    for s in ts: iv.append((s, Q, u, v))
  size = 1
  while size < max(Q, 1): size *= 2
  seg = [[] for _ in range(2 * size)]
  for l, r, u, v in iv:  # edge alive on time [l, r)
    l += size; r += size
    while l < r:
      if l & 1: seg[l].append((u, v)); l += 1
      if r & 1: r -= 1; seg[r].append((u, v))
      l >>= 1; r >>= 1
  p = list(range(n)); sz = [1] * n; hist = []; mark = [0] * (2 * size); ans = []
  def find(x):  # no path compression (rollback)
    while p[x] != x: x = p[x]
    return x
  st = [1]
  while st:
    x = st.pop()
    if x < 0:  # exit node: rollback
      x = ~x
      while len(hist) > mark[x]:
        a, b = hist.pop(); p[a] = a; sz[b] -= sz[a]
      continue
    mark[x] = len(hist)
    for u, v in seg[x]:
      a, b = find(u), find(v)
      if a != b:
        if sz[a] > sz[b]: a, b = b, a
        p[a] = b; sz[b] += sz[a]; hist.append((a, b))
    st.append(~x)
    if x >= size:
      t = x - size
      if t < Q and queries[t][0] == 2:
        ans.append(find(queries[t][1]) == find(queries[t][2]))
    else:
      st.append(2 * x + 1); st.append(2 * x)
  return ans  # component count: keep comps -= 1 per union, += 1 per rollback
```

– 6.7 Eulerian and Hamiltonian paths

```
def euler_directed(n, edges):  # Hierholzer O(n+m); edges (u,v). returns (verts, edge ids) or None
  g = [[] for _ in range(n)]; deg = [0] * n
  for i, (u, v) in enumerate(edges): g[u].append(i); deg[u] += 1; deg[v] -= 1
  m = len(edges)
  if m == 0: return [0], []
  st_ = [v for v in range(n) if deg[v] == 1]
  if any(abs(d) > 1 for d in deg) or len(st_) > 1: return None
  start = st_[0] if st_ else edges[0][0]  # circuit wanted: require all deg==0
  it = [0] * n; st = [(start, -1)]; vs = []; es = []
  while st:
    u, e = st[-1]
    if it[u] < len(g[u]):
      i = g[u][it[u]]; it[u] += 1; st.append((edges[i][1], i))
    else:
      st.pop(); vs.append(u); es.append(e)
  if len(vs) != m + 1: return None  # disconnected
  return vs[::-1], es[::-1][1:]

def euler_undirected(n, edges):  # Hierholzer O(n+m); (verts, edge ids) or None; self-loops ok
  g = [[] for _ in range(n)]
  for i, (u, v) in enumerate(edges): g[u].append((v, i)); g[v].append((u, i))
  m = len(edges)
  if m == 0: return [0], []
  odd = [v for v in range(n) if len(g[v]) & 1]
  if len(odd) > 2: return None
  start = odd[0] if odd else edges[0][0]
  used = [False] * m; it = [0] * n; st = [(start, -1)]; vs = []; es = []
  while st:
    u, e = st[-1]
    while it[u] < len(g[u]) and used[g[u][it[u]][1]]: it[u] += 1
    if it[u] < len(g[u]):
      v, i = g[u][it[u]]; it[u] += 1; used[i] = True; st.append((v, i))
    else:
      st.pop(); vs.append(u); es.append(e)
  if len(vs) != m + 1: return None
  return vs[::-1], es[::-1][1:]

def de_bruijn(k, n):  # O(k^n) cyclic sequence with every length-n word over [0,k) once (FKM, Lyndon words)
  seq = []; w = [-1]  # linear version: seq + seq[:n-1]
  while w:
    w[-1] += 1; m = len(w)
    if n % m == 0: seq.extend(w)
    while len(w) < n: w.append(w[-m])
    while w and w[-1] == k - 1: w.pop()
  return seq  # lexicographically smallest. Alternative: Euler circuit on (n-1)-words graph.

def ham_path(W):  # O(2^n n^2) min cost Hamiltonian path (any endpoints); W[u][v] or INF. (cost, path)
  n = len(W); N = 1 << n; dp = [INF] * (N * n); par = [-1] * (N * n)
  for v in range(n): dp[(1 << v) * n + v] = 0
  for S in range(1, N):
    base = S * n
    for v in range(n):
      cur = dp[base + v]
      if cur == INF: continue
      row = W[v]
      for u in range(n):
        if S >> u & 1: continue
        j = (S | 1 << u) * n + u
        if cur + row[u] < dp[j]: dp[j] = cur + row[u]; par[j] = v
  full = N - 1; v = min(range(n), key=lambda v: dp[full * n + v]); best = dp[full * n + v]
  if best == INF: return INF, None
  path = []; S = full
  while v != -1: path.append(v); pv = par[S * n + v]; S ^= 1 << v; v = pv
  return best, path[::-1]  # cycle: fix start 0, close with W[v][0]. count paths: same DP with +=
# Chinese postman (undirected): sum(w) + min-weight perfect matching of odd-degree vertices
#   under shortest-path distances (bitmask DP over odd set, <=~20). Directed: min cost flow
#   from vertices with in>out to out>in. Must be connected.
```

– 6.10 Special graph topics

```
def chromatic_number(adj):  # O(2^n n); adj[v] bitmask. inclusion-exclusion over independent sets
  n = len(adj); N = 1 << n; MOD = (1 << 61) - 1  # mod: tiny false-negative risk, k-coloring count
  if n == 0: return 0
  f = [0] * N; f[0] = 1; sgn = [0] * N  # f[S] = #independent subsets of S (incl. empty)
  for S in range(1, N):
    v = (S & -S).bit_length() - 1
    f[S] = f[S ^ (1 << v)] + f[S & ~adj[v] & ~(1 << v)]; sgn[S] = sgn[S >> 1] ^ (S & 1)
  odd = n & 1; pw = [1] * N
  for k in range(1, n + 1):
    tot = 0
    for S in range(N):
      pw[S] = pw[S] * f[S] % MOD
      tot += -pw[S] if sgn[S] ^ odd else pw[S]  # sign (-1)^(n-|S|)
    if tot % MOD: return k
  return n
# Greedy/DSatur give upper bounds; 2-colorable = bipartite; planar <= 4; interval/chordal: greedy on PEO.

def max_clique(adj):  # Bron-Kerbosch with pivot, iterative, bitmasks; n <= ~50-60. returns clique mask
  n = len(adj); best = 0; bc = 0; st = [(0, (1 << n) - 1, 0, 0)]  # max independent set: complement graph
  while st:
    R, P, X, rc = st.pop()
    if not P:
      if rc > bc: best, bc = R, rc
      continue
    if rc + bin(P).count('1') <= bc: continue  # bound
    PX = P | X; piv = 0; pc = -1
    while PX:
      b = PX & -PX; u = b.bit_length() - 1; PX ^= b
      c = bin(P & adj[u]).count('1')
      if c > pc: pc = c; piv = u
    cand = P & ~adj[piv]
    while cand:
      b = cand & -cand; v = b.bit_length() - 1; cand ^= b
      st.append((R | b, P & adj[v], X & adj[v], rc + 1)); P ^= b; X |= b
  return best
# Meet in the middle for MIS n<=40: clique-check masks of the left half, for each right-half
# clique C take dp[common neighbours in left] (dp = best clique size inside a left subset).
# Planar: m <= 3n-6 (n>=3), bipartite planar m <= 2n-4; Euler V - E + F = 1 + C (C components).
#   K5 / K3,3 minors forbidden; planar graphs are 5-degenerate (some vertex deg <= 5).

def tournament_path(n, beats):  # O(n log n) beats() calls; beats(u,v) True iff edge u->v
  path = []  # Hamiltonian path always exists (binary insertion)
  for v in range(n):
    if not path or beats(v, path[0]): path.insert(0, v)
    elif beats(path[-1], v): path.append(v)
    else:
      lo, hi = 0, len(path) - 1  # invariant: path[lo]->v, v->path[hi]
      while hi - lo > 1:
        mid = (lo + hi) // 2
        if beats(path[mid], v): lo = mid
        else: hi = mid
      path.insert(hi, v)
  return path
# Tournaments: SCC condensation is a total order (a path); strongly connected => Hamiltonian
#   cycle (Camion). Max out-degree vertex is a king (reaches all in <= 2). Landau: sorted scores
#   s valid iff prefix sums >= C(k,2) and total = C(n,2).
# Cactus (each edge in <= 1 cycle): blocks are cycles or bridges -> use lowlink bccs / block-cut
#   tree, then tree DP over it; vertex cactus = each vertex in <= 1 cycle.
# Chordal (every cycle >= 4 has a chord): perfect elimination ordering via max cardinality
#   search (reverse MCS order); then max clique / chromatic number / MIS are greedy on PEO.

def count_spanning_trees(n, edges, mod):  # Kirchhoff O(n^3): det of Laplacian minor mod prime p
  L = [[0] * n for _ in range(n)]  # multigraph ok, self-loops ignored. det also in 09
  for u, v in edges:
    if u != v: L[u][u] += 1; L[v][v] += 1; L[u][v] -= 1; L[v][u] -= 1
  a = [[x % mod for x in row[1:]] for row in L[1:]]; m = n - 1; det = 1
  for c in range(m):
    piv = next((r for r in range(c, m) if a[r][c]), -1)
    if piv < 0: return 0
    if piv != c: a[c], a[piv] = a[piv], a[c]; det = -det
    det = det * a[c][c] % mod; inv = pow(a[c][c], mod - 2, mod)
    for r in range(c + 1, m):
      if a[r][c]:
        f = a[r][c] * inv % mod; ar = a[r]; ac = a[c]
        for j in range(c, m): ar[j] = (ar[j] - f * ac[j]) % mod
  return det % mod
# Directed (arborescences rooted at r): L = diag(indeg) - A (A[u][v] = #edges u->v), delete row/col r.
# Weighted: use weights instead of 1 (sum over trees of product of weights).

def prufer_encode(n, edges):  # O(n) tree on 0..n-1 (n>=2) -> code of length n-2
  g = [[] for _ in range(n)]
  for u, v in edges: g[u].append(v); g[v].append(u)
  par = [-1] * n; st = [n - 1]; seen = [False] * n; seen[n - 1] = True
  while st:
    u = st.pop()
    for v in g[u]:
      if not seen[v]: seen[v] = True; par[v] = u; st.append(v)
  deg = [len(x) for x in g]; ptr = 0
  while deg[ptr] != 1: ptr += 1
  leaf = ptr; code = []
  for _ in range(n - 2):
    nx = par[leaf]; code.append(nx); deg[nx] -= 1
    if deg[nx] == 1 and nx < ptr: leaf = nx
    else:
      ptr += 1
      while deg[ptr] != 1: ptr += 1
      leaf = ptr
  return code

def prufer_decode(code):  # O(n) code -> edge list; #labeled trees = n^(n-2); deg(v) = 1 + count(v)
  n = len(code) + 2; deg = [1] * n
  for x in code: deg[x] += 1
  ptr = 0
  while deg[ptr] != 1: ptr += 1
  leaf = ptr; edges = []
  for v in code:
    edges.append((leaf, v)); deg[v] -= 1
    if deg[v] == 1 and v < ptr: leaf = v
    else:
      ptr += 1
      while deg[ptr] != 1: ptr += 1
      leaf = ptr
  edges.append((leaf, n - 1))
  return edges
# Forests: #forests on n labeled vertices with k trees rooted at given k vertices = k n^(n-k-1).
# Tree isomorphism (AHU / hashing): see 06_trees.py. General graph isomorphism heuristic:
#   Weisfeiler-Lehman colour refinement (iteratively hash (colour, sorted neighbour colours)),
#   compare colour histograms; equal histograms != isomorphic (regular graphs fool it).
```

05 Flows and matching

```
from heapq import heappush, heappop
```

– 6.8 Network Flow: max flow / min cut theorem

```
# max s-t flow = min s-t cut capacity. After max flow, S-side of min cut = vertices reachable
# from s in the residual graph (Dinic.min_cut); cut edges = original edges S-side -> T-side.
```

– 6.8 Ford-Fulkerson / Edmonds-Karp (note)

```
# FF: augment any path, O(E * maxflow). EK: BFS shortest path, O(V E^2). Dinic below is
# strictly better; never write EK. Unit-capacity graphs: Dinic O(E sqrt E); bipartite O(E sqrt V).
```

– 6.8 Dinic's algorithm

```
class Dinic:
  """O(V^2 E) (much faster in practice). d=Dinic(n); e=d.add_edge(u,v,c[,rev_c]);
  d.max_flow(s,t); d.flow(e) flow on edge e; d.min_cut(s) -> bool list (True = S side).
  Undirected edge: add_edge(u,v,c,c). d.reset() restores original capacities."""
  def __init__(self, n):
    self.n = n
    self.g = [[] for _ in range(n)]
```

```
    self.to = []
    self.cap = []
    self.orig = []
  def add_edge(self, u, v, c, rc=0):
    e = len(self.to)
    self.g[u].append(e); self.to.append(v); self.cap.append(c)
    self.g[v].append(e + 1); self.to.append(u);
    ↪  self.cap.append(rc)
    self.orig += [c, rc]
    return e  # reverse edge is e^1
  def flow(self, e):
    return self.orig[e] - self.cap[e]
  def reset(self):
    self.cap = self.orig[:]
  def _bfs(self, s, t):
    g, to, cap = self.g, self.to, self.cap
    lv = [-1] * self.n
    lv[s] = 0
    q = [s]
    for u in q:
      for e in g[u]:
        v = to[e]
        if cap[e] > 0 and lv[v] < 0:
          lv[v] = lv[u] + 1
          q.append(v)
    self.lv = lv
    return lv[t] >= 0
  def max_flow(self, s, t):
    g, to, cap = self.g, self.to, self.cap
    total = 0
    while self._bfs(s, t):
      lv = self.lv
      it = [0] * self.n
      while True:  # find one blocking-flow path at a time
        path = []
        u = s
        while u != t:
          adj = g[u]; i = it[u]; L = len(adj); nl = lv[u] + 1
          while i < L and not (cap[adj[i]] > 0 and
          ↪  lv[to[adj[i]]] == nl):
            i += 1
          it[u] = i
          if i < L:
            e = adj[i]; path.append(e); u = to[e]
          else:  # dead end: retreat
            lv[u] = -1
            if not path:
              break
            e = path.pop(); u = to[e ^ 1]; it[u] += 1
        if u != t:
          break
        f = min(cap[e] for e in path)
        for e in path:
          cap[e] -= f
          cap[e ^ 1] += f
        total += f
    return total
  def min_cut(self, s):
    self._bfs(s, s)
    return [x >= 0 for x in self.lv]
```

– 6.8 Push-relabel (note)

```
# Highest-label push-relabel with gap heuristic: O(V^2 sqrt E),
↪  beats Dinic on dense graphs
# in C++. In Python, Dinic is simpler and usually enough; not
↪  included.
```

– 6.8 Min cost max flow (SPFA potentials + Dijkstra with potentials)

```
class MCMF:
  """O(F * E log V) (+ one SPFA if negative costs). Negative
  ↪  costs OK, no negative cycles.
  m=MCMF(n); m.add_edge(u,v,cap,cost); flow,cost =
↪  m.flow(s,t[,maxf]).
  Min cost for ANY flow amount: stop when path cost h[t] >= 0
↪  (see comment)."""
  def __init__(self, n):
    self.n = n
    self.g = [[] for _ in range(n)]
    self.to = []; self.cap = []; self.cost = []
  def add_edge(self, u, v, c, w):
    e = len(self.to)
    self.g[u].append(e); self.to += [v]; self.cap += [c];
    ↪  self.cost += [w]
    self.g[v].append(e + 1); self.to += [u]; self.cap += [0];
    ↪  self.cost += [-w]
    return e
  def flow(self, s, t, maxf=INF):
    n, g, to, cap, cost = self.n, self.g, self.to, self.cap,
↪  self.cost
    h = [0] * n
    if any(cost[e] < 0 for e in range(0, len(cost), 2)):  # SPFA
    ↪  for initial potentials
      d = [INF] * n; d[s] = 0; inq = [False] * n; q = [s];
↪  inq[s] = True; qi = 0
      while qi < len(q):
        u = q[qi]; qi += 1; inq[u] = False
        for e in g[u]:
          if cap[e] > 0 and d[u] + cost[e] < d[to[e]]:
            v = to[e]; d[v] = d[u] + cost[e]
            if not inq[v]:
              inq[v] = True; q.append(v)
      h = [x if x < INF else 0 for x in d]
    fl = tc = 0
    while fl < maxf:
      dist = [INF] * n; dist[s] = 0; pe = [-1] * n; pq = [(0, s)]
      while pq:
        du, u = heappop(pq)
        if du > dist[u]:
          continue
        hu = h[u]
        for e in g[u]:
          if cap[e] > 0:
            v = to[e]; nd = du + cost[e] + hu - h[v]
            if nd < dist[v]:
              dist[v] = nd; pe[v] = e; heappush(pq, (nd, v))
      if dist[t] == INF:
        break
      for v in range(n):
        if dist[v] < INF:
          h[v] += dist[v]
      # if h[t] - h[s] >= 0: break  # <- for min-cost (not max)
      ↪  flow
      f = maxf - fl; v = t
      while v != s:
        e = pe[v]; f = min(f, cap[e]); v = to[e ^ 1]
      v = t
      while v != s:
        e = pe[v]; cap[e] -= f; cap[e ^ 1] += f; v = to[e ^ 1]
      fl += f
      tc += f * (h[t] - h[s])
    return fl, tc
```

– 6.8 Flows with lower bounds (circulations)

```
def flow_lower_bounds(n, edges, s=-1, t=-1):
  """edges: (u,v,lo,hi). s=t=-1: feasible circulation; else MAX
  ↪  s-t flow (net out of s).
  Returns (value, per-edge flow list) or None if infeasible.
↪  One/two Dinic runs.
  Min flow instead: after feasibility, value - max_flow(t,s)
↪  (with t-s/s-t edges removed)."""
  S, T = n, n + 1
  D = Dinic(n + 2)
  exc = [0] * n
  ids = []
  for u, v, lo, hi in edges:
    ids.append(D.add_edge(u, v, hi - lo)); exc[v] += lo; exc[u]
↪  -= lo
  need = 0
  for v in range(n):
    if exc[v] > 0:
      D.add_edge(S, v, exc[v]); need += exc[v]
    elif exc[v] < 0:
      D.add_edge(v, T, -exc[v])
  if s >= 0:
    ts = D.add_edge(t, s, INF); st = D.add_edge(s, t, INF)  #
↪  both: net may be negative
  if D.max_flow(S, T) != need:
    return None
  val = 0
  if s >= 0:
    val = D.flow(ts) - D.flow(st)
    for e in (ts, st):
      D.cap[e] = D.cap[e ^ 1] = 0
    val += D.max_flow(s, t)
  return val, [hi - D.cap[e] for e, (u, v, lo, hi) in zip(ids,
  ↪  edges)]
```

– 6.8 Modeling problems as flows (notes)

```
# - Vertex capacity c(v): split v -> v_in, v_out with edge
↪  (v_in, v_out, c); in-edges go to v_in,
#   out-edges leave v_out.
# - Multiple sources/sinks: super source S->s_i, t_i->T.
# - Undirected edge: add_edge(u, v, c, c) (one pair, both
↪  directions capacity c).
# - Bipartite with capacities (assignment, b-matching): S->left
↪  (cap), left->right, right->T.
# - "At least" constraints -> lower bounds (above). Min cost
↪  with time layers -> time-expanded graph.
# - Min cut applications: cut = choose S/T side per vertex. Pay
↪  a if v in S, b if v in T:
#   s->v cap b, v->t cap a. Pay c if u in S and v in T: u->v cap
↪  c. Forbid u in S, v in T: cap INF.
#   (only "submodular" pairwise costs representable; flip one
↪  bipartite side for "same side" costs.)
# - Edge connectivity of (u,v): max flow with unit caps
↪  (undirected: cap 1 both ways).
#   Vertex connectivity (u,v non-adjacent): split each vertex
↪  with cap 1 (u,v cap INF), edges INF.
#   Global edge connectivity: Stoer-Wagner below, or n-1 flows
↪  from vertex 0.
```

– 6.8 Project selection / maximum weight closure

```
def max_closure(w, deps):
  """w[i]: profit of taking i (may be negative). deps (u,v):
  ↪  taking u requires taking v.
  Returns (best total, chosen bool list). One min cut: S->i
↪  (w>0), i->T (-w), u->v INF."""
  n = len(w)
  S, T = n, n + 1
  D = Dinic(n + 2)
  pos = 0
  for i, x in enumerate(w):
    if x > 0:
      D.add_edge(S, i, x); pos += x
    elif x < 0:
      D.add_edge(i, T, -x)
  for u, v in deps:
    D.add_edge(u, v, INF)
  f = D.max_flow(S, T)
  return pos - f, D.min_cut(S)[:n]
```

– 6.8 Gomory-Hu tree (Gusfield)

```
def gomory_hu(n, edges):
  """Undirected weighted graph -> tree [(u, parent, w)] with
  ↪  n-1 edges; min cut(u,v) =
  min weight on tree path. n-1 max flows. edges: (u,v,c)."""
  D = Dinic(n)
  for u, v, c in edges:
    D.add_edge(u, v, c, c)
  par = [0] * n
  tree = []
  for i in range(1, n):
    D.reset()
    f = D.max_flow(i, par[i])
    side = D.min_cut(i)
    tree.append((i, par[i], f))
    for j in range(i + 1, n):
      if par[j] == par[i] and side[j]:
        par[j] = i
  return tree
```

– 6.8 Global min cut (Stoer-Wagner)

```
def stoer_wagner(mat):
  """O(n^3). mat: symmetric n x n weight matrix (n >= 2).
  ↪  Returns (cut value, one side list)."""
  n = len(mat)
  w = [r[:] for r in mat]
  alive = list(range(n))
  co = [[i] for i in range(n)]
  best, best_set = INF, []
  while len(alive) > 1:
    key = [0] * n
    inA = [False] * n
    prev = last = -1
    for _ in range(len(alive)):  # maximum adjacency ordering
      sel = -1
      for v in alive:
        if not inA[v] and (sel < 0 or key[v] > key[sel]):
          sel = v
      inA[sel] = True
      prev, last = last, sel
      ws = w[sel]
      for v in alive:
        if not inA[v]:
          key[v] += ws[v]
    if key[last] < best:  # cut of the phase: {last group} vs
    ↪  rest
      best, best_set = key[last], co[last][:]
    co[prev] += co[last]
    for v in alive:
      w[prev][v] += w[last][v]
      w[v][prev] = w[prev][v]
    alive.remove(last)
  return best, best_set
```

– 6.9 Bipartite matching: Kuhn (note)

```
# Kuhn: for each left u, DFS for an augmenting path, O(VE).
↪  Hopcroft-Karp below is O(E sqrt V)
# and also iterative; use it. (Greedy initial matching speeds
↪  Kuhn a lot if you do write it.)
```

– 6.9 Hopcroft-Karp

```
def hopcroft_karp(nl, nr, adj):
  """O(E sqrt V). adj[u] = right neighbours of left u. Returns
  ↪  (size, ml, mr); -1 = free."""
  ml = [-1] * nl
  mr = [-1] * nr
  res = 0
  while True:
    dist = [-1] * nl
    q = [u for u in range(nl) if ml[u] < 0]
    for u in q:
      dist[u] = 0
    found = False
    for x in q:
      for y in adj[x]:
        w = mr[y]
        if w < 0:
          found = True
        elif dist[w] < 0:
          dist[w] = dist[x] + 1
          q.append(w)
    if not found:
      break
    it = [0] * nl
    for u in range(nl):
      if ml[u] >= 0:
        continue
      st = [u]
      while st:
        x = st[-1]
        if it[x] < len(adj[x]):
          y = adj[x][it[x]]; it[x] += 1
          w = mr[y]
          if w < 0:  # augment along stack
            for z in st:
              yy = adj[z][it[z] - 1]; ml[z] = yy; mr[yy] = z
            res += 1
            break
          if dist[w] == dist[x] + 1:
            st.append(w)
        else:
          dist[x] = -1  # dead
          st.pop()
  return res, ml, mr
```

– 6.9 Konig's theorem: min vertex cover / max independent set (bipartite)

```
def konig(nl, nr, adj, ml, mr):
  """Given a MAX matching: returns (coverL, coverR, indepL,
  ↪  indepR) as index lists.
  |cover| = matching size; independent set = complement of
↪  cover (size n - matching)."""
  visL = [False] * nl
  visR = [False] * nr
  q = [u for u in range(nl) if ml[u] < 0]
  for u in q:
    visL[u] = True
  for x in q:  # alternating BFS from free left vertices
    for y in adj[x]:
      if not visR[y]:
        visR[y] = True
        w = mr[y]
        if w >= 0 and not visL[w]:
          visL[w] = True
          q.append(w)
  coverL = [u for u in range(nl) if not visL[u]]
  coverR = [y for y in range(nr) if visR[y]]
  indepL = [u for u in range(nl) if visL[u]]
  indepR = [y for y in range(nr) if not visR[y]]
  return coverL, coverR, indepL, indepR
```

– 6.9 Hall's marriage theorem (note)

```
# Left side L has a perfect matching into R iff |N(S)| >= |S|
↪  for every S subset of L.
# Deficiency version: max matching = |L| - max_S(|S| - |N(S)|).
↪  A violating S = free left
```

```
# vertices + left vertices reached by Konig's alternating BFS
↪ (visL). Regular bipartite graphs
# always have perfect matchings. Use for "is it always
↪ possible" / greedy-proof arguments.
```

– 6.9 Weighted bipartite matching: Hungarian algorithm

```
def hungarian(a):
  """O(n^2 m). a: n x m cost matrix, n <= m. Min-cost assignment
  ↪ of every row to a distinct
  column. Returns (cost, col) with col[i] = column of row i. Max
↪ weight: negate costs."""
  n, m = len(a), len(a[0])
  u = [0] * (n + 1); v = [0] * (m + 1); p = [0] * (m + 1); way =
↪ [0] * (m + 1)
  for i in range(1, n + 1):
    p[0] = i; j0 = 0
    minv = [INF] * (m + 1); used = [False] * (m + 1)
    while True:
      used[j0] = True
      i0 = p[j0]; ai = a[i0 - 1]; ui = u[i0]
      delta = INF; j1 = 0
      for j in range(1, m + 1):
        if not used[j]:
          cur = ai[j - 1] - ui - v[j]
          if cur < minv[j]:
            minv[j] = cur; way[j] = j0
          if minv[j] < delta:
            delta = minv[j]; j1 = j
      for j in range(m + 1):
        if used[j]:
          u[p[j]] += delta; v[j] -= delta
        else:
          minv[j] -= delta
      j0 = j1
      if p[j0] == 0:
        break
    while j0:
      j1 = way[j0]; p[j0] = p[j1]; j0 = j1
  col = [0] * n
  for j in range(1, m + 1):
    if p[j]:
      col[p[j] - 1] = j - 1
  return sum(a[i][col[i]] for i in range(n)), col
```

– 6.9 General matching: Edmonds' blossom

```
def blossom(n, adj):
  """O(n^3) max cardinality matching in a general graph. adj:
  ↪ undirected adjacency lists.
  Returns (size, match) with match[v] = partner or -1."""
  match = [-1] * n
  for v in range(n):  # greedy init
    if match[v] < 0:
      for u in adj[v]:
        if match[u] < 0 and u != v:
          match[u] = v; match[v] = u
          break
  def find_path(root):
    used = [False] * n; p = [-1] * n; base = list(range(n))
    used[root] = True
    q = [root]; qi = 0
    def lca(a, b):
      seen = [False] * n
      while True:
        a = base[a]; seen[a] = True
        if match[a] < 0:
          break
        a = p[match[a]]
      while True:
        b = base[b]
        if seen[b]:
          return b
        b = p[match[b]]
    def mark(v, b, child, bl):
      while base[v] != b:
        bl[base[v]] = bl[base[match[v]]] = True
        p[v] = child; child = match[v]; v = p[match[v]]
    while qi < len(q):
      v = q[qi]; qi += 1
      for to in adj[v]:
        if base[v] == base[to] or match[v] == to:
          continue
        if to == root or (match[to] >= 0 and p[match[to]] >= 0):
        ↪ # odd cycle
          cur = lca(v, to)
          bl = [False] * n
          mark(v, cur, to, bl); mark(to, cur, v, bl)
          for i in range(n):
            if bl[base[i]]:
              base[i] = cur
              if not used[i]:
                used[i] = True; q.append(i)
        elif p[to] < 0:
          p[to] = v
          if match[to] < 0:
            return to, p
          used[match[to]] = True; q.append(match[to])
    return -1, p
  for r in range(n):
    if match[r] < 0:
      v, p = find_path(r)
      while v >= 0:  # augment
        pv = p[v]; ppv = match[pv]
        match[v] = pv; match[pv] = v; v = ppv
  return sum(1 for x in match if x >= 0) // 2, match
# Weighted general matching (O(n^3) weighted blossom) is very
↪ long -- bitmask DP for n<=20.
```

– 6.9 Stable marriage (Gale-Shapley)

```
def gale_shapley(mpref, wpref):
  """O(n^2). mpref[m] = women in m's order, wpref[w] = men in
  ↪ w's order. Proposers (men)
  get their best stable partner; women their worst. Returns
↪ wife[m]."""
  n = len(mpref)
  rank = [[0] * n for _ in range(n)]
  for w in range(n):
    for r, m in enumerate(wpref[w]):
      rank[w][m] = r
  nxt = [0] * n; husband = [-1] * n; wife = [-1] * n
  free = list(range(n))
  while free:
    m = free.pop()
    w = mpref[m][nxt[m]]; nxt[m] += 1
    h = husband[w]
    if h < 0:
      husband[w] = m; wife[m] = w
    elif rank[w][m] < rank[w][h]:
      husband[w] = m; wife[m] = w; wife[h] = -1; free.append(h)
    else:
      free.append(m)
  return wife
```

– 6.9 Minimum path cover in a DAG

```
def min_path_cover(n, edges, disjoint=True):
  """Vertex-disjoint: n - maxmatching(u_out -> v_in).
  ↪ disjoint=False (paths may share
  vertices): same on the transitive closure (O(n^3/64)
↪ bitsets). Returns (count, paths);
  in non-disjoint mode consecutive path vertices are reachable,
↪ not necessarily adjacent."""
  if disjoint:
    adj = [[] for _ in range(n)]
    for u, v in edges:
      adj[u].append(v)
  else:
    reach = [0] * n
    for u, v in edges:
      reach[u] |= 1 << v
    for k in range(n):
      bk = 1 << k; rk = reach[k]
      for i in range(n):
        if reach[i] & bk:
          reach[i] |= rk
    adj = [[v for v in range(n) if reach[u] >> v & 1] for u in
↪ range(n)]
  res, ml, mr = hopcroft_karp(n, n, adj)
  paths = []
  for v in range(n):
    if mr[v] < 0:
      path = [v]
      while ml[path[-1]] >= 0:
        path.append(ml[path[-1]])
      paths.append(path)
  return n - res, paths
```

– 6.9 Dilworth's theorem

```
# In a finite poset, min number of chains covering it = max
↪ antichain size. Min chain cover =
# non-disjoint path cover of the comparability DAG = n -
↪ matching on (i < j) pairs.
# Mirsky (dual): min antichain partition = longest chain (layer
↪ by longest-path depth).
# 2D special case: points (a,b), order a<a',b<b' -> chains via
↪ LIS-type greedy, O(n log n).
def dilworth(n, less):
  """O(n^2 sqrt n). less(i,j): STRICT partial order (must be
  ↪ transitive).
  Returns (chains, max_antichain); len(chains) ==
↪ len(max_antichain)."""
  adj = [[j for j in range(n) if less(i, j)] for i in range(n)]
  res, ml, mr = hopcroft_karp(n, n, adj)
  chains = []
  for v in range(n):
    if mr[v] < 0:
      c = [v]
      while ml[c[-1]] >= 0:
        c.append(ml[c[-1]])
      chains.append(c)
  cL, cR, _, _ = konig(n, n, adj, ml, mr)
  inc = [False] * n
  for x in cL + cR:
    inc[x] = True
  return chains, [x for x in range(n) if not inc[x]]
```

– 12.4 Matroid intersection

```
# Weighted (note): max weight common independent set. Same
↪ exchange graph; vertex weight
# -w(y) for y not in I, +w(x) for x in I; take the shortest path
↪ from X1 to X2 by
# (total weight, #edges) with Bellman-Ford/SPFA (no negative
↪ cycles). After k augmentations
# I is a max-weight common independent set of size k; take the
↪ best over k. O(r^2 n) oracles + BF.
def matroid_intersection(n, ind1, ind2):
  """Max-cardinality common independent set of two matroids on
  ↪ elements 0..n-1.
  ind1/ind2(list of elements) -> bool. O(r^2 * n) oracle calls
↪ (r = answer size).
  Examples: bipartite matching = partition(left) x
↪ partition(right);
  colorful spanning forest = graphic x partition(color)."""
  inI = [False] * n
  while True:
    I = [i for i in range(n) if inI[i]]
    out = [i for i in range(n) if not inI[i]]
    X2 = [False] * n
    for y in out:
      X2[y] = ind2(I + [y])
    adj = [[] for _ in range(n)]
    for x in I:
      J = [z for z in I if z != x]
      for y in out:
        if ind1(J + [y]):
          adj[x].append(y)
        if ind2(J + [y]):
          adj[y].append(x)
    par = [-2] * n
    q = []
    for y in out:
      if ind1(I + [y]):
        par[y] = -1; q.append(y)
    end = -1
    for u in q:  # BFS: shortest path X1 -> X2
      if X2[u]:
        end = u
        break
      for v in adj[u]:
        if par[v] == -2:
          par[v] = u; q.append(v)
    if end < 0:
      return I
    while end >= 0:
      inI[end] = not inI[end]; end = par[end]
```

06 Trees

– 7.1 Tree basics: rooting (parent, depth, BFS order, subtree sizes)

```
# O(n). order is BFS order: parents before children;
↪ reversed(order) = children before parents.
def root_tree(adj, root=0):
  n = len(adj)
  par = [-1] * n
  dep = [0] * n
  order = [root]
  for v in order:  # list grows while iterating: that's the BFS
  ↪ queue
    for u in adj[v]:
      if u != par[v]:
        par[u] = v
        dep[u] = dep[v] + 1
        order.append(u)
  sz = [1] * n
  for v in reversed(order):
    if par[v] >= 0:
      sz[par[v]] += sz[v]
  return par, dep, order, sz
# height of node v (longest downward path) = same reversed loop
↪ with h[p] = max(h[p], h[v]+1).
```

– 7.1 Tree traversals (note)

```
# Preorder = order nodes are popped in a stack DFS
↪ (euler_tin_tout). Postorder = reverse of a
# preorder that visits children in reversed order. For
↪ bottom-up DP any "children before parent"
# order works: reversed(BFS order). Level order = BFS order.
↪ Inorder (binary trees only): walk
# left pushing nodes, pop + visit, go right.
```

– 7.1 Diameter (two BFS) and center(s)

```
def bfs_dist(adj, s):  # O(n) unweighted distances from s
  d = [-1] * len(adj)
  d[s] = 0
  q = [s]
  for v in q:
    for u in adj[v]:
      if d[u] < 0:
        d[u] = d[v] + 1
        q.append(u)
  return d

def diameter(adj):  # O(n) -> (length in edges, endpoint a,
↪ endpoint b)
  d = bfs_dist(adj, 0)
  a = d.index(max(d))
  d = bfs_dist(adj, a)
  b = d.index(max(d))
  return d[b], a, b

def tree_centers(adj):  # O(n) -> [c] or [c1, c2]: middle of a
↪ diameter path
  D, a, b = diameter(adj)
  par = root_tree(adj, a)[0]
  path = [b]
  while path[-1] != a:
    path.append(par[path[-1]])
  return [path[D // 2]] if D % 2 == 0 else [path[D // 2], path[D
  ↪ // 2 + 1]]
# weighted diameter: same two-pass idea with weighted distances
↪ (or DP: best two child depths).
```

– 7.1 Centroid(s)

```
def tree_centroids(adj):  # O(n) -> 1 or 2 nodes whose removal
↪ leaves parts <= n/2
  n = len(adj)
  par, _, _, sz = root_tree(adj)
  res = []
  for v in range(n):
    mx = n - sz[v]
    for u in adj[v]:
      if u != par[v] and sz[u] > mx:
        mx = sz[u]
    if 2 * mx <= n:
      res.append(v)
  return res
```

– 7.2 LCA via binary lifting (+ k-th ancestor, distance)

```
# Build O(n log n), query O(log n).  L = LCA(adj); L.lca(u,v);
↪ L.kth(v,k); L.dist(u,v)
class LCA:
  def __init__(self, adj, root=0):
    par, self.dep, _, _ = root_tree(adj, root)
    n = len(adj)
    self.up = up = [[root if p < 0 else p for p in par]]
    for _ in range(max(1, (n - 1).bit_length()) - 1):
      prv = up[-1]
      up.append([prv[prv[v]] for v in range(n)])
  def kth(self, v, k):  # k-th ancestor of v, -1 if k > depth
    if k > self.dep[v]:
      return -1
    i = 0
    while k:
      if k & 1:
        v = self.up[i][v]
      k >>= 1
      i += 1
    return v
```

```
  def lca(self, u, v):
    dep, up = self.dep, self.up
    if dep[u] < dep[v]:
      u, v = v, u
    u = self.kth(u, dep[u] - dep[v])
    if u == v:
      return u
    for k in range(len(up) - 1, -1, -1):
      a = up[k]
      if a[u] != a[v]:
        u, v = a[u], a[v]
    return up[0][u]
  def dist(self, u, v):
    return self.dep[u] + self.dep[v] - 2 * self.dep[self.lca(u,
    ↪  v)]
# k-th node on path u->v: w=lca; if k<=dep[u]-dep[w]: kth(u,k)
↪  else kth(v, dist-k).
# weighted dist: keep wdep[] (root distance) and use
↪  wdep[u]+wdep[v]-2*wdep[lca].
```

– 7.2 LCA via Euler tour + sparse table: O(n log n) build, O(1) query

```
class LCAEuler:
  def __init__(self, adj, root=0):
    n = len(adj)
    dep = [0] * n
    par = [-1] * n
    first = [0] * n
    it = [0] * n
    euler = [root]
    stack = [root]
    while stack:
      v = stack[-1]
      if it[v] < len(adj[v]):
        u = adj[v][it[v]]
        it[v] += 1
        if u != par[v]:
          par[u] = v
          dep[u] = dep[v] + 1
          first[u] = len(euler)
          euler.append(u)
          stack.append(u)
      else:
        stack.pop()
        if stack:
          euler.append(stack[-1])
    self.dep, self.first = dep, first
    sp = [euler]  # sp[k][i] = min-depth node in euler[i : i+2^k]
    j = 1
    while 2 * j <= len(euler):
      p = sp[-1]
      sp.append([a if dep[a] < dep[b] else b for a, b in zip(p,
↪  p[j:])])
      j *= 2
    self.sp = sp
  def lca(self, u, v):
    l, r = self.first[u], self.first[v]
    if l > r:
      l, r = r, l
    k = (r - l + 1).bit_length() - 1
    a, b = self.sp[k][l], self.sp[k][r - (1 << k) + 1]
    return a if self.dep[a] < self.dep[b] else b
# Memory trick: sparse table over preorder (n entries): for
↪  tin[u]<tin[v], lca = parent of the
# min-depth node in preorder positions (tin[u], tin[v]].
```

– 7.2 Tarjan offline LCA (iterative DFS + DSU), O((n+q) alpha)

```
def tarjan_lca(adj, queries, root=0):  # queries: list of (u,
↪  v) -> list of lca
  n = len(adj)
  qs = [[] for _ in range(n)]
  for i, (u, v) in enumerate(queries):
    qs[u].append((v, i))
    qs[v].append((u, i))
  dsu = list(range(n))
  anc = list(range(n))
  done = [False] * n
  par = [-1] * n
  it = [0] * n
  ans = [-1] * len(queries)
  def find(x):
    while dsu[x] != x:
      dsu[x] = dsu[dsu[x]]
      x = dsu[x]
    return x
  stack = [root]
  while stack:
    v = stack[-1]
    if it[v] < len(adj[v]):
      u = adj[v][it[v]]
      it[v] += 1
      if u != par[v]:
        par[u] = v
        stack.append(u)
      continue
    stack.pop()  # v finished: its children's sets are already
↪  merged into v
    done[v] = True
    for w, i in qs[v]:
      if done[w]:
        ans[i] = anc[find(w)]
    p = par[v]
    if p >= 0:
      rp = find(p)
      dsu[find(v)] = rp
      anc[rp] = p
  return ans
```

– 7.2 Farach-Colton-Bender (conceptual)

```
# O(n) build / O(1) LCA: Euler tour depths differ by +-1; split
↪  into blocks of (log n)/2,
# sparse table over block minima, and precompute in-block
↪  answers per block "shape" (2^(b-1)
# shapes). Never needed in contests: LCAEuler / binary lifting
↪  are fast enough.
```

– 7.3 Euler tour flattening (tin/tout for subtree queries)

```
# O(n). Subtree of v = positions [tin[v], tout[v]) in preorder;
↪  ord[tin[v]] = v.
# u in subtree(v)  <=>  tin[v] <= tin[u] < tout[v].
def euler_tin_tout(adj, root=0):
  n = len(adj)
  tin = [0] * n
  par = [-1] * n
  ord_ = []
  stack = [root]
  while stack:
    v = stack.pop()
    tin[v] = len(ord_)
    ord_.append(v)
    for u in adj[v]:
      if u != par[v]:
        par[u] = v
        stack.append(u)
  sz = [1] * n
  for v in reversed(ord_):
    if par[v] >= 0:
      sz[par[v]] += sz[v]
  tout = [tin[v] + sz[v] for v in range(n)]
  return tin, tout, ord_
# Example: subtree-sum with point updates -> Fenwick over
↪  positions: add(tin[v], x),
# query sum(tin[v], tout[v]). Path-to-root sums with point
↪  updates: add +x at tin[v], -x at tout[v],
# then prefix(tin[u]) = sum on root..u. (Fenwick: see
↪  02_data_structures.py)
```

– 7.3 Small-to-large / DSU on tree (sack): distinct colors in every subtree

```
# O(n log n). Light children computed and wiped; heavy child's
↪  counts kept and extended.
def subtree_distinct_colors(adj, col, root=0):
  n = len(adj)
  par, _, _, sz = root_tree(adj, root)
  tin, tout, ord_ = euler_tin_tout(adj, root)
  heavy = [-1] * n
  for v in range(n):
    for u in adj[v]:
      if u != par[v] and (heavy[v] < 0 or sz[u] > sz[heavy[v]]):
        heavy[v] = u
  cnt = [0] * (max(col) + 1)
  ans = [0] * n
  distinct = 0
  stack = [(root, 1, 0)]
  while stack:
    v, keep, phase = stack.pop()
    h = heavy[v]
    if phase == 0:
      stack.append((v, keep, 1))
      if h >= 0:
        stack.append((h, 1, 0))  # heavy runs after all light
↪  ones (LIFO)
      for c in adj[v]:
        if c != par[v] and c != h:
          stack.append((c, 0, 0))
    else:
      if c != par[v] and c != h:
        for u in ord_[tin[c]:tout[c]]:
          cu = col[u]
          if cnt[cu] == 0:
            distinct += 1
          cnt[cu] += 1
      ans[v] = distinct  # <- answer subtree queries of v here
      if not keep:
        for u in ord_[tin[v]:tout[v]]:
          cnt[col[u]] -= 1
        distinct = 0
  return ans
# Alternative small-to-large with sets: for v in reversed(BFS
↪  order): S[v] = largest child set,
# merge the others into it (S[v] |= S[c]); O(n log n) set ops;
↪  easiest when values are hashable.
```

– 7.3 Heavy-light decomposition (path query / point update)

```
# Build O(n). Each path = O(log n) contiguous segments of
↪  pos[]. Subtree v = [pos[v], pos[v]+sz[v]).
class HLD:
  def __init__(self, adj, root=0):
    n = len(adj)
    self.par, self.dep, _, self.sz = par, dep, _, sz =
    ↪  root_tree(adj, root)
    heavy = [-1] * n
    for v in range(n):
      for u in adj[v]:
        if u != par[v] and (heavy[v] < 0 or sz[u] >
        ↪  sz[heavy[v]]):
          heavy[v] = u
    self.head = head = [root] * n
    self.pos = pos = [0] * n
    cur = 0
    stack = [root]
    while stack:
      v = stack.pop()
      pos[v] = cur
      cur += 1
      for u in adj[v]:
        if u != par[v] and u != heavy[v]:
          head[u] = u
          stack.append(u)
      if heavy[v] >= 0:  # pushed last -> popped next -> chain
      ↪  contiguous
        head[heavy[v]] = head[v]
        stack.append(heavy[v])
  def segments(self, u, v, edge=False):  # inclusive (l, r)
  ↪  pos-ranges covering path u-v
    head, pos, par, dep = self.head, self.pos, self.par,
↪  self.dep
    res = []
    while head[u] != head[v]:
      if dep[head[u]] < dep[head[v]]:
        u, v = v, u
      res.append((pos[head[u]], pos[u]))
      u = par[head[u]]
    if dep[u] > dep[v]:
      u, v = v, u
    if pos[u] + edge <= pos[v]:  # edge=True: values stored on
    ↪  child endpoint, skip lca
      res.append((pos[u] + edge, pos[v]))
    return res
  def lca(self, u, v):
    head, par, dep = self.head, self.par, self.dep
    while head[u] != head[v]:
      if dep[head[u]] < dep[head[v]]:
        u, v = v, u
      u = par[head[u]]
    return u if dep[u] < dep[v] else v

class _BIT:  # tiny Fenwick for the HLD example (full one: 02)
  def __init__(self, n):
    self.n, self.t = n, [0] * (n + 1)
  def add(self, i, x):
    i += 1
    while i <= self.n:
      self.t[i] += x
      i += i & -i
  def pre(self, i):  # sum of [0, i)
    s = 0
    while i:
      s += self.t[i]
      i &= i - 1
    return s

class HLDPathSum:  # point set / path sum, O(log^2 n) per query
  def __init__(self, adj, vals, root=0):
    self.h = HLD(adj, root)
    self.val = list(vals)
    self.bit = _BIT(len(adj))
    for v, x in enumerate(vals):
      self.bit.add(self.h.pos[v], x)
  def set(self, v, x):
    self.bit.add(self.h.pos[v], x - self.val[v])
    self.val[v] = x
  def query(self, u, v):
    b = self.bit
    return sum(b.pre(r + 1) - b.pre(l) for l, r in
    ↪  self.h.segments(u, v))
# For max/min or range updates on paths use a segment tree /
↪  lazy segtree (02) over pos instead.
```

– 7.3 Centroid decomposition

```
# O(n log n). Returns cpar (parent in centroid tree, -1 for top)
↪  and order (ancestors first).
# Depth of centroid tree <= log2 n. Any path u-v passes through
↪  the centroid-tree LCA of u, v.
def centroid_decomp(adj):
  n = len(adj)
  removed = [False] * n
  cpar = [-1] * n
  order = []
  par = [-1] * n
  sz = [0] * n
  stack = [(0, -1)]
  while stack:
    s, p = stack.pop()
    par[s] = -1
    comp = [s]
    for v in comp:
      for u in adj[v]:
        if u != par[v] and not removed[u]:
          par[u] = v
          comp.append(u)
    for v in comp:
      sz[v] = 1
    for v in reversed(comp):
      if par[v] >= 0:
        sz[par[v]] += sz[v]
    tot = len(comp)
    c = s
    while True:  # walk toward the heavy side
      for u in adj[c]:
        if u != par[c] and not removed[u] and 2 * sz[u] > tot:
          c = u
          break
      else:
        break
    removed[c] = True
    cpar[c] = p
    order.append(c)
    for u in adj[c]:
      if not removed[u]:
        stack.append((u, c))
  return cpar, order
# Example: number of unordered pairs (u != v) with dist(u, v) <=
↪  K.  O(n log^2 n).
# Process centroids in `order`; when c is processed, its
↪  component = reachable without removed.
def count_pairs_dist_le(adj, K):
  n = len(adj)
  _, order = centroid_decomp(adj)
  removed = [False] * n
  def pairs(ds):  # pairs i<j with ds[i]+ds[j] <= K
    ds.sort()
    i, j, r = 0, len(ds) - 1, 0
    while i < j:
      if ds[i] + ds[j] <= K:
        r += j - i
        i += 1
      else:
        j -= 1
    return r
  total = 0
  for c in order:
    alld = [0]
```

```
    for s in adj[c]:
      if removed[s]:
        continue
      ds = []
      q = [(s, c, 1)]
      for v, p, d in q:
        ds.append(d)
        for u in adj[v]:
          if u != p and not removed[u]:
            q.append((u, v, d + 1))
      total -= pairs(ds)  # remove pairs inside the same child subtree
      alld += ds
    total += pairs(alld)
    removed[c] = True
  return total
# Nearest marked node queries: best[c] = min dist to a marked node in c's component;
# mark v: for a in centroid ancestors of v (follow cpar): best[a] = min(best[a], dist(a, v));
# query v: min over ancestors a of best[a] + dist(a, v). Use LCA.dist, or store per-level dists.
```

– 7.3 Rerooting DP

```
# see 03_dp.py: rerooting
```

– 7.3 Virtual (auxiliary) tree

```
# O(k log k) given O(1)/O(log) lca. Returns (nodes sorted by tin, vpar dict), vpar[top] = -1.
# Contains the keys plus pairwise LCAs; vpar[x] = nearest proper ancestor of x in that set.
def virtual_tree(keys, tin, lca):
  vs = sorted(set(keys), key=tin.__getitem__)
  vs += [lca(vs[i], vs[i + 1]) for i in range(len(vs) - 1)]
  vs = sorted(set(vs), key=tin.__getitem__)
  vpar = {vs[0]: -1}
  for i in range(1, len(vs)):  # parent = lca of tin-adjacent nodes (set is LCA-closed)
    vpar[vs[i]] = lca(vs[i - 1], vs[i])
  return vs, vpar
# Edge weights = depth differences. Do the DP on vs in reverse tin order (children first).
```

– 7.3 Tree hashing / isomorphism (AHU canonical labels)

```
# O(n log n). ids: dict shared by all trees being compared (tuple of child labels -> int).
# Rooted trees isomorphic <=> equal labels. Unrooted: root at center(s), take min label.
def rooted_label(adj, root, ids):
  par, _, order, _ = root_tree(adj, root)
  ch = [[] for _ in adj]
  lab = 0
  for v in reversed(order):
    key = tuple(sorted(ch[v]))
    lab = ids.get(key)
    if lab is None:
      lab = ids[key] = len(ids)
    if par[v] >= 0:
      ch[par[v]].append(lab)
  return lab  # last processed = root

def unrooted_label(adj, ids):
  return min(rooted_label(adj, c, ids) for c in tree_centers(adj))
# Random hashing alternative (one pass, mergeable): h(v) = prod(r[depth(v)] + h(c)) mod P with
# random r per depth, or h(v) = 1 + sum f(h(c)) with f a random-ish mixing function (mod 2^64).
# Subtree-isomorphism counting: number of distinct labels among all subtrees = len(set(labels)).
```

– 7.3 Link-cut trees

```
# see 02_data_structures.py: LinkCutTree
```

– 7.3 Mo's algorithm on trees (note)

```
# Path queries: Euler tour where each node appears twice (in at st[v], out at en[v]), length 2n.
# For path u-v with st[u] <= st[v]: if lca == u -> range [st[u], st[v]], else [en[u], st[v]] plus
# the lca handled separately. Nodes appearing twice inside the range cancel (toggle on each add).
# Then run plain Mo over these ranges: see 01_techniques.py: Mo's algorithm.
# Subtree queries need no Mo: they're contiguous [tin, tout) ranges (euler_tin_tout).
```

– 7.3 Long-path decomposition: depth-count DP in O(n)

```
# Example (CF 1009F "Dominant Indices"): for every v, smallest d maximizing #nodes at depth d in
# subtree(v). Each node's depth array lives in one shared buffer; the longest child's array starts
# at off[v]+1 so it is inherited for free; short children merged in O(their height).
def dominant_indices(adj, root=0):
  n = len(adj)
  par, _, order, _ = root_tree(adj, root)
  h = [1] * n
  lng = [-1] * n
  for v in reversed(order):
    p = par[v]
    if p >= 0 and h[v] + 1 > h[p]:
      h[p] = h[v] + 1
      lng[p] = v
  off = [0] * n
  nxt = h[root]
  for v in order:  # parents first
    for c in adj[v]:
      if c == par[v]:
        continue
      if c == lng[v]:
        off[c] = off[v] + 1
      else:
        off[c] = nxt
        nxt += h[c]
  buf = [0] * n  # buf[off[v] + d] = #nodes at depth d below v
  ans = [0] * n
  for v in reversed(order):
    o = off[v]
    buf[o] = 1
    b, bv = 0, 1
    if lng[v] >= 0:
      b2 = ans[lng[v]] + 1
      if buf[o + b2] > 1:
        b, bv = b2, buf[o + b2]
    for c in adj[v]:
      if c == par[v] or c == lng[v]:
        continue
      oc = off[c]
      for i in range(h[c]):
        x = buf[o + i + 1] + buf[oc + i]
        buf[o + i + 1] = x
        if x > bv or (x == bv and i + 1 < b):
          b, bv = i + 1, x
    ans[v] = b
  return ans
```

07 Strings

```
import random
```

– 8.1 String basics (notes)

```
# - str is immutable: build with list + "".join(); s[::-1] reverses; bytes ops (translate, count,
#   find) run in C and are much faster than Python loops.
# s.find/s.count are fine for one pattern (count is non-overlapping!). Map chars: [ord(c) - 97 for c in s].
# - Lexicographic: Python compares str/list/tuple lexicographically; a proper prefix is smaller.
#   Sorting suffixes/rotations with key=lambda i: s[i:] is O(n^2 log n) -- only for brute force.
# - Palindromes: s == s[::-1]; all palindromic substrings -> Manacher / eertree below;
#   interval DP pal[i][j] = s[i]==s[j] and pal[i+1][j-1] for O(n^2).
# - Subsequence (not contiguous) vs substring (contiguous). n(n+1)/2 substrings, 2^n subsequences.
#   Is s a subsequence of t: it = iter(t); all(c in it for c in s)   (O(|t|)).
#   Many queries: nxt[i][c] = first position >= i of c in t (O(26|t|) table), then greedy jumps.
#   Distinct subsequences DP: dp[i+1] = 2*dp[i] - dp[last[c]] (last = index before prev occurrence).
```

– 8.2 Naive matching

```
# O(nm) worst; in Python prefer str.find in a loop: i = t.find(p, i + 1) (C speed, overlapping).
```

– 8.2 Prefix function (KMP) + search + automaton

```
def prefix_function(s):  # O(n). pi[i] = longest proper border of s[:i+1]
  n = len(s)
  pi = [0] * n
  for i in range(1, n):
    k = pi[i - 1]
    while k and s[i] != s[k]:
      k = pi[k - 1]
    if s[i] == s[k]:
      k += 1
    pi[i] = k
  return pi

def kmp_search(text, pat):  # O(n+m) -> start indices of (overlapping) occurrences
  m = len(pat)
  if m == 0:
    return list(range(len(text) + 1))
  pi = prefix_function(pat)
  res = []
  k = 0
  for i, c in enumerate(text):
    while k and c != pat[k]:
      k = pi[k - 1]
    if c == pat[k]:
      k += 1
    if k == m:
      res.append(i - m + 1)
      k = pi[k - 1]
  return res

def kmp_automaton(s, alpha="abcdefghijklmnopqrstuvwxyz"):
  # O(n * |alpha|). aut[i][c] = matched length after reading char index c in state i (0..n).
  # Use for DP over "strings avoiding s as substring" etc.
  n = len(s)
  pi = prefix_function(s)
  A = len(alpha)
  idx = {ch: j for j, ch in enumerate(alpha)}
  aut = [[0] * A for _ in range(n + 1)]
  for i in range(n + 1):
    for c in range(A):
      if i < n and idx[s[i]] == c:
        aut[i][c] = i + 1
      else:
        aut[i][c] = aut[pi[i - 1]][c] if i else 0
  return aut
```

– 8.2 Z-function

```
def z_function(s):  # O(n). z[i] = lcp(s, s[i:]), z[0] = n
  n = len(s)
  z = [0] * n
  if n:
    z[0] = n
  l = r = 0
  for i in range(1, n):
    if i < r:
      z[i] = min(r - i, z[i - l])
    while i + z[i] < n and s[z[i]] == s[i + z[i]]:
      z[i] += 1
    if i + z[i] > r:
      l, r = i, i + z[i]
  return z
# Matching: z_function(p + "\x00" + t), occurrence at i where z[m+1+i] >= m.
```

– 4.3 / 8.4 Polynomial rolling hash mod 2^61-1 (THE library rolling hash)

```
# Build O(n), substring hash O(1). Random base -> safe against anti-hash tests. Collision prob per
# comparison ~ n / 2^61. Hashes from different RollingHash objects are comparable (shared BASE).
class RollingHash:
  MOD = (1 << 61) - 1
  BASE = random.randrange(1 << 40, (1 << 61) - 2)
  def __init__(self, s):
    M, B = self.MOD, self.BASE
    self.s = s
    n = len(s)
    h = [0] * (n + 1)
    p = [1] * (n + 1)
    for i, c in enumerate(s):
      h[i + 1] = (h[i] * B + (ord(c) if type(c) is str else c + 1)) % M
      p[i + 1] = p[i] * B % M
    self.h, self.p = h, p
  def get(self, l, r):  # hash of s[l:r]
    return (self.h[r] - self.h[l] * self.p[r - l]) % self.MOD
  @staticmethod
  def combine(h1, h2, len2):  # hash(A + B) from hash(A), hash(B), |B| (needs pow)
    M = RollingHash.MOD
    return (h1 * pow(RollingHash.BASE, len2, M) + h2) % M

def hash_lcp(A, i, B, j):  # O(log n): lcp of A.s[i:] and B.s[j:] (A, B RollingHash)
  lo, hi = 0, min(len(A.s) - i, len(B.s) - j)
  while lo < hi:
    mid = (lo + hi + 1) // 2
    if A.get(i, i + mid) == B.get(j, j + mid):
      lo = mid
    else:
      hi = mid - 1
  return lo

def hash_compare(A, l1, r1, B, l2, r2):  # O(log n): -1/0/1 comparing A.s[l1:r1] vs B.s[l2:r2]
  k = min(hash_lcp(A, l1, B, l2), r1 - l1, r2 - l2)
  if k == r1 - l1 or k == r2 - l2:
    a, b = r1 - l1, r2 - l2
  else:
    a, b = A.s[l1 + k], B.s[l2 + k]
  return (a > b) - (a < b)
# Equal substrings test: A.get(l1, r1) == B.get(l2, r2). Palindrome test: hash s and s[::-1].
# Hashing sets/multisets / Zobrist: see 02_data_structures.py 4.3 (multiset_prefix_hash; anti-hash salt in 00_core.py).
```

– 8.2 Rabin-Karp (via RollingHash)

```
def rabin_karp(text, pat):  # O(n+m) expected -> start indices
  m = len(pat)
  H = RollingHash(text)
  hp = RollingHash(pat).get(0, m)
  return [i for i in range(len(text) - m + 1) if H.get(i, i + m) == hp]
# Many patterns of the same length: put their hashes in a set, slide once.
```

– 8.2 Aho-Corasick (goto table, BFS, count occurrences of every pattern)

```
# Build O(total_len * A), scan O(|text|). Alphabet = lowercase by default.
class AhoCorasick:
  def __init__(self, pats, A=26, base=97):
    self.A, self.base = A, base
    go = [[-1] * A]
    term = []
    for p in pats:
      v = 0
      for ch in p:
        c = ord(ch) - base
        if go[v][c] < 0:
          go[v][c] = len(go)
          go.append([-1] * A)
        v = go[v][c]
      term.append(v)  # duplicates share a node
    fail = [0] * len(go)
    q = []
    for c in range(A):
      u = go[0][c]
      if u < 0:
        go[0][c] = 0
      else:
        q.append(u)
    for v in q:  # BFS; q grows
      gv, gf = go[v], go[fail[v]]
      for c in range(A):
        u = gv[c]
        if u < 0:
          gv[c] = gf[c]
        else:
          fail[u] = gf[c]
          q.append(u)
    self.go, self.fail, self.term, self.order = go, fail, term, q
  def count(self, text):  # occurrences (overlapping) of each pattern in text
    go, base = self.go, self.base
    cnt = [0] * len(go)
    v = 0
    for ch in text:
      v = go[v][ord(ch) - base]
      cnt[v] += 1
    fail = self.fail
    for v in reversed(self.order):  # push counts up the fail tree
      cnt[fail[v]] += cnt[v]
    return [cnt[t] for t in self.term]
```

```
# "Does any pattern end here": precompute bad[v] |=
↪  bad[fail[v]] in BFS order.
# DP over automaton states (go table) for counting strings
↪  avoiding/containing patterns.
```

– 8.2 Bitset matching (Shift-And) with Python ints

```
def shift_and(text, pat):  # O(n * m/64). Classic: state bit i =
↪  pat[:i+1] matches here
  m = len(pat)
  mask = {}
  for i, c in enumerate(pat):
    mask[c] = mask.get(c, 0) | 1 << i
  D, hi, res = 0, 1 << (m - 1), []
  for i, c in enumerate(text):
    D = ((D << 1) | 1) & mask.get(c, 0)
    if D & hi:
      res.append(i - m + 1)
  return res

def bitset_match(text, pat):  # O(n*m/64) but only m big-int
↪  ops: fastest in Python
  pos = {}
  for i, c in enumerate(text):  # pos[c] bit i set iff text[i]
  ↪  == c
    pos[c] = pos.get(c, 0) | 1 << i
  occ = (1 << len(text)) - 1
  for j, c in enumerate(pat):
    if c != "?":  # '?' in pattern = wildcard
      occ &= pos.get(c, 0) >> j  # bit i survives iff text[i+j]
↪  == pat[j] for all j
  occ &= (1 << max(0, len(text) - len(pat) + 1)) - 1
  return [i for i in range(len(text)) if occ >> i & 1]
# Dynamic text: keep pos[c] updated per point update.
```

– 8.2 Wildcard matching with FFT (note)

```
# Wildcards (value 0) in text and/or pattern, letters 1..26.
↪  Position i matches iff
#   sum_j p_j * t_{i+j} * (p_j - t_{i+j})^2 = sum p^3 t - 2 p^2
↪  t^2 + p t^3 = 0.
# Three convolutions of reversed p with t (see
↪  09_algebra_poly.py: NTT convolution; use 2
# moduli or random letter values to avoid false zeros).
↪  Text-only/pattern-only wildcards: sum p*(p-t)^2.
```

– 8.3 Trie (insert, count words with prefix)

```
class Trie:  # O(len) per op; nodes are dicts
  def __init__(self):
    self.ch = [{}]
    self.pre = [0]  # #words passing through node
    self.end = [0]  # #words ending at node
  def insert(self, w, k=1):  # k=-1 to erase (word must exist)
    v = 0
    self.pre[0] += k
    for c in w:
      nx = self.ch[v].get(c)
      if nx is None:
        nx = self.ch[v][c] = len(self.pre)
        self.ch.append({})
        self.pre.append(0)
        self.end.append(0)
      v = nx
      self.pre[v] += k
    self.end[v] += k
  def count_prefix(self, w):  # #inserted words having prefix w
    v = 0
    for c in w:
      v = self.ch[v].get(c)
      if v is None:
        return 0
    return self.pre[v]
```

– 8.3 Binary trie (max XOR with multiset of ints)

```
class BinaryTrie:  # O(B) per op. Flat child array: ch[2v],
↪  ch[2v+1]
  def __init__(self, B=30):
    self.B = B
    self.ch = [0, 0]
    self.cnt = [0]
  def add(self, x, k=1):  # k=-1 removes one copy
    v = 0
    self.cnt[0] += k
    ch, cnt = self.ch, self.cnt
    for b in range(self.B - 1, -1, -1):
      d = x >> b & 1
      if not ch[2 * v + d]:
        ch[2 * v + d] = len(cnt)
        ch += (0, 0)
        cnt.append(0)
      v = ch[2 * v + d]
      cnt[v] += k
  def max_xor(self, x):  # max x ^ y over stored y (set must be
  ↪  non-empty)
    v, r = 0, 0
    ch, cnt = self.ch, self.cnt
    for b in range(self.B - 1, -1, -1):
      d = (x >> b & 1) ^ 1
      u = ch[2 * v + d]
      if u and cnt[u] > 0:
        r |= 1 << b
        v = u
      else:
        v = ch[2 * v + (d ^ 1)]
    return r
# min_xor: prefer the same bit. k-th smallest / count of y with
↪  x^y < K: walk using cnt.
```

– 8.3 Suffix array: SA-IS O(n) (ACL port)

```
def sa_is(s, upper):  # s: list of ints in [0, upper]. Recursion
↪  depth O(log n).
  n = len(s)
  if n == 0:
    return []
  if n == 1:
    return [0]
  if n == 2:
    return [0, 1] if s[0] < s[1] else [1, 0]
  sa = [0] * n
  ls = [False] * n
  for i in range(n - 2, -1, -1):
    ls[i] = ls[i + 1] if s[i] == s[i + 1] else s[i] < s[i + 1]
  sum_l = [0] * (upper + 1)
  sum_s = [0] * (upper + 1)
  for i in range(n):
    if not ls[i]:
      sum_s[s[i]] += 1
    else:
      sum_l[s[i] + 1] += 1
  for i in range(upper + 1):
    sum_s[i] += sum_l[i]
    if i < upper:
      sum_l[i + 1] += sum_s[i]
  def induce(lms):
    for i in range(n):
      sa[i] = -1
    buf = sum_s[:]
    for d in lms:
      if d != n:
        sa[buf[s[d]]] = d
        buf[s[d]] += 1
    buf = sum_l[:]
    sa[buf[s[n - 1]]] = n - 1
    buf[s[n - 1]] += 1
    for i in range(n):
      v = sa[i]
      if v >= 1 and not ls[v - 1]:
        sa[buf[s[v - 1]]] = v - 1
        buf[s[v - 1]] += 1
    buf = sum_l[:]
    for i in range(n - 1, -1, -1):
      v = sa[i]
      if v >= 1 and ls[v - 1]:
        buf[s[v - 1] + 1] -= 1
        sa[buf[s[v - 1] + 1]] = v - 1
  lms_map = [-1] * (n + 1)
  lms = []
  for i in range(1, n):
    if not ls[i - 1] and ls[i]:
      lms_map[i] = len(lms)
      lms.append(i)
  m = len(lms)
  induce(lms)
  if m:
    sorted_lms = [v for v in sa if lms_map[v] != -1]
    rec_s = [0] * m
    rec_upper = 0
    for i in range(1, m):
      l, r = sorted_lms[i - 1], sorted_lms[i]
      end_l = lms[lms_map[l] + 1] if lms_map[l] + 1 < m else n
      end_r = lms[lms_map[r] + 1] if lms_map[r] + 1 < m else n
      same = True
      if end_l - l != end_r - r:
        same = False
      else:
        while l < end_l:
          if s[l] != s[r]:
            break
          l += 1
          r += 1
        if l == n or s[l] != s[r]:
          same = False
      if not same:
        rec_upper += 1
      rec_s[lms_map[sorted_lms[i]]] = rec_upper
    rec_sa = sa_is(rec_s, rec_upper)
    for i in range(m):
      sorted_lms[i] = lms[rec_sa[i]]
    induce(sorted_lms)
  return sa

def suffix_array(s):  # str or list of ints -> SA (start
↪  indices, sorted)
  a = [ord(c) for c in s] if isinstance(s, str) else list(s)
  if not a:
    return []
  comp = {v: i for i, v in enumerate(sorted(set(a)))}
  return sa_is([comp[v] for v in a], len(comp) + 1)
```

– 8.3 Suffix array: prefix doubling O(n log^2 n) with sort (short, memorable)

```
def suffix_array_doubling(s):
  n = len(s)
  comp = {v: i for i, v in enumerate(sorted(set(s)))}
  rank = [comp[c] for c in s]
  sa = list(range(n))
  k = 1
  while n > 1:
    key = [rank[i] * (n + 1) + (rank[i + k] + 1 if i + k < n
↪  else 0) for i in range(n)]
    sa.sort(key=key.__getitem__)
    new = [0] * n
    for j in range(1, n):
      new[sa[j]] = new[sa[j - 1]] + (key[sa[j]] != key[sa[j -
↪  1]])
    rank = new
    if rank[sa[-1]] == n - 1:
      break
    k <<= 1
  return sa
# SA-IS (conceptual): classify suffixes S/L, sort LMS
↪  substrings by induced sorting, recurse on
# their names if not unique, then induce L then S suffixes from
↪  sorted LMS suffixes. O(n).
```

– 8.3 LCP array (Kasai)

```
def lcp_kasai(s, sa):  # O(n). lcp[i] = lcp(s[sa[i]:],
↪  s[sa[i+1]:]), len n-1
  n = len(s)
  rank = [0] * n
  for i, v in enumerate(sa):
    rank[v] = i
  lcp = [0] * max(0, n - 1)
  h = 0
  for i in range(n):
    if h:
      h -= 1
    r = rank[i]
    if r == n - 1:
      h = 0
      continue
    j = sa[r + 1]
    while i + h < n and j + h < n and s[i + h] == s[j + h]:
      h += 1
    lcp[r] = h
  return lcp
# lcp of arbitrary suffixes i, j = min(lcp[rank[i]..rank[j]-1])
↪  -> sparse table (02).
# Pattern search: binary search on SA comparing
↪  s[sa[mid]:sa[mid]+m] (O(m log n)).
```

– 8.3 Suffix automaton (distinct substrings, occurrences, LCS)

```
# Build O(n) states/transitions (dict transitions). <= 2n-1
↪  states.
# cnt[v] = |endpos(v)| = occurrences of every string in state v.
class SuffixAutomaton:
  def __init__(self, s):
    nxt, link, ln, cnt = [{}], [-1], [0], [0]
    last = 0
    for ch in s:
      cur = len(ln)
      nxt.append({})
      ln.append(ln[last] + 1)
      link.append(0)
      cnt.append(1)
      p = last
      while p != -1 and ch not in nxt[p]:
        nxt[p][ch] = cur
        p = link[p]
      if p != -1:
        q = nxt[p][ch]
        if ln[p] + 1 == ln[q]:
          link[cur] = q
        else:
          cl = len(ln)
          nxt.append(nxt[q].copy())
          ln.append(ln[p] + 1)
          link.append(link[q])
          cnt.append(0)
          while p != -1 and nxt[p].get(ch) == q:
            nxt[p][ch] = cl
            p = link[p]
          link[q] = link[cur] = cl
      last = cur
    # states by decreasing len (counting sort) -> children
    ↪  before suffix-link parents
    buckets = [[] for _ in range(len(s) + 1)]
    for v in range(len(ln)):
      buckets[ln[v]].append(v)
    order = [v for b in reversed(buckets) for v in b]
    for v in order:
      if link[v] >= 0:
        cnt[link[v]] += cnt[v]
    self.nxt, self.link, self.ln, self.cnt, self.order,
    ↪  self.last = nxt, link, ln, cnt, order, last
  def distinct_substrings(self):  # non-empty
    ln, link = self.ln, self.link
    return sum(ln[v] - ln[link[v]] for v in range(1, len(ln)))
  def occurrences(self, p):  # #occurrences of non-empty p
    v = 0
    for ch in p:
      v = self.nxt[v].get(ch)
      if v is None:
        return 0
    return self.cnt[v]
# Also: k-th distinct substring (count paths dp over DAG in
↪  reverse order), first occurrence
# (store firstpos at creation; clone copies q's), shortest
↪  non-occurring string (BFS on DAG).

def lcs_multi(strs):  # longest common substring length of all
↪  strs. O(sum |s|)
  S = SuffixAutomaton(strs[0])
  nxt, link, ln, order = S.nxt, S.link, S.ln, S.order
  best = ln[:]
  for t in strs[1:]:
    cur = [0] * len(ln)
    v = l = 0
    for ch in t:  # standard matching walk: longest suffix of
    ↪  t[:i] in SAM
      while v and ch not in nxt[v]:
        v = link[v]
        l = ln[v]
      if ch in nxt[v]:
        v = nxt[v][ch]
        l += 1
      if l > cur[v]:
        cur[v] = l
    for v in order:  # a match in v fully matches its link state
      if cur[v] and link[v] >= 0:
        cur[link[v]] = ln[link[v]]
    for v in range(len(ln)):
      if cur[v] < best[v]:
        best[v] = cur[v]
    # (cur[v] <= ln[v] always holds)
  return max(best)
# Two strings: just the walk above, tracking max l (and end
↪  position i for the substring).
# SA alternative: concat with unique separators, sliding window
↪  over SA covering all colors,
# answer = max over windows of min LCP inside.
```

– 8.3 Suffix tree / Ukkonen (note)

```
# Ukkonen builds the suffix tree online in O(n) but is long and
↪  bug-prone. Use instead:
# suffix tree of s = suffix-link tree of SAM of reversed s; or
↪  SA + LCP (an LCP interval = node).
```

– 8.3 Palindromic tree (eertree)

```
# O(n). Node 0: len -1 root, node 1: len 0 root. Distinct palindromes = len(ln) - 2.
# cnt[v] after build = occurrences of palindrome v in s.
class Eertree:
  def __init__(self, s):
    ln, link, nxt, cnt = [-1, 0], [0, 0], [{}, {}], [0, 0]
    last = 1
    for i, ch in enumerate(s):
      v = last
      while not (i - 1 - ln[v] >= 0 and s[i - 1 - ln[v]] == ch):
        v = link[v]
      if ch in nxt[v]:
        last = nxt[v][ch]
        cnt[last] += 1
        continue
      w = len(ln)
      ln.append(ln[v] + 2)
      nxt.append({})
      cnt.append(1)
      if ln[w] == 1:
        link.append(1)
      else:
        u = link[v]
       while not (i - 1 - ln[u] >= 0 and s[i - 1 - ln[u]] == ch):
          u = link[u]
        link.append(nxt[u][ch])
      nxt[v][ch] = w
      last = w
    for v in range(len(ln) - 1, 1, -1):  # links point to shorter (earlier) nodes
      cnt[link[v]] += cnt[v]
    self.ln, self.link, self.nxt, self.cnt = ln, link, nxt, cnt
# Distinct palindromes so far after each prefix = number of nodes - 2 at that time.
```

– 8.3 Manacher (odd / even radii)

```
def manacher(s):
  # O(n). d1[i]: #odd palindromes centered at i (longest = s[i-d1+1 : i+d1]).
  #       d2[i]: #even palindromes centered between i-1 and i (longest = s[i-d2 : i+d2]).
  n = len(s)
  d1 = [0] * n
  l, r = 0, -1
  for i in range(n):
    k = 1 if i > r else min(d1[l + r - i], r - i + 1)
    while i - k >= 0 and i + k < n and s[i - k] == s[i + k]:
      k += 1
    d1[i] = k
    if i + k - 1 > r:
      l, r = i - k + 1, i + k - 1
  d2 = [0] * n
  l, r = 0, -1
  for i in range(n):
    k = 0 if i > r else min(d2[l + r - i + 1], r - i + 1)
    while i - k - 1 >= 0 and i + k < n and s[i - k - 1] == s[i + k]:
      k += 1
    d2[i] = k
    if i + k - 1 > r:
      l, r = i - k, i + k - 1
  return d1, d2
# #palindromic substrings = sum(d1) + sum(d2). Longest: max(2*d1-1, 2*d2).
```

– 8.3 Lyndon factorization (Duval)

```
def duval(s):  # O(n) -> s = w1 w2 ... wk, Lyndon words, w1 >= w2 >= ...
  n = len(s)
  i = 0
  res = []
  while i < n:
    j, k = i + 1, i
    while j < n and s[k] <= s[j]:
      k = i if s[k] < s[j] else k + 1
      j += 1
    while i <= k:
      res.append(s[i:i + j - k])
      i += j - k
  return res
```

– 8.3 Minimal string rotation

```
def min_rotation(s):  # O(n) -> smallest i with s[i:]+s[:i] lexicographically min
  n = len(s)
  s2 = s + s
  i, j, k = 0, 1, 0
  while i < n and j < n and k < n:
    a, b = s2[i + k], s2[j + k]
    if a == b:
      k += 1
      continue
    if a > b:
      i += k + 1
    else:
      j += k + 1
    if i == j:
      j += 1
    k = 0
  return min(i, j)
# (Booth's algorithm / Duval on s+s give the same; this two-pointer form is shortest.)
```

– 8.3 Run enumeration (conceptual)

```
# A run = maximal periodic substring s[l:r] with smallest period p, (r-l) >= 2p. There are < n runs
# (Runs theorem). Find them: for each Lyndon root w.r.t. both orders (<, >), candidate from the
# Lyndon array (computed with a stack + LCP/LCE queries via hashing or SA), extend left/right with
# LCE. Simpler O(n log n): Main-Lorentz divide & conquer with Z-functions. Rare in contests.
```

– 8.4 String problems

```
def distinct_substrings_sa(s):  # O(n): n(n+1)/2 - sum(lcp)
  n = len(s)
  return n * (n + 1) // 2 - sum(lcp_kasai(s, suffix_array(s)))
# Via SAM: SuffixAutomaton(s).distinct_substrings().  Per prefix (online): add ln[cur]-ln[link[cur]].

def longest_repeated_substring(s):  # O(n): longest substring occurring >= 2 times (may overlap)
  if len(s) < 2:
    return ""
  sa = suffix_array(s)
  lcp = lcp_kasai(s, sa)
  i = max(range(len(lcp)), key=lcp.__getitem__)
  return s[sa[i]:sa[i] + lcp[i]]
# Occurring >= k times: max over windows of k-1 consecutive lcp values of their min.
# Non-overlapping: binary search length + hashing (track min/max start per hash).

def borders(s):  # all border lengths (proper prefix == suffix), decreasing
  if not s:
    return []
  pi = prefix_function(s)
  res = []
  b = pi[-1]
  while b:
    res.append(b)
    b = pi[b - 1]
  return res

def periods(s):  # all periods p (s[i] == s[i+p]), increasing; includes n
  n = len(s)
  return [n - b for b in borders(s)] + [n]
# Minimal period p = n - pi[-1]; s = t^k for some t iff n % p == 0. Count of each prefix as a
# substring: cnt[pi[i]]++ then propagate cnt[pi[k-1]] += cnt[k] for k decreasing (+1 each).
```

08 Number theory, combinatorics, games

```
import random
from math import gcd, isqrt, comb, factorial
from functools import lru_cache
from fractions import Fraction
from itertools import compress
```

– 9.1 Number theory: divisibility, trial-division factorization, divisors

```
# Builtins: math.gcd(*args) (3.9+ variadic), math.lcm(*args) (3.9+), math.isqrt, math.comb, math.perm.

def factorize(n):  # O(sqrt n). {prime: exponent}
  f = {}
  d = 2
  while d * d <= n:
    while n % d == 0:
      f[d] = f.get(d, 0) + 1
      n //= d
    d += 1 if d == 2 else 2
  if n > 1:
    f[n] = f.get(n, 0) + 1
  return f

def divisors(n):  # O(sqrt n) factorize + O(d(n)). Unsorted -> sort if needed. d(n)<=1344 for n<=1e9
  ds = [1]
  for p, e in factorize(n).items():
    ds = [d * p ** k for d in ds for k in range(e + 1)]
  return ds
```

– 9.1 Sieve of Eratosthenes (bytearray slicing)

```
def sieve(n):  # O(n log log n). is_p[i] in {0,1}; 1e7 in ~1s CPython. primes = list(compress(range(n+1), is_p))
  is_p = bytearray([1]) * (n + 1)
  is_p[0:2] = b'\x00\x00'[:n + 1]
  for i in range(2, isqrt(n) + 1):
    if is_p[i]:
      is_p[i * i::i] = bytes(len(range(i * i, n + 1, i)))
  return is_p

def primes_upto(n):
  return list(compress(range(n + 1), sieve(n))) if n >= 2 else []
```

– 9.1 Linear sieve: spf, primes, phi, mu

```
def linear_sieve(n):  # O(n) (slow-ish in CPython: ~3s for 1e7; use sieve() if only primes needed)
  spf = [0] * (n + 1); phi = [0] * (n + 1); mu = [0] * (n + 1); primes = []
  if n >= 1:
    phi[1] = mu[1] = 1
  for i in range(2, n + 1):
    if spf[i] == 0:
      spf[i] = i; primes.append(i); phi[i] = i - 1; mu[i] = -1
    si = spf[i]
    for p in primes:
      ip = i * p
      if p > si or ip > n:
        break
      spf[ip] = p
      if p == si:
        phi[ip] = phi[i] * p  # mu[ip] stays 0
      else:
        phi[ip] = phi[i] * (p - 1); mu[ip] = -mu[i]
  return spf, primes, phi, mu
# factor x<=n with spf: while x>1: p=spf[x]; x//=p  -> O(log x)
```

– 9.1 Segmented sieve

```
def segmented_sieve(L, R):  # primes in [L, R]; O((R-L) log log R + sqrt R). R-L ~1e7 ok
  L = max(L, 2)
  if L > R:
    return []
  seg = bytearray([1]) * (R - L + 1)
  for p in primes_upto(isqrt(R)):
    s = max(p * p, (L + p - 1) // p * p)
    if s <= R:
      seg[s - L::p] = bytes(len(range(s - L, R - L + 1, p)))
  return [L + i for i in compress(range(R - L + 1), seg)]
```

– 9.1 GCD/LCM, extended Euclid, Bezout

```
# gcd(a,b)=gcd(b,a%b); lcm=a//gcd(a,b)*b. Bezout: ax+by=c solvable iff gcd(a,b)|c.
def ext_gcd(a, b):  # O(log). returns (g, x, y): a*x + b*y = g = gcd(a,b) >= 0. |x|<=|b|, |y|<=|a|
  x0, y0, x1, y1 = 1, 0, 0, 1
  while b:
    q = a // b
    a, b = b, a - q * b
    x0, x1 = x1, x0 - q * x1
    y0, y1 = y1, y0 - q * y1
  if a < 0:
    a, x0, y0 = -a, -x0, -y0
  return a, x0, y0
```

– 9.1 Linear Diophantine equations

```
def diophantine(a, b, c):  # ax+by=c, (a,b)!=(0,0). None or (x0,y0,dx,dy): x=x0+k*dx, y=y0-k*dy, k in Z
  g, x, y = ext_gcd(a, b)
  if c % g:
    return None
  return x * (c // g), y * (c // g), b // g, a // g

def _krange(v0, step, lo, hi):  # k with lo <= v0 + k*step <= hi
  INF = float('inf')
  if step == 0:
    return (-INF, INF) if lo <= v0 <= hi else (1, 0)
  if step < 0:
    v0, step, lo, hi = -v0, -step, -hi, -lo
  return -((v0 - lo) // step), (hi - v0) // step  # ceil((lo-v0)/s), floor((hi-v0)/s)

def count_diophantine(a, b, c, x1, x2, y1, y2):  # #solutions with x in [x1,x2], y in [y1,y2]
  s = diophantine(a, b, c)
  if s is None:
    return 0
  x0, y0, dx, dy = s
  l1, r1 = _krange(x0, dx, x1, x2)
  l2, r2 = _krange(y0, -dy, y1, y2)
  lo, hi = max(l1, l2), min(r1, r2)
  return 0 if lo > hi else int(hi - lo + 1)  # (infinite only if a=0 or b=0 with unbounded k: n/a here)
```

– 9.1 Modular arithmetic, inverse, fast exponentiation

```
# - Python % is always non-negative for positive mod: (-3) % 7 == 4. Bigints never overflow.
# - Fast exp: built-in pow(a, e, m) (C speed). Inverse: pow(a, -1, m) (3.8+, any m with gcd=1,
#   raises ValueError otherwise). Fermat: pow(a, m-2, m) for prime m. Euler: a^phi(m) = 1 (gcd=1).
# - a^e mod m with huge e, gcd(a,m)>1: if e >= log2(m): a^e = a^(e mod phi + phi) (mod m).
# - Keep values reduced; in PyPy, products < 2^63 stay machine ints (mod ~1e9 ok, mod 2^61 not).
def inv_table(n, m):  # O(n). inv[i] = i^{-1} mod m for 1<=i<=n, m prime > n
  inv = [0, 1] + [0] * (n - 1)
  for i in range(2, n + 1):
    inv[i] = (m - m // i) * inv[m % i] % m
  return inv[:n + 1]
```

– 9.1 Euler's totient

```
def phi(n):  # O(sqrt n)
  r = n
  for p in factorize(n):
    r -= r // p
  return r
# sum_{d|n} phi(d) = n. phi multiplicative. #{1<=k<=n: gcd(k,n)=1} = phi(n); their sum = n*phi(n)/2 (n>1)
```

– 9.1 Chinese Remainder Theorem (generalized, non-coprime moduli)

```
def crt(rs, ms):  # x = r_i (mod m_i). returns (x, lcm) with 0<=x<lcm, or None if inconsistent
  x, M = 0, 1
  for r, m in zip(rs, ms):
    g, p, _ = ext_gcd(M, m)
    if (r - x) % g:
      return None
    t = (r - x) // g * p % (m // g)
    x += M * t
    M = M // g * m
    x %= M
  return x, M
```

– 9.1 Wilson's theorem, Lucas's theorem

```
# Wilson: (p-1)! = -1 (mod p) iff p prime. n! mod p for n>=p is 0; for n near p use (p-1)! = -1 and divide.
def lucas(n, k, p):  # C(n,k) mod small prime p; O(log_p n * cost). Precompute fact mod p if p small & many queries
  res = 1
  while n or k:
    ni, ki = n % p, k % p
    if ki > ni:
      return 0
    res = res * comb(ni, ki) % p
    n //= p; k //= p
  return res
# C(n,k) mod prime power / composite: Granville / CRT over prime powers (rare).
```

– 9.1 Primitive roots

```
def primitive_root(m):  # smallest g; exists iff m in {1,2,4,p^k,2p^k}. else -1. O(m^{?}) small in practice
  if m <= 4:
    return m - 1 if m > 1 else 0
  ph = phi(m)
  fs = list(factorize(ph))
  for g in range(2, m):
    if gcd(g, m) == 1 and all(pow(g, ph // q, m) != 1 for q in fs):
```

```
      return g
  return -1
```

– 9.1 Discrete logarithm (BSGS, gcd(a,m) != 1 allowed)

```
def discrete_log(a, b, m):  # smallest x>=0 with a^x = b (mod
↪  m), or -1. O(sqrt m)
  a %= m; b %= m
  if m == 1:
    return 0
  k, add = 1, 0
  g = gcd(a, m)
  while g > 1:
    if b == k:
      return add
    if b % g:
      return -1
    b //= g; m //= g; add += 1
    k = k * (a // g) % m
    g = gcd(a, m)
  n = isqrt(m) + 1
  an = pow(a, n, m)
  tbl = {}
  cur = b
  for q in range(n + 1):  # baby: b*a^q (later q overwrites ->
  ↪  smallest x)
    tbl[cur] = q
    cur = cur * a % m
  cur = k
  for p in range(1, n + 1):  # giant: k*a^(np)
    cur = cur * an % m
    if cur in tbl:
      return n * p - tbl[cur] + add
  return -1
```

– 9.1 Discrete root

```
def discrete_root(k, a, p):  # all x in [0,p) with x^k = a (mod
↪  p), p prime. sorted list. O(sqrt p)
  a %= p
  if a == 0:
    return [0]
  if p == 2:
    return [1]
  g = primitive_root(p)
  y = discrete_log(pow(g, k, p), a, p)  # (g^y)^k = a
  if y < 0:
    return []
  step = (p - 1) // gcd(k, p - 1)
  return sorted(pow(g, y + i * step, p) for i in range(gcd(k, p
  ↪  - 1)))
```

– 9.1 Quadratic residues, Tonelli-Shanks

```
# Euler criterion: a^((p-1)/2) = 1 iff a is QR mod odd prime p.
↪  (p-1)/2 nonzero QRs.
def sqrt_mod(a, p):  # x with x*x = a (mod p), p prime; -1 if
↪  none. Other root: p-x. O(log^2 p)
  a %= p
  if a == 0 or p == 2:
    return a
  if pow(a, (p - 1) // 2, p) != 1:
    return -1
  if p % 4 == 3:
    return pow(a, (p + 1) // 4, p)
  q, s = p - 1, 0
  while q % 2 == 0:
    q //= 2; s += 1
  z = 2
  while pow(z, (p - 1) // 2, p) != p - 1:
    z += 1
  m, c, t, r = s, pow(z, q, p), pow(a, q, p), pow(a, (q + 1) //
↪  2, p)
  while t != 1:
    i, t2 = 0, t
    while t2 != 1:
      t2 = t2 * t2 % p; i += 1
    b = pow(c, 1 << (m - i - 1), p)
    m, c, t, r = i, b * b % p, t * b * b % p, r * b % p
  return r
```

– 9.1 Miller-Rabin (deterministic for n < 3.18e23, so all 64-bit)

```
# Bases = first 12 primes; 318665857834031151167461 is the
↪  first strong pseudoprime to all of them.
# Arbitrary size: same code is a probable-prime test; add
↪  random bases for more confidence.
_MR = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37)

def is_prime(n):  # O(12 log n) modmuls
  if n < 2:
    return False
  for p in _MR:
    if n % p == 0:
      return n == p
  d, s = n - 1, 0
  while d % 2 == 0:
    d //= 2; s += 1
  for a in _MR:
    x = pow(a, d, n)
    if x == 1 or x == n - 1:
      continue
    for _ in range(s - 1):
      x = x * x % n
      if x == n - 1:
        break
    else:
      return False
  return True
```

– 9.1 Pollard's rho (Brent) full factorization

```
def pollard_rho(n):  # nontrivial factor of composite n,
↪  expected O(n^{1/4})
  if n % 2 == 0:
    return 2
  while True:
    y, c, m = random.randrange(1, n), random.randrange(1, n),
↪  128
    g = r = q = 1
    while g == 1:
      x = y
      for _ in range(r):
        y = (y * y + c) % n
      k = 0
      while k < r and g == 1:
        ys = y
        for _ in range(min(m, r - k)):
          y = (y * y + c) % n
          q = q * abs(x - y) % n
        g = gcd(q, n); k += m
      r *= 2
    if g == n:
      g = 1
      while g == 1:
        ys = (ys * ys + c) % n
        g = gcd(abs(x - ys), n)
    if g != n:
      return g

def factor(n):  # sorted prime factors with multiplicity; n
↪  ~1e18 in ms. Counter(factor(n)) for dict
  res = []
  for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
    while n % p == 0:
      res.append(p); n //= p
  st = [n]
  while st:
    n = st.pop()
    if n == 1:
      continue
    if is_prime(n):
      res.append(n); continue
    d = pollard_rho(n)
    st += [d, n // d]
  return sorted(res)
```

– 9.1 Multiplicative functions, Mobius, Dirichlet convolution (notes)

```
# f multiplicative: f(ab)=f(a)f(b) for gcd(a,b)=1 -> determined
↪  by f(p^k). Linear sieve computes any
#   such f if you also track the power of spf (cnt[i], or i's
↪  spf-free part).
# Examples: phi, mu, d(n)=prod(e+1), sigma(n)=prod
↪  (p^(e+1)-1)/(p-1), id, 1, e(n)=[n==1].
# Dirichlet: (f*g)(n) = sum_{d|n} f(d) g(n/d). mu*1=e,
↪  phi*1=id, mu*id=phi, 1*1=d, id*1=sigma.
#   Computing f*g up to n by "for d: for multiples" is O(n log
↪  n).
# Mobius inversion: g = f*1  <=>  f = g*mu. Typical:
↪  [gcd(a,b)==1] = sum_d mu(d)*(n//d)^2. O(n) sieve + O(sqrt n)
def coprime_pairs(n, mu_prefix=None):  # #{(a,b) in [1,n]^2 :
↪  gcd(a,b)=1} = sum_d mu(d)*(n//d)^2. O(n) sieve + O(sqrt n)
  if mu_prefix is None:
    mu = linear_sieve(n)[3]
    mu_prefix = [0] * (n + 1)
    for i in range(1, n + 1):
      mu_prefix[i] = mu_prefix[i - 1] + mu[i]
  res = 0
  for l, r, v in floor_blocks(n):
    res += (mu_prefix[r] - mu_prefix[l - 1]) * v * v
  return res
```

– 9.1 Floor-division blocks (sqrt(n) distinct values of n//i)

```
def floor_blocks(n):  # yields (l, r, v): n//i == v for all
↪  l<=i<=r; ~2sqrt(n) blocks
  l = 1
  while l <= n:
    v = n // l
    r = n // v
    yield l, r, v
    l = r + 1
# Divisor-sum trick: sum_{i<=n} d(i) = sum_i n//i ; sum_{i<=n}
↪  sigma(i) = sum_i i*(n//i).
def divisor_count_sum(n):  # sum_{i=1}^n d(i), O(sqrt n)
  return sum((r - l + 1) * v for l, r, v in floor_blocks(n))
```

– 9.1 Prefix sums of multiplicative functions: Lucy_Hedgehog prime counting

```
def prime_pi(n):  # pi(n) in O(n^{3/4}); 1e10 ~ seconds in
↪  PyPy, 1e9 ~ few s CPython
  # Sum of primes: init with v(v+1)/2-1 and subtract
  ↪  p*(S[v/p]-S[p-1]).
  if n < 2:
    return 0
  r = isqrt(n)
  lo = [i - 1 for i in range(r + 1)]  # lo[v] = S(v), v<=r
  hi = [0] + [n // i - 1 for i in range(1, r + 1)]  # hi[i] =
↪  S(n//i)
  for p in range(2, r + 1):
    if lo[p] == lo[p - 1]:
      continue
    sp = lo[p - 1]
    p2 = p * p
    for i in range(1, min(r, n // p2) + 1):
      d = i * p
      hi[i] -= (hi[d] if d <= r else lo[n // d]) - sp
    for v in range(r, p2 - 1, -1):
      lo[v] -= lo[v // p] - sp
  return hi[1]
```

– 9.1 Du's sieve (prefix sum of phi / mu, n ~ 1e9..1e10)

```
def du_sieve(n, pre, total):
  # S(x) = sum_{i<=x} f(i), given g=1 (so f*1 known): S(x) =
  ↪  total(x) - sum_{d=2}^x S(x//d).
  # pre[x] = S(x) for x < len(pre) (from linear sieve, size ~
  ↪  n^{2/3}). O(n^{2/3}).
  # phi: total = x(x+1)/2 ; mu: total = 1.
  memo = {}
  lim = len(pre)
  def S(x):
    if x < lim:
      return pre[x]
    if x in memo:
      return memo[x]
    res = total(x)
    l = 2
    while l <= x:
      v = x // l
      r = x // v
      res -= (r - l + 1) * S(v)
      l = r + 1
    memo[x] = res
    return res
  return S(n)  # recursion depth only O(log n)

def phi_sum(n, lim=None):  # sum_{i<=n} phi(i)
  lim = lim or max(10, int(n ** (2 / 3)))
  ph = linear_sieve(lim)[2]
  pre = [0] * (lim + 1)
  for i in range(1, lim + 1):
    pre[i] = pre[i - 1] + ph[i]
  return du_sieve(n, pre, lambda x: x * (x + 1) // 2)

def mertens(n, lim=None):  # sum_{i<=n} mu(i)
  lim = lim or max(10, int(n ** (2 / 3)))
  mu = linear_sieve(lim)[3]
  pre = [0] * (lim + 1)
  for i in range(1, lim + 1):
    pre[i] = pre[i - 1] + mu[i]
  return du_sieve(n, pre, lambda x: 1)
# Min_25 sieve (note): prefix sums of multiplicative f with
↪  f(p) a low-degree poly in p, n ~1e10.
#   Part 1 = Lucy-style sums of p^k over primes at all n//i;
↪  part 2 = recursive
#   S(x, j) = g(x) - g(p_{j-1}) + sum_{k>=j, p_k^2<=x} sum_e
↪  f(p_k^e)(S(x/p_k^e, k+1) + [e>1]). O(n^{3/4}/log n).
```

– 9.1 Legendre's formula

```
def legendre(n, p):  # v_p(n!) = sum n//p^i = (n -
↪  digitsum_p(n))/(p-1). Kummer: v_p(C(m+n,m)) = #carries
↪  adding m+n base p
  e = 0
  while n:
    n //= p; e += n
  return e
```

– 9.1 Continued fractions, Stern-Brocot, best rational approximation

```
def cont_frac(a, b):  # a/b -> [q0; q1, q2, ...]
  res = []
  while b:
    q = a // b
    res.append(q)
    a, b = b, a - q * b
  return res

def convergents(cf):  # list of (p, q) convergents p_k/q_k
  p0, q0, p1, q1 = 0, 1, 1, 0
  out = []
  for a in cf:
    p0, q0, p1, q1 = p1, q1, a * p1 + p0, a * q1 + q0
    out.append((p1, q1))
  return out
# Best rational approx with denominator <= N:
↪  Fraction(x).limit_denominator(N) (exact, uses
↪  CF+semiconvergents).
# Stern-Brocot: mediant (a+c)/(b+d) of a/b < c/d; path to x is
↪  run-length = its CF. Neighbours
#   a/b, c/d in SB/Farey satisfy bc - ad = 1.
↪  Smallest-denominator fraction in (L,R): walk SB with
#   CF steps (jump k at once) -> O(log).
```

– 9.1 Pell's equation

```
def pell(D):  # fundamental (x, y) of x^2 - D y^2 = 1, D>0
↪  non-square. Via CF of sqrt(D) (period <~ sqrt(D) log D)
  # All solutions: x_k + y_k sqrt(D) = (x1 + y1 sqrt(D))^k.
  ↪  x^2-Dy^2=-1 solvable iff period odd.
  a0 = isqrt(D)
  m, d, a = 0, 1, a0
  p0, p1, q0, q1 = 1, a0, 0, 1
  while p1 * p1 - D * q1 * q1 != 1:
    m = d * a - m
    d = (D - m * m) // d
    a = (a0 + m) // d
    p0, p1 = p1, a * p1 + p0
    q0, q1 = q1, a * q1 + q0
  return p1, q1
```

– 9.2 Combinatorics: factorials, inverse factorials, C, P, multinomial

```
# Sum rule / product rule. P(n,k)=n!/(n-k)!,
↪  C(n,k)=n!/(k!(n-k)!), Pascal: C(n,k)=C(n-1,k-1)+C(n-1,k).
# Exact bigints: math.comb / math.perm / math.factorial. Pascal
↪  table O(n^2) works for any modulus.
class Comb:  # O(n) precompute, O(1) query. mod must be prime >
↪  n
  def __init__(self, n, mod=998244353):
    self.mod = mod
    f = [1] * (n + 1)
    for i in range(1, n + 1):
      f[i] = f[i - 1] * i % mod
    fi = [1] * (n + 1)
    fi[n] = pow(f[n], mod - 2, mod)
    for i in range(n, 0, -1):
      fi[i - 1] = fi[i] * i % mod
    self.f, self.fi = f, fi
  def C(self, n, k):
    if k < 0 or k > n or n < 0:
      return 0
    return self.f[n] * self.fi[k] % self.mod * self.fi[n - k] %
    ↪  self.mod
  def P(self, n, k):
    return 0 if k < 0 or k > n else self.f[n] * self.fi[n - k] %
    ↪  self.mod
  def multinomial(self, ks):  # (sum ks)! / prod(k!)
    r = self.f[sum(ks)]
    for k in ks:
      r = r * self.fi[k] % self.mod
    return r
  def H(self, n, k):  # stars & bars: #ways to put k identical
  ↪  balls into n boxes = C(n+k-1, k)
    return self.C(n + k - 1, k) if n else int(k == 0)
```

– 9.2 Stars and bars, pigeonhole (notes)

```
# x1+..+xn = k, xi>=0: C(k+n-1, n-1); xi>=1: C(k-1, n-1);
↪  xi>=li: subtract sum li first.
# xi<=ui: inclusion-exclusion over the set of violated bounds.
# Lattice paths (0,0)->(a,b) right/up: C(a+b,a). Not crossing
↪  y=x (reflection): C(2n,n)-C(2n,n+1).
# Pigeonhole: n+1 items in n boxes -> some box has >=2. Prefix
↪  sums mod n: among n+1 prefixes two
#   are equal -> nonempty subarray with sum = 0 mod n.
↪  Erdos-Szekeres: seq of length rs+1 has
#   increasing subseq of length r+1 or decreasing of length s+1.
```

– 9.2 Inclusion-exclusion

```
def count_coprime_to(n, primes):  # #{1<=x<=n : no p in primes
↪  divides x}. O(2^k)
  k = len(primes)
  res = 0
  for mask in range(1 << k):
    d, bits = 1, 0
    for i in range(k):
      if mask >> i & 1:
        d *= primes[i]; bits += 1
    res += (-1) ** bits * (n // d)
  return res

def surjections(n, k):  # #onto maps [n]->[k] = sum_i (-1)^i
↪  C(k,i) (k-i)^n = k! S(n,k)
  return sum((-1) ** i * comb(k, i) * (k - i) ** n for i in
  ↪  range(k + 1))
```

– 9.2 Catalan, Stirling (1st/2nd), Bell, derangements

```
def catalan(n):  # C(2n,n)/(n+1); mod: C(2n,n)-C(2n,n+1).
↪  1,1,2,5,14,42.. (parens, BSTs, triangulations, Dyck paths)
  return comb(2 * n, n) // (n + 1)

def stirling1(n, mod=None):  # unsigned c[i][k]: perms of i
↪  with k cycles. c(i,k)=c(i-1,k-1)+(i-1)c(i-1,k). O(n^2)
  c = [[0] * (n + 1) for _ in range(n + 1)]
  c[0][0] = 1
  for i in range(1, n + 1):
    for k in range(1, i + 1):
      v = c[i - 1][k - 1] + (i - 1) * c[i - 1][k]
      c[i][k] = v % mod if mod else v
  return c

def stirling2(n, mod=None):  # S[i][k]: partitions of i-set
↪  into k nonempty blocks. S(i,k)=S(i-1,k-1)+k S(i-1,k)
  S = [[0] * (n + 1) for _ in range(n + 1)]
  S[0][0] = 1
  for i in range(1, n + 1):
    for k in range(1, i + 1):
      v = S[i - 1][k - 1] + k * S[i - 1][k]
      S[i][k] = v % mod if mod else v
  return S

def bell(n, mod=None):  # B_0..B_n via Bell triangle, O(n^2). B
↪  = 1,1,2,5,15,52. B_{n+1} = sum C(n,k) B_k
  res = [1]
  row = [1]
  for _ in range(n):
    nr = [row[-1]]
    for x in row:
      v = nr[-1] + x
      nr.append(v % mod if mod else v)
    row = nr
    res.append(row[0])
  return res

def derangements(n, mod=None):  # D_0..D_n. D(i) =
↪  (i-1)(D(i-1)+D(i-2)); D(n) = round(n!/e)
  D = [1, 0] + [0] * max(0, n - 1)
  for i in range(2, n + 1):
    v = (i - 1) * (D[i - 1] + D[i - 2])
    D[i] = v % mod if mod else v
  return D[:n + 1]
```

– 9.2 Partition numbers (pentagonal number theorem)

```
def partitions(n, mod=None):  # p(0..n), O(n sqrt n). p(i) =
↪  sum_k (-1)^{k+1} [p(i-k(3k-1)/2) + p(i-k(3k+1)/2)]
  p = [1] + [0] * n
  for i in range(1, n + 1):
    s, k = 0, 1
    while True:
      g1 = k * (3 * k - 1) // 2
      if g1 > i:
        break
      g2 = g1 + k
      t = p[i - g1] + (p[i - g2] if g2 <= i else 0)
      s += t if k & 1 else -t
      k += 1
    p[i] = s % mod if mod else s
  return p
# Partitions into distinct parts / bounded parts: knapsack DP
↪  (O(n^2)) or GF ops in 09.
```

– 9.2 Burnside / Polya: necklaces and bracelets

```
# Burnside: #orbits = (1/|G|) sum_g |Fix(g)|. Polya: coloring
↪  with k colors fixed by g: k^{#cycles(g)}.
# Rotation by i of n beads has gcd(i,n) cycles.
def necklaces(n, k):  # colorings of n-cycle up to rotation:
↪  (1/n) sum_{d|n} phi(d) k^{n/d}
  return sum(phi(d) * k ** (n // d) for d in divisors(n)) // n

def bracelets(n, k):  # up to rotation + reflection (dihedral
↪  group, size 2n)
  rot = sum(phi(d) * k ** (n // d) for d in divisors(n))
  if n % 2:
    refl = n * k ** ((n + 1) // 2)
  else:
    refl = n // 2 * (k ** (n // 2 + 1) + k ** (n // 2))
  return (rot + refl) // (2 * n)
```

– 9.2 Generating functions, LGV, bijections, Cayley (notes)

```
# OGF: sum a_n x^n (unlabeled; product = convolution). EGF: sum
↪  a_n x^n/n! (labeled; product =
#   binomial convolution; exp(F) = sets of F-structures).
↪  1/(1-x)^k = sum C(n+k-1,k-1) x^n.
#   Series inverse/log/exp/pow and NTT -> see
↪  09_algebra_poly.py.
# LGV: non-intersecting paths a_i->b_i in a DAG (only identity
↪  permutation possible) =
#   det(M), M[i][j] = #paths a_i->b_j. Determinant mod p -> see
↪  09_algebra_poly.py.
# Cayley: n^{n-2} labeled trees; forests on n vertices with k
↪  given roots: k n^{n-k-1};
#   spanning trees of K_{a,b}: a^{b-1} b^{a-1}; general graph:
↪  Kirchhoff matrix-tree (det of Laplacian minor).
#   Prufer code <-> labeled tree; vertex degree = 1 +
↪  occurrences. #trees with degrees d_i: (n-2)!/prod(d_i-1)!
# Labeled graphs on n vertices: 2^{C(n,2)}; connected ones via
↪  c_n = g_n - sum_{k<n} C(n-1,k-1) c_k g_{n-k}.
```

– 9.2 Young tableaux, hook length formula

```
def hook_length(shape):  # #standard Young tableaux of
↪  partition shape (rows non-increasing) = n!/prod hooks
  cols = [sum(1 for r in shape if r > j) for j in
↪  range(shape[0])] if shape else []
  prod = 1
  for i, r in enumerate(shape):
    for j in range(r):
      prod *= (r - j - 1) + (cols[j] - i - 1) + 1
  return factorial(sum(shape)) // prod
# RSK: permutations of n <-> pairs of SYT of same shape; sum
↪  f_lambda^2 = n!. LIS = first row length.
```

– 9.3 Probability and expected value (notes)

```
# P(A|B) = P(A and B)/P(B). Bayes: P(A|B) = P(B|A)P(A)/P(B).
# Linearity: E[sum X_i] = sum E[X_i] even if dependent -> split
↪  into indicator variables.
# Contribution technique: answer = sum over elements/pairs of
↪  (value * #configs/prob it counts).
# Geometric: success prob p per trial -> E[#trials] = 1/p.
↪  Coupon collector: n*H_n.
#   "Expected time until": E[T] = sum_{t>=0} P(T > t).
# Expected value DP / game DP -> see 03_dp.py. Markov chains /
↪  random walks: E[steps from i] gives
#   linear system E_i = 1 + sum p_ij E_j -> Gaussian elimination
↪  (09_algebra_poly.py); on a path/tree
#   solve in O(n) by expressing E_i = a_i E_parent + b_i. Random
↪  walk on line +-1: E[hit 0 or n from k] = k(n-k).
# Answers mod p: probability P/Q -> P * pow(Q, -1, mod) % mod.
↪  Fractions add as a/b + c/d = (ad+bc)/bd.
def frac_mod(p, q, mod=998244353):
  return p % mod * pow(q, -1, mod) % mod
```

– 9.6 Game theory: Nim, Sprague-Grundy

```
# Impartial, normal play: P-position (prev player wins) iff all
↪  moves go to N-positions.
# Nim: first player wins iff xor of piles != 0. Winning move:
↪  pile a with a ^ x < a, reduce to a ^ x.
# Sprague-Grundy: g(s) = mex{g(t)}; sum of games -> xor of
↪  Grundy values; lose iff 0.
def mex(vals):
  s = set(vals)
  m = 0
  while m in s:
    m += 1
  return m

def grundy_table(n, moves):  # states 0..n, moves(s) -> next
↪  states all < s. O(n * moves)
  g = [0] * (n + 1)
  for s in range(n + 1):
    g[s] = mex(g[t] for t in moves(s))
  return g
# General hashable states: @lru_cache def G(s): return mex(G(t)
↪  for t in moves(s)) (watch recursion depth).
# Tip: print grundy table for small n and look for a period /
↪  pattern.
```

– 9.6 Nim variants: staircase, misere, Moore's, Wythoff

```
def staircase_nim_win(c):  # c[i] coins on stair i (c[0] =
↪  ground); move coins i -> i-1. win iff xor odd stairs
  x = 0
  for i in range(1, len(c), 2):
    x ^= c[i]
  return x != 0

def misere_nim_win(piles):  # last to move loses
  x = 0
  for a in piles:
    x ^= a
  if max(piles, default=0) <= 1:
    return x == 0  # even number of 1-piles -> win
  return x != 0

def moore_nim_win(piles, k):  # may reduce up to k piles per
↪  move. lose iff every bit column count = 0 mod (k+1)
  for b in range(max(piles, default=0).bit_length()):
    if sum(a >> b & 1 for a in piles) % (k + 1):
      return True
  return False

def wythoff_losing(a, b):  # take any from one pile or equal
↪  from both. P-positions: (floor(k phi), floor(k phi^2))
  if a > b:
    a, b = b, a
  k = b - a
  return a == (k + isqrt(5 * k * k)) // 2  # exact integer
  ↪  floor(k*phi)
# Partisan games (conceptual): Conway surreal numbers;
↪  Hackenbush values; positions = {L | R}.
```

– 9.6 Minimax with alpha-beta

```
def alphabeta(s, depth, children, evaluate,
↪  alpha=float('-inf'), beta=float('inf'), maxi=True):
  # children(s) -> list of states (order best-first for
  ↪  pruning), evaluate(s) -> score for max player
  ch = children(s) if depth else []
  if not ch:
    return evaluate(s)
  if maxi:
    v = float('-inf')
    for t in ch:
      v = max(v, alphabeta(t, depth - 1, children, evaluate,
↪  alpha, beta, False))
      alpha = max(alpha, v)
      if alpha >= beta:
        break
  else:
    v = float('inf')
    for t in ch:
      v = min(v, alphabeta(t, depth - 1, children, evaluate,
↪  alpha, beta, True))
      beta = min(beta, v)
      if alpha >= beta:
        break
  return v
```

– 9.7 Misc: series formulas (notes)

```
# sum i = n(n+1)/2, sum i^2 = n(n+1)(2n+1)/6, sum i^3 =
↪  (n(n+1)/2)^2.
# Geometric: sum_{i<n} r^i = (r^n - 1)/(r - 1); mod p with r-1 =
↪  0 mod p -> n; else use inverse,
#   or divide-and-conquer / matrix [[r,1],[0,1]]^n for any
↪  modulus. Arithmetic: n(a1+an)/2.
# sum i^k: Faulhaber / Lagrange interpolation (09). H_n ~ ln n +
↪  0.5772.
```

– 9.7 Fibonacci: fast doubling, Zeckendorf, Pisano period

```
# F(2k) = F(k)(2F(k+1) - F(k)), F(2k+1) = F(k)^2 + F(k+1)^2.
↪  gcd(F_m,F_n) = F_gcd(m,n).
# Cassini F(n-1)F(n+1) - F(n)^2 = (-1)^n. sum_{i<=n} F_i =
↪  F_{n+2} - 1.
def fib(n, mod=None):  # (F(n), F(n+1)), O(log n)
  a, b = 0, 1
  for bit in bin(n)[2:]:
    c = a * (2 * b - a)
    d = a * a + b * b
    if mod:
      c %= mod; d %= mod
    if bit == '1':
      a, b = d, (c + d) % mod if mod else c + d
    else:
      a, b = c, d
  return a, b

def zeckendorf(n):  # n = sum of non-consecutive Fibonacci
↪  numbers (greedy), descending
  fs = [1, 2]
  while fs[-1] <= n:
    fs.append(fs[-1] + fs[-2])
  res = []
  for f in reversed(fs):
    if f <= n:
      res.append(f); n -= f
  return res

def pisano(m):  # period of F mod m, O(pi(m)) <= 6m. pi(lcm) =
↪  lcm of pi(prime powers)
  if m == 1:
    return 1
  a, b, i = 0, 1, 0
  while True:
    a, b = b, (a + b) % m
    i += 1
    if a == 0 and b == 1:
      return i
```

– 9.7 Big integers, fractions (notes)

```
# Python int is arbitrary precision: + - * fast (Karatsuba), //
↪  and str() of 1e5+ digit numbers slow
#   (str(int) limited to 4300 digits in 3.11+:
↪  sys.set_int_max_str_digits(0)). Use int.bit_length,
#   int.to_bytes, divmod, pow(a, e, m). isqrt exact for
↪  bigints; use it, never int(x**0.5).
# Fraction(a, b): exact rationals, auto-reduced, slow (gcd each
↪  op). Fraction(0.1) is exact binary
#   value -> use Fraction('0.1'). limit_denominator(N) = best
↪  approximation. Compare a/b < c/d as a*d < c*b.
# Decimal with getcontext().prec = 50 for high-precision
↪  decimals.
```

– 9.7 Base conversion (arbitrary base, negative base)

```
def to_base(n, b):  # digits MSB-first. b >= 2 needs n >= 0; b
↪  <= -2 works for any n. int(s, b) for b<=36 strings
  if n == 0:
    return [0]
  d = []
  while n:
    n, r = divmod(n, b)
    if r < 0:  # only for negative base
      n += 1; r -= b
    d.append(r)
  return d[::-1]

def from_base(d, b):
  v = 0
  for x in d:
    v = v * b + x
  return v
# bin(n), oct(n), hex(n), format(n, 'b'); int('ff', 16). Gray
↪  code: g = n ^ (n >> 1).
```

– 9.7 Numerical methods: Simpson, Newton, bisection

```
def simpson(f, a, b, eps=1e-10):  # adaptive Simpson integral
↪  of f over [a,b]
  def S(a, b, fa, fm, fb):
    return (b - a) / 6 * (fa + 4 * fm + fb)
  def rec(a, b, fa, fm, fb, whole, eps, depth):
    m = (a + b) / 2
    lm, rm = (a + m) / 2, (m + b) / 2
    flm, frm = f(lm), f(rm)
    left, right = S(a, m, fa, flm, fm), S(m, b, fm, frm, fb)
    if depth <= 0 or abs(left + right - whole) <= 15 * eps:
      return left + right + (left + right - whole) / 15
    return rec(a, m, fa, flm, fm, left, eps / 2, depth - 1) +
    ↪  rec(m, b, fm, frm, fb, right, eps / 2, depth - 1)
  fa, fm, fb = f(a), f((a + b) / 2), f(b)
```

```
  return rec(a, b, fa, fm, fb, S(a, b, fa, fm, fb), eps, 50)
# Bisection: for _ in range(100): m=(lo+hi)/2; (lo,hi) = (m,hi)
↪  if pred(m) else (lo,m). Fixed
#   iteration count beats eps checks. Ternary search for
↪  unimodal (or golden-section).
# Newton: x -= f(x)/f'(x), quadratic convergence near simple
↪  root. Integer sqrt: math.isqrt;
#   integer k-th root: r = int(round(n ** (1/k))) then adjust r
↪  +-1 while r**k > n / (r+1)**k <= n.
```

– 9.7 Linear programming: simplex

```
def simplex(A, b, c, eps=1e-9):
  # maximize c^T x s.t. A x <= b, x >= 0. returns (value, x);
  ↪  value = -inf infeasible, inf unbounded.
  # Two-phase, Bland's rule (no cycling). O(m n * pivots), fine
  ↪  for ~100x100. Use Fraction for exact.
  m, n = len(b), len(c)
  N = list(range(n)) + [-1]
  B = list(range(n, n + m))
  D = [[0.0] * (n + 2) for _ in range(m + 2)]
  for i in range(m):
    for j in range(n):
      D[i][j] = A[i][j]
    D[i][n] = -1; D[i][n + 1] = b[i]
  for j in range(n):
    D[m][j] = -c[j]
  D[m + 1][n] = 1
  def pivot(r, s):
    Dr = D[r]
    inv = 1 / Dr[s]
    for i in range(m + 2):
      if i != r and abs(D[i][s]) > eps:
        Di = D[i]
        f = Di[s] * inv
        for j in range(n + 2):
          Di[j] -= Dr[j] * f
        Di[s] = Dr[s] * f
    for j in range(n + 2):
      if j != s:
        Dr[j] *= inv
    for i in range(m + 2):
      if i != r:
        D[i][s] *= -inv
    Dr[s] = inv
    B[r], N[s] = N[s], B[r]
  def run(phase):
    x = m + phase - 1
    while True:
      s = -1
      for j in range(n + 1):
        if N[j] != -phase and (s == -1 or (D[x][j], N[j]) <
        ↪  (D[x][s], N[s])):
          s = j
      if D[x][s] >= -eps:
        return True
      r = -1
      for i in range(m):
        if D[i][s] <= eps:
          continue
        if r == -1 or (D[i][n + 1] / D[i][s], B[i]) < (D[r][n +
        ↪  1] / D[r][s], B[r]):
          r = i
      if r == -1:
        return False
      pivot(r, s)
  r = min(range(m), key=lambda i: D[i][n + 1]) if m else 0
  if m and D[r][n + 1] < -eps:
    pivot(r, n)
    if not run(2) or D[m + 1][n + 1] < -eps:
      return float('-inf'), None
    for i in range(m):
      if B[i] == -1:
        s = 0
        for j in range(1, n + 1):
          if s == -1 or (D[i][j], N[j]) < (D[i][s], N[s]):
            s = j
        pivot(i, s)
  ok = run(1)
  x = [0.0] * n
  for i in range(m):
    if B[i] < n:
      x[B[i]] = D[i][n + 1]
  return (D[m][n + 1] if ok else float('inf')), x
# Duality: max c^T x, Ax<=b, x>=0  ==  min b^T y, A^T y >= c,
↪  y>=0. Equality -> two inequalities;
#   free var x = x+ - x-; minimize -> maximize -c.
```

– 9.7 Floor sum

```
def floor_sum(n, m, a, b):  # sum_{i=0}^{n-1} floor((a*i+b)/m),
↪  m>0, any sign a,b. O(log m)
  ans = 0
  while True:
    if a >= m or a < 0:
      ans += n * (n - 1) // 2 * (a // m); a %= m
    if b >= m or b < 0:
      ans += n * (b // m); b %= m
    y = a * n + b
    if y < m:
      return ans
    n, b, m, a = y // m, y % m, a, m
```

09 Linear algebra and polynomials

```
from operator import mul
import decimal
```

– 9.4 Linear Algebra: matrix multiply / power

```
# O(n^3). C = A*B mod p. Transpose B once, inner loop = C-level
↪  sum(map(mul)), one % per entry.
# PyPy gotcha: the un-reduced sum is a bigint (> 2^63) -> slow
↪  on PyPy; there use a loop with % per term.
def mat_mul(A, B, mod):
  Bt = list(zip(*B))
  return [[sum(map(mul, row, col)) % mod for col in Bt] for row
  ↪  in A]
# O(n^3 log e). A^e mod p.
def mat_pow(A, e, mod):
  n = len(A)
  R = [[int(i == j) for j in range(n)] for i in range(n)]
  while e:
    if e & 1:
      R = mat_mul(R, A, mod)
    A = mat_mul(A, A, mod)
    e >>= 1
  return R
# O(d^3 log k). a[n] = sum_{i<d} c[i]*a[n-1-i]; a = first d
↪  terms. Returns a[k].
# (Bostan-Mori below is O(d log d log k) -- prefer it for big
↪  d.)  Path counting: (adj^k)[u][v].
def linrec_matpow(c, a, k, mod):
  d = len(c)
  if k < d:
    return a[k] % mod
  M = [list(c)] + [[int(j == i) for j in range(d)] for i in
↪  range(d - 1)]
  P = mat_pow(M, k - d + 1, mod)
  return sum(P[0][j] * a[d - 1 - j] for j in range(d)) % mod
```

– 9.4 Gaussian elimination / systems of linear equations

```
# O(n*m*min(n,m)). Over reals, partial pivoting. Returns
↪  (status, x): 0 = no solution (x None),
# 1 = unique, 2 = infinitely many (x = one solution, free vars =
↪  0). Gotcha: tune eps to input scale.
def gauss_real(A, b, eps=1e-9):
  n, m = len(A), len(A[0])
  M = [[float(v) for v in A[i]] + [float(b[i])] for i in
↪  range(n)]
  where = [-1] * m
  r = 0
  for c in range(m):
    if r == n:
      break
    p = max(range(r, n), key=lambda i: abs(M[i][c]))
    if abs(M[p][c]) < eps:
      continue
    M[r], M[p] = M[p], M[r]
    iv = 1.0 / M[r][c]
    Mr = M[r] = [v * iv for v in M[r]]
    for i in range(n):
      f = M[i][c]
      if i != r and f != 0.0:
        M[i] = [x - f * y for x, y in zip(M[i], Mr)]
    where[c] = r
    r += 1
  for i in range(r, n):
    if abs(M[i][m]) > eps:
      return 0, None
  x = [M[where[c]][m] if where[c] >= 0 else 0.0 for c in
↪  range(m)]
  return (1 if r == m else 2), x
# O(n*m*min(n,m)). Gauss-Jordan mod prime p on matrix M (n x
↪  ncols+extra), only first ncols pivoted.
# Returns (M in RREF, where[c] = pivot row of column c or -1,
↪  rank).
def rref_mod(M, ncols, p):
  M = [[v % p for v in row] for row in M]
  n = len(M)
  where = [-1] * ncols
  r = 0
  for c in range(ncols):
    piv = next((i for i in range(r, n) if M[i][c]), -1)
    if piv < 0:
      continue
    M[r], M[piv] = M[piv], M[r]
    iv = pow(M[r][c], p - 2, p)
    Mr = M[r] = [v * iv % p for v in M[r]]
    for i in range(n):
      f = M[i][c]
      if i != r and f:
        M[i] = [(x - f * y) % p for x, y in zip(M[i], Mr)]
    where[c] = r
    r += 1
    if r == n:
      break
  return M, where, r
# Solve A x = b mod prime p. Returns (status, x) like
↪  gauss_real.
def solve_mod(A, b, p):
  m = len(A[0])
  M, where, r = rref_mod([A[i] + [b[i]] for i in range(len(A))],
↪  m, p)
  if any(M[i][m] for i in range(r, len(M))):
    return 0, None
  return (1 if r == m else 2), [M[where[c]][m] if where[c] >= 0
  ↪  else 0 for c in range(m)]

def rank_mod(A, p):
  return rref_mod(A, len(A[0]), p)[2]
# Inverse mod prime p, or None if singular. O(n^3).
def mat_inv_mod(A, p):
  n = len(A)
  M, where, r = rref_mod([A[i] + [int(i == j) for j in range(n)]
↪  for i in range(n)], n, p)
  return None if r < n else [row[n:] for row in M]
```

– 9.4 Determinants

```
# O(n^3). det mod prime p.
def det_mod(A, p):
  M = [[v % p for v in row] for row in A]
  n, det = len(M), 1
  for c in range(n):
    piv = next((i for i in range(c, n) if M[i][c]), -1)
    if piv < 0:
      return 0
    if piv != c:
      M[c], M[piv] = M[piv], M[c]
      det = -det
    det = det * M[c][c] % p
    iv = pow(M[c][c], p - 2, p)
    Mc = M[c]
    for i in range(c + 1, n):
      f = M[i][c] * iv % p
      if f:
        M[i] = [(x - f * y) % p for x, y in zip(M[i], Mc)]
  return det % p
# O(n^3 log m). det mod ANY m (composite ok): Euclid-style row
↪  reduction, no inverses needed.
def det_any_mod(A, m):
  M = [[v % m for v in row] for row in A]
  n, det = len(M), 1
  for c in range(n):
    for r in range(c + 1, n):
      while M[r][c]:
        q = M[c][c] // M[r][c]
        if q:
          M[c] = [(x - q * y) % m for x, y in zip(M[c], M[r])]
        M[c], M[r] = M[r], M[c]
        det = -det
    if M[c][c] == 0:
      return 0
    det = det * M[c][c] % m
  return det % m
```

– 9.4 Gaussian elimination over GF(2)

```
# O(n*m*n/64). rows: ints, bit c (c<m) = coeff of x_c, bit m =
↪  rhs. Returns (rank, x bitmask or None).
# For plain rank pass m = number of columns and rhs bits 0.
def gauss_gf2(rows, m):
  rows = list(rows)
  n = len(rows)
  where = [-1] * m
  r = 0
  for c in range(m):
    piv = next((i for i in range(r, n) if rows[i] >> c & 1), -1)
    if piv < 0:
      continue
    rows[r], rows[piv] = rows[piv], rows[r]
    pr = rows[r]
    for i in range(n):
      if i != r and rows[i] >> c & 1:
        rows[i] ^= pr
    where[c] = r
    r += 1
  if any(rows[i] >> m & 1 for i in range(r, n)):
    return r, None
  x = 0
  for c in range(m):
    if where[c] >= 0 and rows[where[c]] >> m & 1:
      x |= 1 << c
  return r, x
```

– 9.4 XOR basis (linear basis over GF(2))

```
# insert/can/max O(B). kth(k): k-th smallest (0-indexed, k=0 ->
↪  0) DISTINCT value in span, -1 if none.
# Gotcha: if the multiset has len(vals) > rank, 0 is also a
↪  nonempty-subset xor (adjust k for "nonempty").
class XorBasis:
  def __init__(self, bits=62):
    self.B = bits
    self.b = [0] * bits
    self.rank = 0
  def insert(self, x):
    b = self.b
    for i in range(self.B - 1, -1, -1):
      if x >> i & 1:
        if not b[i]:
          b[i] = x
          self.rank += 1
          return True
        x ^= b[i]
    return False
  def can(self, x):
    for i in range(self.B - 1, -1, -1):
      if x >> i & 1:
        x ^= self.b[i]
    return x == 0
  def max_xor(self, x=0):
    for i in range(self.B - 1, -1, -1):
      if x ^ self.b[i] > x:
        x ^= self.b[i]
    return x
  def kth(self, k):
    b = self.b[:]
    for i in range(self.B):
      for j in range(i - 1, -1, -1):
        if b[j] and b[i] >> j & 1:
          b[i] ^= b[j]
    basis = [v for v in b if v]
    if k >= 1 << len(basis):
      return -1
    res = 0
    for j, v in enumerate(basis):
      if k >> j & 1:
        res ^= v
    return res
```

– 9.4 Berlekamp-Massey

```
# O(n^2). Shortest c with s[i] = sum_j c[j]*s[i-1-j] (mod
↪  prime). Needs 2*L terms to be sure.
def berlekamp_massey(s, mod=MOD):
  C, B = [1], [1]
  L, m, b = 0, 1, 1
  for n in range(len(s)):
    d = s[n]
    for i in range(1, L + 1):
      d = (d + C[i] * s[n - i]) % mod
    if d == 0:
      m += 1
      continue
    T = C[:]
    coef = d * pow(b, mod - 2, mod) % mod
    if len(C) < len(B) + m:
      C += [0] * (len(B) + m - len(C))
    for i in range(len(B)):
      C[i + m] = (C[i + m] - coef * B[i]) % mod
    if 2 * L <= n:
      L, B, b, m = n + 1 - L, T, d, 1
    else:
```

```
      m += 1
  C += [0] * (L + 1 - len(C))
  return [(-c) % mod for c in C[1:L + 1]]
# k-th term of linear recurrence: see linrec_kth (Bostan-Mori)
↪  in 9.5. Kitamasa (x^k mod charpoly)
# has the same complexity; Bostan-Mori is shorter.
```

– 9.5 Polynomial multiplication: Kronecker substitution (any modulus)

```
# convolve(a, b, mod=None): exact. Pack coeffs into one huge
↪  number with w digits per slot (w large
# enough that no slot overflows), multiply once, unpack. Works
↪  for ANY mod (or exact ints, negatives ok).
# BENCHMARK (CPython 3.14, mod 998244353, len n each):
#   int (Karatsuba)          n=1e4: 0.16s  n=1e5: 5.9s  n=2e5:
↪  17s   <- CPython int mul is NOT FFT!
#   decimal (libmpdec NTT)   n=1e4: 0.06s  n=1e5: 0.66s n=2e5:
↪  1.2s  <- used for big inputs
#   pure-Python ntt_convolve n=1e5: 5.9s.   Built on convolve:
↪  poly_inv n=1e5 2.2s, poly_exp n=1e5 5.8s.
# So: decimal-based Kronecker for big sizes, int-based for
↪  small. On PyPy use ntt_convolve (decimal
# is slow there). Alternatives for arbitrary mod with NTT: 3
↪  NTT primes + CRT, or split-FFT (15-bit).
_DCTX = decimal.Context(prec=decimal.MAX_PREC,
↪  Emax=decimal.MAX_EMAX, Emin=decimal.MIN_EMIN)

def _kron(a, b):  # nonneg ints
  n = len(a) + len(b) - 1
  ma, mb = max(a), max(b)
  if ma == 0 or mb == 0:
    return [0] * n
  bound = ma * mb * min(len(a), len(b))
  if min(len(a), len(b)) <= 300:  # int path (bytes packing)
    W = (bound.bit_length() + 7) >> 3
    fa = int.from_bytes(b''.join(x.to_bytes(W, 'little') for x
↪  in a), 'little')
    fb = fa if a is b else int.from_bytes(b''.join(x.to_bytes(W,
↪  'little') for x in b), 'little')
    B = (fa * fb).to_bytes(n * W, 'little')
    return [int.from_bytes(B[i:i + W], 'little') for i in
    ↪  range(0, n * W, W)]
  w = len(str(bound))
  f = '%0' + str(w) + 'd'
  fa = _DCTX.create_decimal(''.join([f % x for x in
↪  reversed(a)]))
  fb = fa if a is b else _DCTX.create_decimal(''.join([f % x for
↪  x in reversed(b)]))
  s = str(_DCTX.multiply(fa, fb)).zfill(n * w)
  return [int(s[i:i + w]) for i in range(n * w - w, -1, -w)]

def convolve(a, b, mod=None):
  if not a or not b:
    return []
  if mod is not None:
    if len(a) <= 32 or len(b) <= 32:  # naive is faster for tiny
    ↪  inputs
      res = [0] * (len(a) + len(b) - 1)
      for i, x in enumerate(a):
        if x:
          for j, y in enumerate(b):
            res[i + j] += x * y
      return [v % mod for v in res]
    sq = a is b
    a = [x % mod for x in a]
    b = a if sq else [x % mod for x in b]
    return [v % mod for v in _kron(a, b)]
  if min(a) >= 0 and min(b) >= 0:
    return _kron(a, b)
  ap, an = [max(x, 0) for x in a], [max(-x, 0) for x in a]
  bp, bn = [max(x, 0) for x in b], [max(-x, 0) for x in b]
  def k(u, v):
    return _kron(u, v) if any(u) and any(v) else [0] * (len(a) +
    ↪  len(b) - 1)
  return [w + x - y - z for w, x, y, z in zip(k(ap, bp), k(an,
  ↪  bn), k(ap, bn), k(an, bp))]
```

– 9.5 NTT (mod 998244353, primitive root 3)

```
# O(n log n). Iterative in-place; len(a) power of 2. Reference
↪  / PyPy (in CPython convolve is faster).
def ntt(a, invert=False):
  n = len(a)
  j = 0
  for i in range(1, n):
    bit = n >> 1
    while j & bit:
      j ^= bit
      bit >>= 1
    j |= bit
    if i < j:
      a[i], a[j] = a[j], a[i]
  length = 2
  while length <= n:
    w = pow(3, (MOD - 1) // length, MOD)
    if invert:
      w = pow(w, MOD - 2, MOD)
    half = length >> 1
    ws = [1] * half
    for i in range(1, half):
      ws[i] = ws[i - 1] * w % MOD
    for s in range(0, n, length):
      for k in range(half):
        u = a[s + k]
        v = a[s + k + half] * ws[k] % MOD
        a[s + k] = (u + v) % MOD
        a[s + k + half] = (u - v) % MOD
    length <<= 1
  if invert:
    ni = pow(n, MOD - 2, MOD)
    for i in range(n):
      a[i] = a[i] * ni % MOD

def ntt_convolve(a, b):
  n = len(a) + len(b) - 1
  sz = 1 << (n - 1).bit_length()
  fa = [x % MOD for x in a] + [0] * (sz - len(a))
  fb = [x % MOD for x in b] + [0] * (sz - len(b))
  ntt(fa)
  ntt(fb)
  fa = [x * y % MOD for x, y in zip(fa, fb)]
  ntt(fa, True)
  return fa[:n]
```

– 9.5 FFT (complex, cmath)

```
# O(n log n) floats. Exact only while result coeffs < ~1e14
↪  (double precision); rarely needed in
# Python since convolve() is exact. Kept short for reference.
import cmath

def fft(a, invert=False):
  n = len(a)
  j = 0
  for i in range(1, n):
    bit = n >> 1
    while j & bit:
      j ^= bit
      bit >>= 1
    j |= bit
    if i < j:
      a[i], a[j] = a[j], a[i]
  length = 2
  while length <= n:
    ang = (-2 if invert else 2) * cmath.pi / length
    half = length >> 1
    ws = [cmath.exp(1j * ang * k) for k in range(half)]
    for s in range(0, n, length):
      for k in range(half):
        u, v = a[s + k], a[s + k + half] * ws[k]
        a[s + k], a[s + k + half] = u + v, u - v
    length <<= 1
  if invert:
    for i in range(n):
      a[i] /= n

def fft_multiply(a, b):
  n = len(a) + len(b) - 1
  sz = 1 << (n - 1).bit_length()
  fa = [complex(x) for x in a] + [0j] * (sz - len(a))
  fb = [complex(x) for x in b] + [0j] * (sz - len(b))
  fft(fa)
  fft(fb)
  fa = [x * y for x, y in zip(fa, fb)]
  fft(fa, True)
  return [round(x.real) for x in fa[:n]]
```

– 9.5 Power series mod 998244353: inverse, log, exp, sqrt, pow (Newton)

```
# All take f (list) and n = number of output terms; O(n log
↪  n)-ish (convolve-bound).
def _conv(a, b):
  return convolve(a, b, MOD)

def poly_inv(f, n):  # needs f[0] != 0. g = g*(2 - f*g)
  g = [pow(f[0], MOD - 2, MOD)]
  k = 1
  while k < n:
    k <<= 1
    h = _conv(f[:k], g)[:k]
    h = [-x % MOD for x in h]
    h[0] = (h[0] + 2) % MOD
    g = _conv(g, h)[:k]
  return g[:n]

def poly_deriv(f):
  return [f[i] * i % MOD for i in range(1, len(f))]

def poly_integ(f):
  n = len(f) + 1
  inv = [0, 1] + [0] * (n - 1)
  for i in range(2, n + 1):
    inv[i] = (MOD - (MOD // i) * inv[MOD % i] % MOD) % MOD
  return [0] + [f[i] * inv[i + 1] % MOD for i in range(len(f))]

def poly_log(f, n):  # needs f[0] == 1
  if n == 1:
    return [0]
  r = _conv(poly_deriv(f[:n]), poly_inv(f, n))[:n - 1]
  r += [0] * (n - 1 - len(r))
  return poly_integ(r)

def poly_exp(f, n):  # needs f[0] == 0. g = g*(1 - log g + f)
  g = [1]
  k = 1
  while k < n:
    k <<= 1
    l = poly_log(g, k)
    h = [((f[i] if i < len(f) else 0) - l[i]) % MOD for i in
↪  range(k)]
    h[0] = (h[0] + 1) % MOD
    g = _conv(g, h)[:k]
  return g[:n]

def poly_sqrt(f, n):  # needs f[0] == 1 (else: sqrt of f[0] via
↪  Tonelli, see 08; shift even zeros)
  g = [1]
  inv2 = (MOD + 1) // 2
  k = 1
  while k < n:
    k <<= 1
    h = _conv(f[:k], poly_inv(g, k))[:k]
    h += [0] * (k - len(h))
    g = [(h[i] + (g[i] if i < len(g) else 0)) * inv2 % MOD for i
↪  in range(k)]
  return g[:n]

def poly_pow(f, e, n):  # f^e mod x^n, any f, e >= 0 (e may be
↪  huge)
  if e == 0:
    return [1] + [0] * (n - 1)
  t = next((i for i, c in enumerate(f) if c % MOD), -1)
  if t < 0 or t * e >= n:
    return [0] * n
  c = f[t] % MOD
  ic = pow(c, MOD - 2, MOD)
  g = [x * ic % MOD for x in f[t:t + n - t * e]]
  m = n - t * e
  l = poly_log(g, m)
  em = e % MOD
  g = poly_exp([x * em % MOD for x in l], m)
  ce = pow(c, e, MOD)
  return [0] * (t * e) + [x * ce % MOD for x in g]
```

– 9.5 Polynomial division / mod

```
# O(n log n). a = b*q + r, deg r < deg b. Needs b[-1] != 0.
↪  Returns (q, r) with len(r) = len(b)-1.
def poly_divmod(a, b):
  n, m = len(a), len(b)
  if n < m:
    return [0], a + [0] * (m - 1 - n)
  k = n - m + 1
  if m <= 64 or k <= 64:  # naive long division
    r = [x % MOD for x in a]
    il = pow(b[-1], MOD - 2, MOD)
    q = [0] * k
    for i in range(k - 1, -1, -1):
      c = q[i] = r[i + m - 1] * il % MOD
      if c:
        for j in range(m):
          r[i + j] = (r[i + j] - c * b[j]) % MOD
    return q, r[:m - 1]
  q = _conv(a[::-1][:k], poly_inv(b[::-1], k))[:k][::-1]
  bq = _conv(b, q)
  return q, [(a[i] - bq[i]) % MOD for i in range(m - 1)]

def poly_mod(a, b):
  return a if len(a) < len(b) else poly_divmod(a, b)[1]
```

– 9.5 Multipoint evaluation (subproduct tree)

```
# O(n log^2 n). Values f(x) for x in xs. Small subtrees fall
↪  back to Horner.
def multipoint_eval(f, xs):
  m = len(xs)
  if m == 0:
    return []
  sz = 1
  while sz < m:
    sz <<= 1
  T = [[1]] * (2 * sz)
  for i, x in enumerate(xs):
    T[sz + i] = [-x % MOD, 1]
  for i in range(sz - 1, 0, -1):
    T[i] = _conv(T[2 * i], T[2 * i + 1])
  res = [0] * m
  stack = [(1, poly_mod(f, T[1]), 0, sz)]
  while stack:
    v, r, lo, hi = stack.pop()
    if lo >= m:
      continue
    if hi - lo <= 32 or len(r) <= 32:
      for i in range(lo, min(hi, m)):
        x, acc = xs[i], 0
        for c in reversed(r):
          acc = (acc * x + c) % MOD
        res[i] = acc
      continue
    mid = (lo + hi) >> 1
    stack.append((2 * v, poly_mod(r, T[2 * v]), lo, mid))
    stack.append((2 * v + 1, poly_mod(r, T[2 * v + 1]), mid, hi))
  return res
# (Fast interpolation O(n log^2 n) = same tree + derivative
↪  trick; rarely needed, O(n^2) below.)
```

– 9.5 Lagrange interpolation

```
# O(n^2). Coefficients of the unique poly of deg < n through
↪  (xs[i], ys[i]); xs distinct, mod prime.
def interpolate(xs, ys, mod=MOD):
  n = len(xs)
  P = [1]  # prod (X - xs[i])
  for x in xs:
    P = [(u - x * v) % mod for u, v in zip([0] + P, P + [0])]
  res = [0] * n
  for i in range(n):
    xi, d = xs[i], 1
    for j in range(n):
      if j != i:
        d = d * (xi - xs[j]) % mod
    coef = ys[i] * pow(d, mod - 2, mod) % mod
    acc = 0
    for k in range(n, 0, -1):  # synthetic division P / (X - xi)
      acc = (P[k] + acc * xi) % mod
      res[k - 1] += coef * acc
  return [v % mod for v in res]
# O(n). ys = f(0), ..., f(n) (deg f <= n); returns f(x) mod
↪  prime (prime > n). E.g. sum of k-th powers.
def lagrange_consecutive(ys, x, mod=MOD):
  n = len(ys) - 1
  x %= mod
  pre = [1] * (n + 2)
  suf = [1] * (n + 2)
  for i in range(n + 1):
    pre[i + 1] = pre[i] * (x - i) % mod
  for i in range(n, -1, -1):
    suf[i] = suf[i + 1] * (x - i) % mod
  fact = [1] * (n + 1)
  for i in range(1, n + 1):
    fact[i] = fact[i - 1] * i % mod
  ifact = [1] * (n + 1)
  ifact[n] = pow(fact[n], mod - 2, mod)
  for i in range(n, 0, -1):
    ifact[i - 1] = ifact[i] * i % mod
  res = 0
  for i in range(n + 1):
    t = ys[i] * pre[i] % mod * suf[i + 1] % mod * ifact[i] % mod
↪  * ifact[n - i] % mod
    res += -t if (n - i) & 1 else t
  return res % mod
```

– 9.5 Walsh-Hadamard transform (XOR convolution), AND / OR convolutions

```
# O(n log n), len power of 2, in place. Slice+zip butterflies (fast in CPython).
def fwht_xor(a, invert=False, mod=MOD):
  n, h = len(a), 1
  while h < n:
    for i in range(0, n, 2 * h):
      L, R = a[i:i + h], a[i + h:i + 2 * h]
      a[i:i + h] = [(x + y) % mod for x, y in zip(L, R)]
      a[i + h:i + 2 * h] = [(x - y) % mod for x, y in zip(L, R)]
    h <<= 1
  if invert:
    ni = pow(n, mod - 2, mod)
    a[:] = [x * ni % mod for x in a]

def fwht_or(a, invert=False, mod=MOD):  # zeta over subsets: A[S] = sum_{T subset S} a[T]
  n, h, s = len(a), 1, (-1 if invert else 1)
  while h < n:
    for i in range(0, n, 2 * h):
      a[i + h:i + 2 * h] = [(y + s * x) % mod for x, y in zip(a[i:i + h], a[i + h:i + 2 * h])]
    h <<= 1

def fwht_and(a, invert=False, mod=MOD):  # zeta over supersets
  n, h, s = len(a), 1, (-1 if invert else 1)
  while h < n:
    for i in range(0, n, 2 * h):
      a[i:i + h] = [(x + s * y) % mod for x, y in zip(a[i:i + h], a[i + h:i + 2 * h])]
    h <<= 1

def bit_convolve(a, b, kind, mod=MOD):  # kind in 'xor','or','and'; c[i op j] += a[i]*b[j]
  tr = {'xor': fwht_xor, 'or': fwht_or, 'and': fwht_and}[kind]
  a, b = a[:], b[:]
  tr(a, mod=mod)
  tr(b, mod=mod)
  c = [x * y % mod for x, y in zip(a, b)]
  tr(c, True, mod=mod)
  return c
```

– 9.5 Subset convolution

```
# O(2^n n^2). c[S] = sum_{T subset S} a[T]*b[S\T]. Ranked zeta. CPython: n<=~15 ok; n=20 needs PyPy.
def subset_convolve(a, b, mod=MOD):
  N = len(a)
  n = N.bit_length() - 1
  pc = [0] * N
  for i in range(1, N):
    pc[i] = pc[i >> 1] + (i & 1)
  def ranked(f):
    F = [[0] * N for _ in range(n + 1)]
    for i in range(N):
      F[pc[i]][i] = f[i] % mod
    for row in F:
      fwht_or(row, mod=mod)
    return F
  A, B = ranked(a), ranked(b)
  c = [0] * N
  for k in range(n + 1):
    h = [0] * N
    for i in range(k + 1):
      h = [(z + x * y) for z, x, y in zip(h, A[i], B[k - i])]
    h = [z % mod for z in h]
    fwht_or(h, True, mod=mod)
    for s in range(N):
      if pc[s] == k:
        c[s] = h[s]
  return c
```

– 9.5 Online convolution (CDQ / relaxed)

```
# O(n log^2 n). Computes f[0..n-1] where f[i] = calc(i, s_i), s_i = sum_{j<i} f[j]*g[i-j] (mod),
# g fully known in advance (g[0] unused). E.g. calc = lambda i, s: (h[i] + s) % mod.
# If g also depends on f (e.g. f = 1 + x f^2), use the fully relaxed variant (extra term for l==0 block).
def online_convolve(n, g, calc, mod=MOD):
  g = list(g[:n]) + [0] * (n - len(g))
  f = [0] * n
  acc = [0] * n
  stack = [(0, n, False)]
  while stack:  # iterative CDQ: (l, r, left_done)
    l, r, done = stack.pop()
    if r - l <= 32:
      for i in range(l, r):
        fi = f[i] = calc(i, acc[i] % mod) % mod
        if fi:
          for k in range(i + 1, r):
            acc[k] += fi * g[k - i]
      continue
    m = (l + r) >> 1
    if not done:
      stack.append((l, r, True))
      stack.append((l, m, False))
    else:
      c = convolve(f[l:m], g[1:r - l], mod)
      for i in range(m, r):
        acc[i] += c[i - l - 1]
      stack.append((m, r, False))
  return f
```

– 9.5 Bostan-Mori: [x^k] P(x)/Q(x), and k-th term of linear recurrence

```
# O(d log d log k). Q[0] invertible. Any prime mod (convolve is any-mod).
def bostan_mori(P, Q, k, mod=MOD):
  P = [x % mod for x in P]
  Q = [x % mod for x in Q]
  while k:
    Qm = [c if i % 2 == 0 else (mod - c) % mod for i, c in enumerate(Q)]
    U = convolve(P, Qm, mod)
    V = convolve(Q, Qm, mod)
    P, Q = U[k & 1::2], V[::2]
    k >>= 1
  return P[0] * pow(Q[0], mod - 2, mod) % mod if P else 0
# a[n] = sum_{i<d} c[i]*a[n-1-i], a = first d terms. Returns a[k]. Pair with berlekamp_massey.
def linrec_kth(c, a, k, mod=MOD):
  d = len(c)
  if d == 0:
    return 0
  Q = [1] + [(-x) % mod for x in c]
  P = convolve(a[:d], Q, mod)[:d]
  return bostan_mori(P, Q, k, mod)
```

10 Geometry

```
import math
import random
from bisect import bisect_left, bisect_right, insort
from collections import deque
from functools import cmp_to_key
```

– 10.1 Basics

```
EPS = 1e-9
def sgn(x): return (x > EPS) - (x < -EPS)  # float sign with tolerance
def add(a, b): return (a[0] + b[0], a[1] + b[1])
def sub(a, b): return (a[0] - b[0], a[1] - b[1])
def mul(a, k): return (a[0] * k, a[1] * k)
def dot(a, b): return a[0] * b[0] + a[1] * b[1]
def cross(a, b): return a[0] * b[1] - a[1] * b[0]
def cross3(o, a, b): return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
def norm2(a): return a[0] * a[0] + a[1] * a[1]
def norm(a): return math.hypot(a[0], a[1])
def dist2(a, b): return norm2(sub(a, b))
def dist(a, b): return math.hypot(a[0] - b[0], a[1] - b[1])
def perp(a): return (-a[1], a[0])  # rotate +90 deg (exact)
def rotate(a, t):  # rotate CCW by t radians
  c, s = math.cos(t), math.sin(t)
  return (a[0] * c - a[1] * s, a[0] * s + a[1] * c)
def orient(a, b, c):  # +1 ccw (left turn), -1 cw, 0 collinear (exact for ints)
  v = cross3(a, b, c)
  return (v > 0) - (v < 0)

def dist_point_line(p, a, b):  # distance p to infinite line ab (a != b)
  return abs(cross3(a, b, p)) / dist(a, b)

def dist_point_seg(p, a, b):  # distance p to segment ab
  d = sub(b, a)
  if d == (0, 0): return dist(p, a)
  t = dot(sub(p, a), d)
  if t <= 0: return dist(p, a)
  if t >= norm2(d): return dist(p, b)
  return abs(cross(d, sub(p, a))) / norm(d)
# Polar sort by angle in [0, 2pi) around origin, exact (no atan2). Points != (0,0).
# Same direction: tie (here by distance). Sort around c: key on sub(p, c).
def _half(p): return 0 if (p[1] > 0 or (p[1] == 0 and p[0] > 0)) else 1
def _polar_cmp(a, b):
  ha, hb = _half(a), _half(b)
  if ha != hb: return ha - hb
  c = cross(a, b)
  if c: return -1 if c > 0 else 1
  return norm2(a) - norm2(b)
def polar_sort(pts): return sorted(pts, key=cmp_to_key(_polar_cmp))
```

– 10.2 Lines and Segments

```
# Line forms: two points (a, b) [used here]; point+direction (p, d); ax+by=c with
# a=y2-y1, b=x1-x2, c=a*x1+b*y1 (integer-friendly; normalize by gcd & sign to hash lines);
# slope/intercept -- avoid (vertical lines, precision).

def line_inter(a, b, c, d):  # lines ab, cd -> point or None if parallel
  den = cross(sub(b, a), sub(d, c))
  if den == 0: return None
  t = cross(sub(c, a), sub(d, c)) / den  # Fraction inputs -> exact
  return (a[0] + (b[0] - a[0]) * t, a[1] + (b[1] - a[1]) * t)

def on_segment(p, a, b):  # p on closed segment ab (exact for ints)
  return cross3(a, b, p) == 0 and min(a[0], b[0]) <= p[0] <= max(a[0], b[0]) \
    and min(a[1], b[1]) <= p[1] <= max(a[1], b[1])

def seg_intersect(a, b, c, d):  # closed segments ab, cd share a point? exact; handles collinear/degenerate
  d1, d2 = orient(a, b, c), orient(a, b, d)
  d3, d4 = orient(c, d, a), orient(c, d, b)
  if d1 * d2 < 0 and d3 * d4 < 0: return True
  return (d1 == 0 and on_segment(c, a, b)) or (d2 == 0 and on_segment(d, a, b)) or \
     (d3 == 0 and on_segment(a, c, d)) or (d4 == 0 and on_segment(b, c, d))

def seg_inter_points(a, b, c, d):  # [] none, [p] single point, [p, q] overlap segment (p<q)
  if not seg_intersect(a, b, c, d): return []
  if cross(sub(b, a), sub(d, c)) != 0: return [line_inter(a, b, c, d)]
  s = sorted(set(p for p in (a, b) if on_segment(p, c, d)) |
             set(p for p in (c, d) if on_segment(p, a, b)))
  return [s[0]] if len(s) == 1 else [s[0], s[-1]]
# Bentley-Ottmann (report all k intersections, O((n+k) log n)): sweep x, status = BST of
# segments ordered by y at sweep x, event queue = endpoints + found intersections; check only
# neighbours on insert/delete/swap. Painful in Python (needs ordered BST with changing key) --
# in contests usually n small -> O(n^2) pairwise seg_intersect.
# Shamos-Hoey ("does ANY pair intersect?", O(n log n)): same sweep without intersection events;
# on insert check new seg vs its above/below neighbours, on delete check the two neighbours
# that become adjacent; stop at first hit. Status order compares y at current x (exact via cross).
```

– 10.3 Polygons

```
def area2(P):  # 2 * signed area (ccw > 0), exact integer
  s = 0
  for i in range(len(P)):
    x1, y1 = P[i - 1]; x2, y2 = P[i]
    s += x1 * y2 - x2 * y1
  return s

def boundary_points(P):  # lattice points on boundary of integer polygon
  return sum(math.gcd(abs(P[i][0] - P[i - 1][0]), abs(P[i][1] - P[i - 1][1])) for i in range(len(P)))

def pick_interior(P):  # Pick: A = I + B/2 - 1  ->  I = (2A - B + 2) / 2
  return (abs(area2(P)) - boundary_points(P) + 2) // 2

def point_in_polygon(P, q):  # any simple polygon: 1 inside, 0 boundary, -1 outside. O(n), exact
  wn = 0
  n = len(P)
  for i in range(n):
    a, b = P[i], P[(i + 1) % n]
    if on_segment(q, a, b): return 0
    if a[1] <= q[1]:
      if b[1] > q[1] and cross3(a, b, q) > 0: wn += 1
    elif b[1] <= q[1] and cross3(a, b, q) < 0: wn -= 1
  return 1 if wn else -1  # winding number (also works for self-intersecting, nonzero rule)

def point_in_convex(P, q):  # P ccw strictly convex (n>=3): 1/0/-1, O(log n)
  n, a = len(P), P[0]
  if cross3(a, P[1], q) < 0 or cross3(a, P[-1], q) > 0: return -1
  lo, hi = 1, n - 1
  while hi - lo > 1:
    m = (lo + hi) // 2
    if cross3(a, P[m], q) >= 0: lo = m
    else: hi = m
  c = cross3(P[lo], P[lo + 1], q)
  if c < 0: return -1
  if c == 0: return 0
  if (lo == 1 and cross3(a, P[1], q) == 0) or (lo == n - 2 and cross3(a, P[-1], q) == 0): return 0
  return 1

def convex_cut(P, a, b):  # part of convex polygon P left of (on) directed line a->b. O(n)
  res, n = [], len(P)
  for i in range(n):
    p, q = P[i], P[(i + 1) % n]
    cp, cq = cross3(a, b, p), cross3(a, b, q)
    if cp >= 0: res.append(p)
    if cp * cq < 0: res.append(line_inter(p, q, a, b))
  return res  # may contain duplicate/collinear pts in degenerate cases
# Sutherland-Hodgman: clip polygon S (any) by CONVEX clipper C = apply convex_cut(S, C[i], C[i+1])
# for each ccw edge of C. O(|S||C|). Result for concave S may have zero-width "bridges".
# Polygon triangulation (conceptual): ear clipping O(n^2): repeatedly find convex vertex i whose
# triangle (i-1,i,i+1) contains no other vertex, cut it. Fan triangulation for convex polygons.
# O(n log n): split into monotone pieces (sweep) then triangulate each in O(n).
```

– 10.4 Convex Hull

```
def convex_hull(pts, keep_collinear=False):  # Andrew monotone chain O(n log n); ccw, starts at min (x,y)
  pts = sorted(set(pts))  # dedupe!
  if len(pts) <= 2: return pts
  def chain(seq):
    h = []
    for p in seq:
      while len(h) >= 2:
        c = cross3(h[-2], h[-1], p)
        if c < 0 or (c == 0 and not keep_collinear): h.pop()
        else: break
      h.append(p)
    return h
  lo, up = chain(pts), chain(reversed(pts))
  if keep_collinear and len(lo) == len(pts): return lo  # all collinear
  return lo[:-1] + up[:-1]  # strict mode: all collinear -> 2 endpoints
# Graham scan: pick lowest point, polar_sort others around it (ties by distance), stack-pop on
# non-left turns. Same O(n log n); monotone chain is simpler (no angle sort).

def hull_diameter2(H):  # rotating calipers: max squared dist, H strict ccw hull. O(n)
  n = len(H)
  if n == 1: return 0
  if n == 2: return dist2(H[0], H[1])
  best, j = 0, 1
  for i in range(n):
    ni = (i + 1) % n
    while cross(sub(H[ni], H[i]), sub(H[(j + 1) % n], H[j])) > 0: j = (j + 1) % n
    best = max(best, dist2(H[i], H[j]), dist2(H[ni], H[j]))
  return best  # farthest pair: record (i,j) at the max

def hull_width(H):  # min width (min over edges of max dist), H strict ccw, n>=3. O(n)
  n, j, best = len(H), 1, float('inf')
  for i in range(n):
```

```
    a, b = H[i], H[(i + 1) % n]
    while cross3(a, b, H[(j + 1) % n]) > cross3(a, b, H[j]): j =
    ↪  (j + 1) % n
    best = min(best, cross3(a, b, H[j]) / dist(a, b))
  return best

def minkowski(P, Q):  # convex ccw P, Q -> ccw P+Q, O(n+m) (may
↪  keep collinear pts)
  def reorder(R):
    k = min(range(len(R)), key=lambda t: (R[t][1], R[t][0]))
    R = R[k:] + R[:k]
    return R + R[:2]
  P, Q = reorder(P), reorder(Q)
  res, i, j = [], 0, 0
  while i < len(P) - 2 or j < len(Q) - 2:
    res.append(add(P[i], Q[j]))
    c = cross(sub(P[i + 1], P[i]), sub(Q[j + 1], Q[j]))
    if c >= 0 and i < len(P) - 2: i += 1
    if c <= 0 and j < len(Q) - 2: j += 1
  return res or [add(P[0], Q[0])]
# Uses: collision of convex P and Q <=> origin in P + (-Q);
↪  distance between polygons = dist(0, P-Q).
# Dynamic convex hull (online point insertion): keep upper &
↪  lower hulls separately in a
# sorted container keyed by x (SortedList / treap). Insert p:
↪  if under upper hull (check
# neighbours) skip; else insert and pop neighbours on both
↪  sides while they make non-right turns.
# Amortized O(log n). For max-of-lines queries (CHT) use Li
↪  Chao tree (02_data_structures.py).

def half_plane_intersection(H):  # H: list of (p, q) keep LEFT
↪  of p->q. O(n log n)
  # Include a big bounding box if the region may be unbounded.
  ↪  Returns ccw vertices (floats), [] if empty.
  # Integer p, q -> exact angle sort/parallel tests; Fraction
  ↪  coords -> fully exact (set EPS=0 semantics).
  def dcmp(h1, h2):
    a, b = sub(h1[1], h1[0]), sub(h2[1], h2[0])
    ha, hb = _half(a), _half(b)
    if ha != hb: return ha - hb
    c = cross(a, b)
    return -1 if c > 0 else (1 if c < 0 else 0)
  H = sorted(H, key=cmp_to_key(dcmp))
  def out(h, r): return cross(sub(h[1], h[0]), sub(r, h[0])) <
  ↪  -EPS
  def inter(h1, h2): return line_inter(h1[0], h1[1], h2[0],
  ↪  h2[1])
  dq = deque()
  for h in H:
    while len(dq) > 1 and out(h, inter(dq[-1], dq[-2])):
    ↪  dq.pop()
    while len(dq) > 1 and out(h, inter(dq[0], dq[1])):
    ↪  dq.popleft()
    if dq:
      d1, d2 = sub(h[1], h[0]), sub(dq[-1][1], dq[-1][0])
      if cross(d1, d2) == 0:
        if dot(d1, d2) < 0: return []
        if out(h, dq[-1][0]): dq.pop()
        else: continue
    dq.append(h)
  while len(dq) > 2 and out(dq[0], inter(dq[-1], dq[-2])):
  ↪  dq.pop()
  while len(dq) > 2 and out(dq[-1], inter(dq[0], dq[1])):
  ↪  dq.popleft()
  if len(dq) < 3: return []
  res = []
  for i in range(len(dq)):
    p = inter(dq[i], dq[(i + 1) % len(dq)])
    if p is None: return []
    res.append(p)
  return res

def convex_hull_3d(P):  # P: list of int (x,y,z). Incremental
↪  O(n^2), exact.
  # Returns faces as index triples (i,j,k), ccw seen from
  ↪  outside (outward normal). Coplanar
  # hull faces come out triangulated. Returns [] if all points
  ↪  coplanar (handle as 2D).
  def sub3(a, b): return (a[0] - b[0], a[1] - b[1], a[2] - b[2])
  def vol(a, b, c, d):  # 6 * signed volume; > 0: d above plane
  ↪  abc (ccw)
    u, v, w = sub3(P[b], P[a]), sub3(P[c], P[a]), sub3(P[d],
↪  P[a])
    return (u[1] * v[2] - u[2] * v[1]) * w[0] + (u[2] * v[0] -
    ↪  u[0] * v[2]) * w[1] + (u[0] * v[1] - u[1] * v[0]) * w[2]
  n = len(P)
  i0 = 0
  i1 = next((i for i in range(n) if P[i] != P[i0]), None)
  if i1 is None: return []
  def colin(i):
    u, v = sub3(P[i1], P[i0]), sub3(P[i], P[i0])
    return u[1] * v[2] - u[2] * v[1] == 0 and u[2] * v[0] - u[0]
    ↪  * v[2] == 0 and u[0] * v[1] - u[1] * v[0] == 0
  i2 = next((i for i in range(n) if not colin(i)), None)
  if i2 is None: return []
  i3 = next((i for i in range(n) if vol(i0, i1, i2, i) != 0),
↪  None)
  if i3 is None: return []
  if vol(i0, i1, i2, i3) > 0: i1, i2 = i2, i1  # make i3 below
  ↪  face (i0,i1,i2)
  faces = [(i0, i1, i2), (i0, i3, i1), (i1, i3, i2), (i2, i3,
↪  i0)]
  used = {i0, i1, i2, i3}
  for p in range(n):
    if p in used: continue
    vis = [vol(a, b, c, p) > 0 for (a, b, c) in faces]
    if not any(vis): continue  # inside or on hull
    edges = set()
    for f, v in zip(faces, vis):
      if v: edges.update(((f[0], f[1]), (f[1], f[2]), (f[2],
      ↪  f[0])))
    nf = [f for f, v in zip(faces, vis) if not v]
    for (a, b) in edges:
      if (b, a) not in edges: nf.append((a, b, p))  # horizon
      ↪  edge
    faces = nf
  return faces
# Volume = sum over faces of dot(Pa, cross(Pb, Pc)) / 6 ;
↪  surface area = sum |cross(Pb-Pa, Pc-Pa)|/2.
# ==== 10.5 Circles ====  (circle = center tuple c, radius r;
↪  floats)
def circle_line(c, r, a, b):  # intersections of circle with
↪  line ab: 0/1/2 points
  d = sub(b, a)
  t = dot(sub(c, a), d) / norm2(d)
  h = (a[0] + d[0] * t, a[1] + d[1] * t)  # foot of perpendicular
  s = r * r - dist2(c, h)
  if s < -EPS: return []
  if s <= EPS: return [h]
  k = math.sqrt(s / norm2(d))
  return [(h[0] - d[0] * k, h[1] - d[1] * k), (h[0] + d[0] * k,
  ↪  h[1] + d[1] * k)]

def circle_circle(c1, r1, c2, r2):  # 0/1/2 points; identical
↪  circles -> [] (infinite)
  d2 = dist2(c1, c2)
  if d2 == 0: return []
  d = math.sqrt(d2)
  if d > r1 + r2 + EPS or d < abs(r1 - r2) - EPS: return []
  a = (d2 + r1 * r1 - r2 * r2) / (2 * d)
  h = math.sqrt(max(0.0, r1 * r1 - a * a))
  v = ((c2[0] - c1[0]) / d, (c2[1] - c1[1]) / d)
  m = (c1[0] + v[0] * a, c1[1] + v[1] * a)
  if h <= EPS: return [m]
  return [(m[0] - v[1] * h, m[1] + v[0] * h), (m[0] + v[1] * h,
  ↪  m[1] - v[0] * h)]

def tangents_point(c, r, p):  # tangent points on circle from
↪  external point p
  d2 = dist2(c, p)
  if d2 < r * r - EPS: return []
  if d2 <= r * r + EPS: return [p]
  return circle_circle(c, r, p, math.sqrt(d2 - r * r))

def tangents_circles(c1, r1, c2, r2):  # external tangents;
↪  call with -r2 for internal ones
  # returns list of (p1, p2): tangent line touches circle1 at
  ↪  p1, circle2 at p2. r2=0 -> point tangents.
  d = sub(c2, c1)
  dr, d2 = r1 - r2, norm2(d)
  h2 = d2 - dr * dr
  if d2 == 0 or h2 < 0: return []
  out = []
  for s in (-1, 1):
    k = math.sqrt(h2) * s
    v = ((d[0] * dr - d[1] * k) / d2, (d[1] * dr + d[0] * k) / d2)
    out.append(((c1[0] + v[0] * r1, c1[1] + v[1] * r1), (c2[0] +
↪  v[0] * r2, c2[1] + v[1] * r2)))
  if h2 == 0: out.pop()
  return out

def circumcircle(a, b, c):  # center, radius of circle through
↪  3 pts (None if collinear)
  bx, by = b[0] - a[0], b[1] - a[1]
  cx, cy = c[0] - a[0], c[1] - a[1]
  d = 2 * (bx * cy - by * cx)
  if d == 0: return None
  B, C = bx * bx + by * by, cx * cx + cy * cy
  o = (a[0] + (cy * B - by * C) / d, a[1] + (bx * C - cx * B) / d)
  return o, dist(o, a)

def min_enclosing_circle(pts):  # Welzl (iterative, shuffled)
↪  expected O(n) -> (center, r)
  P = [(float(x), float(y)) for x, y in pts]
  random.shuffle(P)
  c, r = P[0], 0.0
  def outside(p): return dist(p, c) > r + 1e-9 * max(1.0, r)
  for i in range(len(P)):
    if not outside(P[i]): continue
    c, r = P[i], 0.0
    for j in range(i):
      if not outside(P[j]): continue
      c = ((P[i][0] + P[j][0]) / 2, (P[i][1] + P[j][1]) / 2); r
↪  = dist(c, P[i])
      for k in range(j):
        if not outside(P[k]): continue
        cc = circumcircle(P[i], P[j], P[k])
        if cc: c, r = cc
  return c, r

def circle_union_area(cs):  # cs: list of (cx, cy, r). Green's
↪  theorem O(n^2 log n)
  cs = list(set(cs))
  keep = []
  for i, (x, y, r) in enumerate(cs):  # drop circles inside
  ↪  another
    if r <= 0: continue
    if not any(j != i and rj >= r and math.hypot(x - xj, y - yj)
    ↪  + r <= rj + EPS
               for j, (xj, yj, rj) in enumerate(cs)): keep.append((x,
               ↪  y, r))
  total = 0.0
  for i, (x, y, r) in enumerate(keep):
    iv = []  # covered angle intervals
    for j, (xj, yj, rj) in enumerate(keep):
      if i == j: continue
      d = math.hypot(xj - x, yj - y)
      if d >= r + rj: continue
      a = math.atan2(yj - y, xj - x)
      h = math.acos(max(-1.0, min(1.0, (r * r + d * d - rj * rj)
↪  / (2 * r * d))))
      lo, hi = a - h, a + h
      if lo < -math.pi: iv += [(lo + 2 * math.pi, math.pi),
      ↪  (-math.pi, hi)]
      elif hi > math.pi: iv += [(lo, math.pi), (-math.pi, hi - 2
      ↪  * math.pi)]
      else: iv.append((lo, hi))
    iv.sort()
    F = lambda t: r * r * t + x * r * math.sin(t) - y * r *
↪  math.cos(t)
    cur = -math.pi
    for lo, hi in iv + [(math.pi, math.pi)]:
      if lo > cur: total += F(lo) - F(cur)  # uncovered arc
      ↪  [cur, lo]
      cur = max(cur, hi)
  return total / 2
# Alternative: Simpson integration over x of union length of
↪  y-intervals, splitting at all
# critical x (cx +- r, pairwise intersection x's) -- see brute
↪  in tests.
```

– 10.6 Advanced Geometry

```
# ---- closest pair (3.7 / 10.6) ----
def closest_pair(pts):  # int pts, n>=2 -> (d2, p, q). Sweep by
↪  x + y-sorted strip. ~O(n log n)
  P = sorted(pts)
  best, bp = float('inf'), None
  strip, left = [], 0  # strip: sorted (y, x) of pts with x
↪  within sqrt(best)
  for p in P:
    while left < len(P) and (p[0] - P[left][0]) ** 2 > best:
      q = P[left]; strip.pop(bisect_left(strip, (q[1], q[0])));
↪  left += 1
    d = math.isqrt(best) if best != float('inf') else 1 << 62  #
↪  floats: use math.sqrt
    lo = bisect_left(strip, (p[1] - d, -float('inf')))
    hi = bisect_right(strip, (p[1] + d, float('inf')))
    for k in range(lo, hi):
      y, x = strip[k]
      dd = (x - p[0]) ** 2 + (y - p[1]) ** 2
      if dd < best:
        best, bp = dd, ((x, y), p)
        if best == 0: return 0, bp[0], bp[1]
    insort(strip, (p[1], p[0]))
  return best, bp[0], bp[1]
# D&C alternative: split by median x, recurse, merge by y,
↪  check strip neighbours (<=7 each).
# Sweep line techniques (notes): sort events by x (ties: insert
↪  before query before delete, or
# as problem demands) and maintain active set in a balanced
↪  structure / segment tree:
#   union area of rectangles: sweep x, segment tree over
↪  compressed y with cover count + covered length.
#   segment intersection: see Bentley-Ottmann / Shamos-Hoey
↪  above. closest pair: above.
#   max overlapping intervals / points covered by most circles:
↪  angular or linear events.
# Voronoi / Delaunay (notes): Delaunay = dual of Voronoi; no
↪  point inside any triangle's circumcircle.
#   naive O(n^4): triangle ijk is Delaunay iff no l inside
↪  circumcircle (exact in_circle det below).
#   O(n^2): 3D lift (x, y, x^2+y^2) -> lower hull of
↪  convex_hull_3d = Delaunay triangles
#     (faces with outward normal z < 0). Bowyer-Watson O(n^2)
↪  worst: insert point, delete triangles
#   whose circumcircle contains it, re-triangulate cavity.
↪  Fortune sweep O(n log n) - rarely coded.
#   EMST is a subset of Delaunay edges. Voronoi cell of i =
↪  half_plane_intersection of bisectors.
#   in_circle(a,b,c,d) (a,b,c ccw) > 0 iff d inside: det of rows
↪  (ax-dx, ay-dy, (ax-dx)^2+(ay-dy)^2), ...
# Point location (notes): offline -> sweep x with sorted-by-y
↪  active edges; online -> slab
#   decomposition (O(n^2) memory) or persistent BST; convex
↪  polygon -> point_in_convex.
# ---- Manhattan MST ----
def manhattan_mst(pts):  # O(n log n): 4 octant sweeps +
↪  Fenwick(min) + Kruskal -> (weight, edges)
  n = len(pts)
  cand = []
  for tf in (lambda x, y: (x, y), lambda x, y: (y, x), lambda x,
  ↪  y: (-x, y), lambda x, y: (y, -x)):
    Q = [tf(x, y) for x, y in pts]
    # for each i find j with xj >= xi and yj - xj >= yi - xi
    ↪  minimizing xj + yj
    keys = sorted(set(y - x for x, y in Q))
    m = len(keys)
    bit = [(float('inf'), -1)] * (m + 1)  # prefix-min over
↪  reversed key index
    for i in sorted(range(n), key=lambda t: (-Q[t][0],
    ↪  -Q[t][1])):
      x, y = Q[i]
      k = m - bisect_left(keys, y - x)  # reversed 1-based: keys
↪  >= y-x  <->  idx <= k
      j, best = k, (float('inf'), -1)
      while j > 0:
        if bit[j] < best: best = bit[j]
        j -= j & -j
      if best[1] >= 0: cand.append((best[0] - x - y, i, best[1]))
      j, v = k, (x + y, i)
      while j <= m:
        if v < bit[j]: bit[j] = v
        j += j & -j
  cand.sort()
  par = list(range(n))
  def find(a):
    while par[a] != a:
      par[a] = par[par[a]]; a = par[a]
    return a
  total, edges = 0, []
  for w, a, b in cand:
    ra, rb = find(a), find(b)
    if ra != rb:
      par[ra] = rb; total += w; edges.append((a, b, w))
  return total, edges
# ---- k-d tree (2D nearest neighbour) ----
class KDTree:  # build O(n log^2 n); query ~O(log n) avg (bad
↪  worst case)
  def __init__(self, pts):
    self.a = list(pts)
    st = [(0, len(self.a), 0)]
    while st:
      lo, hi, d = st.pop()
```

```
      if hi - lo <= 1: continue
      self.a[lo:hi] = sorted(self.a[lo:hi], key=lambda p: p[d])
      m = (lo + hi) // 2
      st.append((lo, m, d ^ 1)); st.append((m + 1, hi, d ^ 1))
  def nearest(self, q, skip_equal=False):  # -> (d2, point);
  ↪ skip_equal: ignore points == q
    best, bp = float('inf'), None
    st = [(0, len(self.a), 0, 0)]  # (lo, hi, axis, lower bound
↪ d2)
    while st:
      lo, hi, d, lb = st.pop()
      if lo >= hi or lb >= best: continue
      m = (lo + hi) // 2
      p = self.a[m]
      dd = (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2
      if dd < best and not (skip_equal and p == q): best, bp =
      ↪ dd, p
      diff = q[d] - p[d]
      near, far = ((lo, m), (m + 1, hi)) if diff < 0 else ((m +
↪ 1, hi), (lo, m))
      st.append((far[0], far[1], d ^ 1, diff * diff))
      st.append((near[0], near[1], d ^ 1, 0))
    return best, bp
# Lattice point counting (notes): points on segment =
↪ gcd(|dx|,|dy|)+1; inside polygon = Pick
#   (pick_interior). Under a line y = (a*x+b)/c for x in [0,n):
↪ floor_sum (08_math_nt_comb.py).
#   Points in circle x^2+y^2<=r^2: sum over x of
↪ 2*isqrt(r^2-x^2)+1.
# Rotational sweep (notes): around a pivot, polar_sort other
↪ points (or events = angles where
#   something enters/leaves), sweep angle maintaining count;
↪ e.g. max points covered by a
#   half-plane/line through pivot, or for all pairs sort O(n^2)
↪ directions and keep points sorted
#   by projection (swap adjacent pair at each direction) ->
↪ O(n^2 log n) for min-area triangle etc.
# Geometric duality (notes): point (a, b) <-> line y = a*x - b.
↪ Preserves incidence & above/below
#   (point above line <=> dual line below dual point). Upper
↪ hull of points <-> lower envelope of
#   lines (CHT). Problems about lines through points become
↪ arrangements of lines.
```

Cheat sheet

Python equivalents of C++ language essentials (2.1)

- **Integers**: arbitrary precision, so there is no overflow and no `__int128`. Overflow bugs become *speed* bugs: reduce `% MOD` early so numbers stay small. To emulate unsigned 64-bit: `x & ((1 << 64) - 1)`.
- **Floats**: `float` is a C double. For more precision use `fractions.Fraction` (exact) or `decimal.Decimal` with `getcontext().prec = 50`. Use `math.isclose`. Prefer `math.isqrt(n)` over `int(n**0.5)` (the latter is wrong for n > 2^52).
- **Bit builtins**: `x.bit_count()` (popcount, 3.10+), `x.bit_length()` (= 64 - clz; `__lg(x) = x.bit_length()-1`), `(x & -x).bit_length()-1` (ctz).
- **Lambdas / recursive lambdas**: nested `def` closures. `nonlocal` for mutation. Use `functools.cache` for memoization.
- **Template**: see `00_core.py`. No macros or pragmas; the equivalent of `-O2` is choosing PyPy.
- **C++-only debug tools** (sanitizers, `_GLIBCXX_DEBUG`, gdb) map to: Python already bounds-checks lists (watch out: negative indices wrap silently!), `python3 -X dev`, `faulthandler`, `pdb` / `breakpoint()`.

– **3. Theorems & facts cheat sheet (no code needed)**

- **Master theorem**: T(n)=aT(n/b)+O(n^d). If d > log_b a -> O(n^d). If d = log_b a -> O(n^d log n). Otherwise O(n^{log_b a}).
- **Matroid / greedy**: greedy by weight is optimal for every weight function <=> the feasible sets form a matroid (graphic -> Kruskal, partition, linear -> XOR basis).
- **Exchange argument**: assume an optimal order. Swapping an adjacent pair that violates your order doesn't make it worse. So sorted order is optimal.
- **Konig**: in a bipartite graph, max matching = min vertex cover. Max independent set = V - max matching.
- **Hall**: a perfect matching of side L exists <=> |N(S)| >= |S| for every S subset of L.
- **Dilworth**: min chains covering a poset = max antichain. Min path cover of a DAG = V - max matching (split graph).
- **Max-flow = min-cut**. Edge connectivity = min over t of maxflow(s,t). Vertex connectivity needs vertex splitting.
- **Euler path**: undirected needs 0 or 2 odd vertices. Directed needs in=out everywhere except start (+1 out) and end (+1 in). The graph must also be connected.
- **Euler's formula (planar)**: V - E + F = 1 + C. Simple planar => E <= 3V - 6 (V >= 3).
- **Tournament**: always has a Hamiltonian path. Strongly connected => Hamiltonian cycle.
- **Cayley**: n^{n-2} labeled trees. A forest of k trees rooted at given k vertices: k*n^{n-k-1}.
- **Matrix-tree**: #spanning trees = any cofactor of the Laplacian.
- **Pick**: A = I + B/2 - 1.
- **Sprague-Grundy**: game sum => XOR of Grundy values. Position is losing <=> value 0.
- **Burnside**: #orbits = (1/|G|) Sum_g |Fix(g)|.
- **Lucas**: C(n,k) mod p = Prod C(n_i, k_i) over base-p digits. **Kummer**: the exponent of p in C(m+n, m) equals the number of carries when adding m and n in base p.
- **Wilson**: (p-1)! == -1 (mod p). **Fermat**: a^{p-1} == 1. **Euler**: a^phi(m) == 1 when gcd(a,m)=1. Generalized: a^k == a^{k mod phi + phi} for k >= log2 m.
- **Erdos-Szekeres / LIS duality**: min number of non-increasing subsequences covering the sequence = LIS length.
- **Linearity of expectation** holds without independence. **Geometric distribution**: with success prob p, expected trials = 1/p.
- **Catalan** C_n = C(2n,n)/(n+1): bracket sequences, binary trees, triangulations, monotone lattice paths under the diagonal.
- Number of divisors of n <= 10^18 is at most 103680. For n <= 10^9 it's at most 1344.
- Prime gaps below 10^18 are at most ~1500. pi(10^6)=78498, pi(10^7)=664579.
- Useful primes: 998244353 = 119*2^23+1 (g=3), 10^9+7, 2^61-1 (hashing).